

GX Series

GX-PLC 매뉴얼



CSCAM

문서번호 : **CS-GX245-PL-26-01**

Contents

Contents	3
1 GX Series PLC 프로그램	1-1
1.1 GX Series의 PLC 프로그램	1-2
1.1.1 GX Series PLC 프로그램의 특징	1-2
1.1.2 성능 사양	1-4
1.2 PLC 프로그램 구성	1-5
1.2.1 PLC 실행 파일	1-5
1.2.2 데이터 메모리	1-7
1.3 프로그램 실행 흐름	1-8
2 PLC 명령	2-1
2.1 PLC 명령	2-2
2.1.1 일반 시퀀스 명령	2-2
2.1.2 내부 연산 처리 명령	2-3
2.1.3 기능 명령	2-4
2.2 명령 구성	2-5
2.2.1 명령 구성	2-5
2.2.2 레지스터 표시 방법	2-6
2.3 PLC 명령 목록	2-7
3 PLC 프로그램 편집기	3-1
3.1 GX Series PLC 프로그램 편집기	3-2
3.1.1 PLC 에디터 특징	3-2
3.1.2 기능 사양	3-3
3.2 GX PLC 프로그램 화면 구성	3-4
3.2.1 전체 화면 구성	3-4
3.2.2 Menu Bar 주요 항목	3-5
3.2.3 Tool Bar 주요 항목	3-7
3.3 PLC 프로그램 작성	3-9
4 일반 명령	4-1
4.1 입출력 명령	4-2
4.1.1 RD / RDN (Read / Read Not)	4-2
4.1.2 RDUP (입력 상승 미분)	4-3
4.1.3 RDDN (입력 하강 미분)	4-4
4.1.4 WR / WRN (Write / Write Not)	4-5
4.1.5 WRUP (출력 상승 미분)	4-6

4.1.6	WRDN (출력 하강 미분)	4-7
4.1.7	SET (Set Coil).....	4-8
4.1.8	RESET (Reset Coil)	4-9
4.1.9	KEEP (자기 유지).....	4-10
4.2	릴레이 회로 스택 연산 명령	4-12
4.2.1	AND	4-12
4.2.2	ANDN	4-12
4.2.3	ANDUP	4-12
4.2.4	ANDDN	4-13
4.2.5	OR.....	4-13
4.2.6	ORN	4-13
4.2.7	ORUP.....	4-14
4.2.8	ORDN	4-14
4.2.9	RDS.....	4-15
4.2.10	RDNS.....	4-15
4.2.11	RDUPS.....	4-16
4.2.12	RDDNS	4-16
4.2.13	ANDS.....	4-17
4.2.14	ORS.....	4-17
4.3	수치 스택 연산 명령	4-18
4.3.1	NMRD.....	4-18
4.3.2	NMRDS	4-18
4.3.3	NMADD.....	4-18
4.3.4	NMSUB	4-19
4.3.5	NMMUL.....	4-19
4.3.6	NMDIV.....	4-19
4.3.7	NMADDs.....	4-20
4.3.8	NMSUBS	4-20
4.3.9	NMMULS.....	4-20
4.3.10	NMDIVS.....	4-21
4.3.11	NMWR	4-21
5	기능 명령.....	5-1
5.1	프로그램 제어 명령	5-2
5.1.1	SCALL (서브 프로그램 호출)	5-2
5.1.2	PCALL (모션 프로그램 호출)	5-4
5.1.3	IFON/ IFOFF (조건문).....	5-5

5.1.4	ELSE (조건문).....	5-7
5.1.5	IEND (조건문)	5-8
5.1.6	FOR (반복문).....	5-9
5.1.7	FEND (반복문).....	5-11
5.1.8	WHILEON/ WHILEOFF (반복문)	5-12
5.1.9	WEND (반복문).....	5-14
5.1.10	BREAK (BREAK 문)	5-15
5.1.11	GOTO (점프 명령)	5-16
5.1.12	LABEL (라벨 표시).....	5-17
5.1.13	PEND (프로그램 끝)	5-18
5.1.14	STOP (프로그램 정지)	5-19
5.1.15	SWITCH (SWITCH-CASE문)	5-20
5.1.16	CASE (SWITCH-CASE문).....	5-22
5.1.17	DEFAULT (SWITCH-CASE문)	5-23
5.1.18	SWEND (SWITCH-CASE문)	5-24
5.2	논리 연산 명령	5-25
5.2.1	ANDMOV (논리곱 이동)	5-25
5.2.2	ORMOV (논리합 이동)	5-27
5.2.3	XORMOV (배타적 논리합 이동).....	5-29
5.3	수치 연산 명령	5-31
5.3.1	ADD (더하기)	5-31
5.3.2	SUB (빼기)	5-33
5.3.3	MUL (곱하기)	5-35
5.3.4	DIV (나누기)	5-37
5.3.5	MOD (나머지).....	5-39
5.3.6	INC (INCREMENT 명령)	5-40
5.3.7	DEC (DECREMENT 명령).....	5-41
5.3.8	TIMEADD (시간 가산).....	5-42
5.3.9	TIMESUB (시간 감산)	5-44
5.4	수치 비교 명령	5-46
5.4.1	LT (Less Than).....	5-46
5.4.2	LE (Less Than / Equal)	5-48
5.4.3	GT (Greater Than)	5-50
5.4.4	GE (Greater Than / Equal)	5-52
5.4.5	EQ (Equal).....	5-54
5.4.6	EQ2 (Equal 2).....	5-56

5.4.7	NE (Not Equal)	5-58
5.4.8	RNG (범위 체크)	5-60
5.5	데이터 변환 명령	5-62
5.5.1	INV (부호 반전)	5-62
5.5.2	INVB (비트 반전)	5-63
5.5.3	COM (2의 보수 변환)	5-65
5.5.4	ABSD (절대치 변환)	5-66
5.5.5	BCD (BIN⇒BCD 변환)	5-67
5.5.6	BIN (BCD⇒BIN 변환)	5-68
5.5.7	WTOF (WORD⇒FLOAT 변환)	5-69
5.5.8	FTOW (FLOAT⇒WORD 변환)	5-71
5.5.9	BITRTOC (비트행⇒비트열 변환)	5-73
5.5.10	BITCTOR (비트열⇒비트행 변환)	5-75
5.5.11	PARITY (패리티 변환)	5-77
5.5.12	BINTOASC (BIN⇒ASC 변환)	5-78
5.5.13	ASCTOBIN (ASC⇒BIN 변환)	5-80
5.6	데이터 전송 명령	5-82
5.6.1	MOVB (비트 복사)	5-82
5.6.2	MOVD (레지스터 복사)	5-84
5.6.3	CHGB (비트 교환, Change Bit)	5-86
5.6.4	CHGD (레지스터 교환, Change Register)	5-88
5.6.5	SWAP (Byte Swap)	5-90
5.6.6	SETD (레지스터 초기화)	5-91
5.6.7	SETD2 (레지스터 초기화 2)	5-93
5.6.8	SETB (Set Bit Data)	5-95
5.6.9	SETB2 (Set Bit Data)	5-97
5.6.10	GETB (Get Bit Data)	5-99
5.6.11	SRCH (Search Data)	5-101
5.6.12	SORT (Sorting)	5-103
5.6.13	INL (=)	5-105
5.6.14	INR (->)	5-106
5.7	타이머 / 카운터 명령	5-107
5.7.1	TMRON (On Delay Timer)	5-107
5.7.2	TMROFF (Off Delay Timer)	5-109
5.7.3	TMRONT (적산식 On Delay Timer)	5-111
5.7.4	TMROFFT (적산식 Off Delay Timer)	5-113

5.7.5	CNTUP (UP 카운터)	5-115
5.7.6	CNTDN (DOWN 카운터).....	5-119
5.7.7	CNTUPDN (UP DOWN 카운터)	5-123
5.8	데이터 조작 명령	5-127
5.8.1	ROTL (비트 좌회전, Rotation Left).....	5-127
5.8.2	ROTL2 (Advanced Rotation Left).....	5-129
5.8.3	ROTR (비트 우회전, Rotation Right)	5-131
5.8.4	ROTR2 (Advanced Rotation Right).....	5-133
5.8.5	SFTL (비트 좌 Shift, Shift Left)	5-135
5.8.6	SFTL2 (Advanced Shift Left)	5-137
5.8.7	SFTR (비트 우 Shift, Shift Right).....	5-139
5.8.8	SFTR2 (Advanced Shift Right).....	5-141
5.9	Math Function 명령	5-143
5.9.1	SQRT (제곱근, Square Root).....	5-143
5.9.2	DEGTORAD (DEG⇒RAD 변환)	5-144
5.9.3	RADTODEG (RAD⇒DEG 변환)	5-145
5.9.4	SIN (Sine 함수).....	5-146
5.9.5	COS (Cosine 함수).....	5-147
5.9.6	TAN (Tangent 함수).....	5-148
5.9.7	ASIN (Arcsine 함수).....	5-149
5.9.8	ACOS (Arccosine 함수)	5-150
5.9.9	ATAN (Arctangent 함수).....	5-151
5.9.10	EXP (밑이 e인 지수 함수)	5-152
5.9.11	POW (지수 함수).....	5-153
5.9.12	LN (자연로그).....	5-154
5.9.13	LOG (상용로그)	5-155
5.9.14	ROUND (반올림).....	5-156
5.9.15	FUP (올림)	5-157
5.9.16	FIX (버림)	5-158
5.10	테이블 데이터 처리 명령	5-159
5.10.1	사용자 정의 스택 (User Define Stack)	5-159
5.10.2	테이블 데이터	5-167
5.11	데이터 제어 명령	5-179
5.11.1	CTRLMT (상하한 리미트 제어).....	5-179
5.11.2	DELAY (일차 지연)	5-181
5.11.3	LINEACC (직선 가속기).....	5-183

5.12 통신 명령	5-185
5.12.1 RDRMT (원격지 레지스터 읽기)	5-185
5.12.2 WRRMT (원격지 레지스터 쓰기)	5-187
5.12.3 SETREG (레지스터 설정)	5-189
5.12.4 GETREG (레지스터 읽기)	5-190
5.12.5 SETEVT (사용자 이벤트)	5-191

Preface

Conventions

본 매뉴얼에는 이해를 돕기 위해서 아래와 같은 기호 표시가 있습니다. 이 표시는 **GX-Builder** 소프트웨어의 조작에 관련된 내용과 **GX** 시스템 설정 및 동작 시에 주의 사항 또는 참고 사항을 나타내는 내용에 표시되어 있습니다.



[주의]

주의사항을 나타내는 표시입니다. 이 표시와 같이 설명된 내용은 시스템에 치명적인 오작동을 유발할 수 있으므로 반드시 숙지하시기 바랍니다.



[Tip]

참고사항을 나타내는 표시입니다.



버튼

GX-Builder 소프트웨어에 있는 조작 버튼을 나타냅니다. **GX-Builder**에는 많은 조작 버튼들이 있습니다. 이 조작 버튼의 사용법 및 기능을 설명할 때 와 같이 표시합니다.

[메뉴]

GX-Builder 소프트웨어에 있는 메뉴를 나타냅니다. 메뉴는 각 항목에 하위 메뉴가 있을 수도 있으며, 이 경우 '➡' 기호로 구분하여 제일 상위 메뉴부터 차례대로 표시합니다. (예. [파일 ➡ 저장]) 또한, 해당 위치에서 마우스의 오른쪽 버튼을 누르면 컨텍스트 메뉴(빠른 실행 메뉴)가 실행됩니다. 이때도 마찬가지로 위와 같은 형식으로 표시됩니다. (예. [프로젝트저장])

1

GX Series PLC 프로그램

GX 시리즈에서 사용되는 PLC 프로그램의 기본적인 특징과 성능 사양에 대해 설명하고, PLC 프로그램의 구성 및 실행 흐름에 대해 간략히 설명하고 있습니다.
PLC 프로그램은 GX-Builder 프로그램에서 지원하는 PLC Editor를 통해 작성되어 동작됩니다.

1.1 GX 시리즈 PLC 프로그램

1.1.1 GX 시리즈 PLC 프로그램 특징

1.1.2 GX 시리즈 PLC 성능 사양

1.2 프로그램 구성

1.2.1 PLC 실행 파일

1.2.2 데이터 메모리

1.3 프로그램 실행 흐름

1.1 GX Series의 PLC 프로그램

GX-S500은 고속 스캔 처리가 가능한 Start PLC와 Normal Speed PLC 방식의 시퀀스 제어가 가능합니다

Ladder와 Text 형태의 언어를 혼용한 프로그램 방식을 지원하며, 사용자 고유의 User Function 작성이 가능합니다.

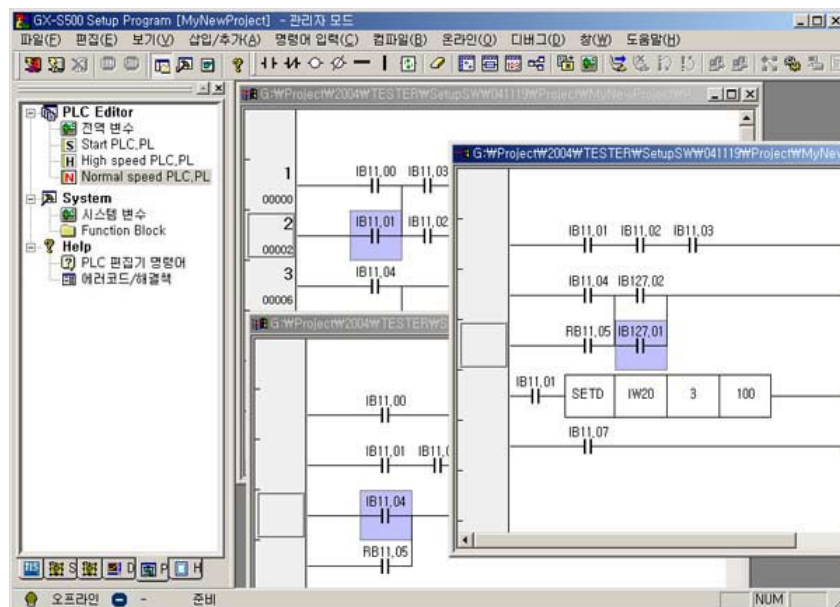
1.1.1 GX Series PLC 프로그램의 특징

■ 고성능의 Sequence 제어

- ◇ High-Speed Processing
- ◇ Start PLC, Normal Speed PLC
- ◇ Direct Register Access 가능
- ◇ 모션 프로그램 호출 기능
- ◇ Large Programming Memory

■ 고기능의 명령 처리

- ◇ 다양한 종류의 풍부한 명령어 지원
- ◇ 자주 사용되는 기본 명령과 기능 명령 분류
- ◇ 기본 명령의 일괄 처리 방식에 의한 빠른 수행
- ◇ 고속 처리 및 대용량 데이터의 처리 가능

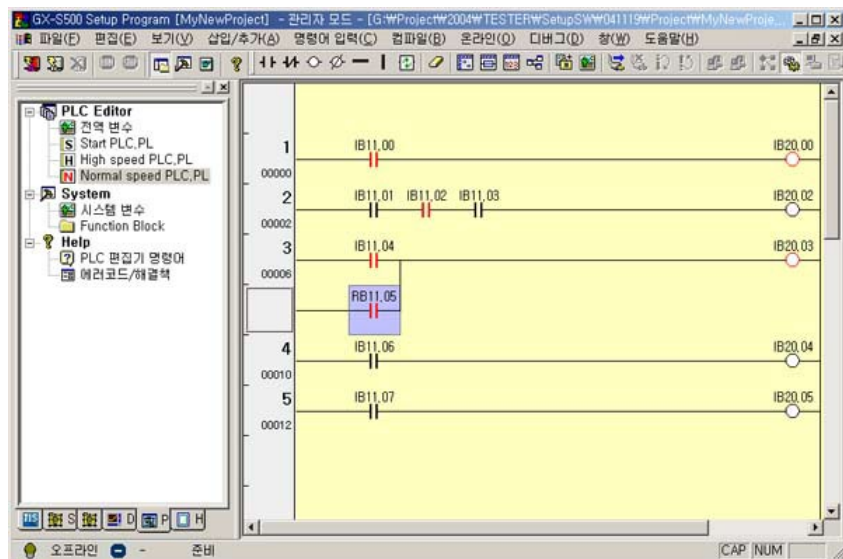


■ User 중심의 PLC 에디터

- ◇ Multi Window를 이용한 PLC 프로그램 편집
- ◇ 사용자에게 익숙한 Interface 방식의 편집 화면 구성
- ◇ Ladder 방식, Text 방식의 PLC 프로그램 편집 가능
- ◇ Symbol Table 편집 및 표시
- ◇ 키보드 및 단축키를 이용한 편집

■ 다양한 PLC 편집 / 모니터링 기능 제공

- ◇ 다양한 편집 지원 기능 (Copy, Delete, Symbol, Undo, Redo 등)
- ◇ PLC간의 통신이나 상위 컴퓨터와의 다양하고 고속의 통신 지원
- ◇ 프로그램 디버깅 / 컴파일 기능
- ◇ 프로그램 실시간 모니터링 기능
- ◇ PLC 프로그램에 대한 시뮬레이션 기능
- ◇ Online Editing 기능 지원



1.1.2 성능 사양

항 목	사 양
프로그램 방식	내장 프로그램 방식
프로그램 언어	Ladder 다이어그램
프로그램 용량	메모리 사용량 : 516 KByte
I/O 영역	최대 8192 Points I/O
제어 방식	Normal Speed 스캔 (Level1, Level2)
지원 명령 개수	160여 개의 다양한 명령 지원
스캔 타임	Normal Speed PLC : Parameter 설정

■ 프로그램 방식

내장 프로그램 방식은 모든 시퀀스 프로그램을 메모리에 저장하고 이것을 차례대로 꺼내어 이를 CPU가 해석하여 실행하는 방식입니다.

■ 제어 방식

정주기 반복 실행은 프로그램 메모리에 저장되어 있는 프로그램을 순차적으로 실행하고 마지막 명령을 실행하고는 다시 처음으로 돌아가는 제어 방식입니다.

인터럽트 제어방식은 외부 및 내부의 인터럽트가 발생 했을 시 인터럽트 수행 프로그램을 실행하는 제어 방식입니다.

■ 입출력 제어방식

입력 모듈로부터 점점 상태 정보를 읽어 이를 바탕으로 순차적으로 명령을 수행한 다음 프로그램의 실행에 의해 변화된 결과를 출력 모듈에 출력하는 일련의 작업을 Scan(스캔)이라고 합니다. 입출력 유니트의 제어방식은 Indirect방식으로 프로그램의 0 스텝 실행전에 일괄하여 실행합니다.

1.2 PLC 프로그램 구성

1.2.1 PLC 실행 파일

항 목	내 용
PLC 채널	◆ Start PLC ◆ Normal Speed PLC
PLC 실행 파일	시동 동작 처리 Ladder 파일
	Normal Speed PLC Ladder 파일
	서브 프로그램 Ladder 파일
	◆ 종류가 다른 서브 프로그램을 호출 할 수는 없음 ◆ Main Program에서 1단계로 Sub Program을 호출 가능 ◆ 비밀 유지 기능 (K-COM)

■ PLC-S Channel

- ◇ Start PLC 프로그램
- ◇ Booting시에 1회만 실행
- ◇ 실제 프로그램이 동작되기 전에 초기화 하는 코드 삽입 가능
- ◇ 프로그램 용량 2000 Step

■ PLC-N Channel

- ◇ Normal Speed PLC 프로그램
- ◇ 프로그램을 2 Level로 나눠 작성하여 수행 우선 순위 부여 가능
- ◇ 프로그램 용량 16000 Step

◇ Normal Speed PLC 파일의 LEVEL 구성

PROGRAM OF LEVEL 1
PEND (LEVEL 1)
PROGRAM OF LEVEL 2
PEND (LEVEL 2)

- ◇ 프로그램의 구성에서 **PEND** 명령은 반드시 존재하여야 합니다.
- ◇ **LEVEL1**은 **Normal Sequence** 처리 중에서 우선적으로 처리해야 하는 기능들에 대한 프로그램을 작성하여 사용합니다.
- ◇ **LEVEL1**은 단시간에 끝낼 수 있도록 작성하여야 합니다.
- ◇ **LEVEL2**는 **LEVEL1** 이외의 **Sequence**를 처리하는 루틴입니다.

■ Sub Program

- ◇ 각 채널의 **Main Program**에서 1단계까지 **Sub Program**을 호출할 수 있습니다.
- ◇ **Sub Program**은 최대 32개까지 생성할 수 있으며, 각 **Sub Program**마다 2000 Step까지 프로그램을 작성할 수 있습니다.

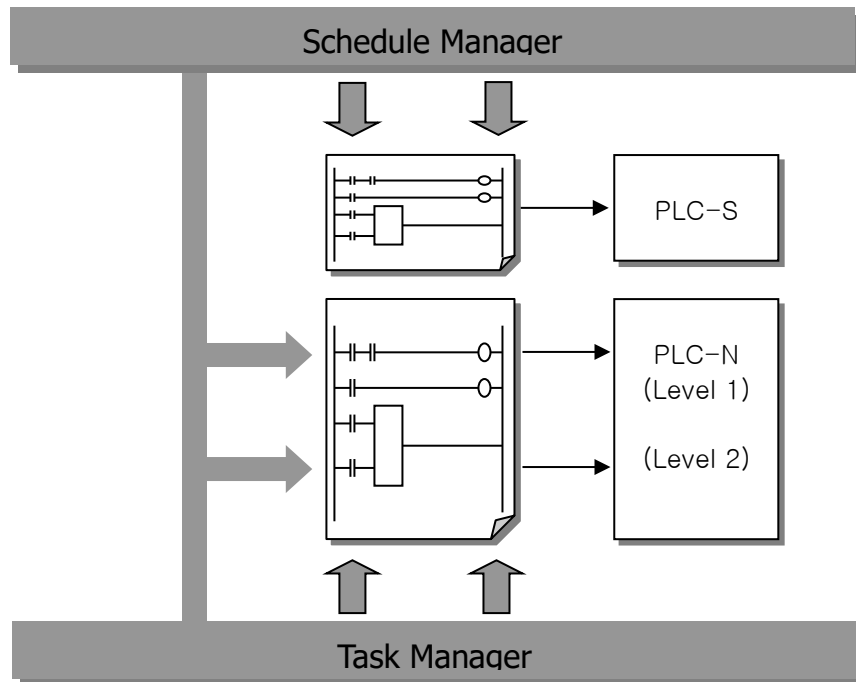
1.2.2 데이터 메모리

PLC 프로그램에서 사용 하는 데이터 메모리 종류와 데이터 타입입니다.

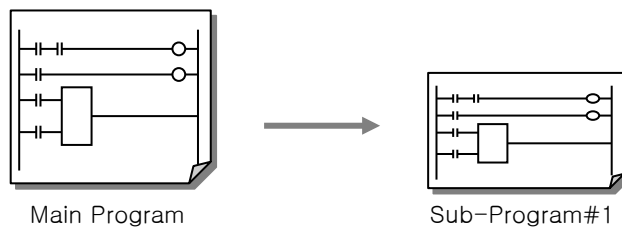
항 목	내 용	
데이터 메모리	G/F	PLC & System Interface Data
	X/Y	Input/Output
	R	PLC Program Global Variable
	#	PLC Program Local Variable
데이터 타입	Bit 형 (32 Bit)	32 Bit
	Word 형 (32 Bit)	-2,147,483,648 to 2,147,483,647
	Float 형 (32 Bit)	3.4E +/- 38 (7 digits)

1.3 프로그램 실행 흐름

PLC 프로그램은 Kernel 시스템의 Schedule Manager와 Task Manager를 통하여 실행 조건과 우선 순위에 의해 전체 PLC 프로그램의 동작 Sequence가 제어됩니다.



- PLC 프로그램은 Cyclic PLC와 Non-Cyclic PLC로 구성되어 있습니다.
- Cyclic PLC인 Normal Speed PLC는 2단계의 Level로 구성되어 있으며 N-PLC(Level 1) > N-PLC(Level 2)의 우선 순위로 수행됩니다.
- 채널의 Main PLC 프로그램은 1단계의 Sub 프로그램을 호출할 수 있습니다.



2

PLC 명령

GX 시리즈의 PLC 프로그램에서 사용되는 PLC 명령의 구성 및 특징에 대해 설명하고 있으며, 명령 타입별 PLC 명령들을 간략히 설명하고 있습니다.

2.1 PLC 명령

2.1.1 일반 시퀀스 명령

2.1.2 내부 연산 처리 명령

2.1.3 기능 명령

2.2 명령 구성

2.2.1 명령 구성

2.2.2 레지스터 표시 방법

2.3 PLC 명령 목록

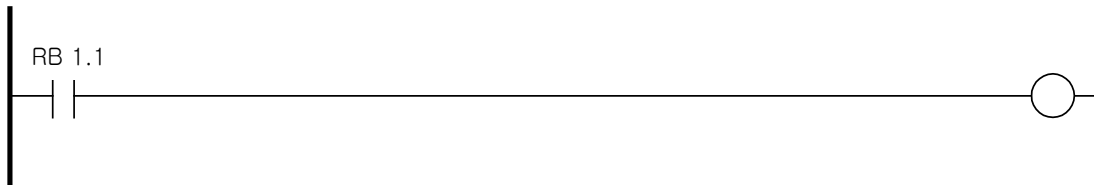
2.1 PLC 명령

GX 시리즈의 PLC 프로그램은 다양한 종류의 160여 개의 명령을 지원하고 있으며, PLC 명령은 크게 일반 시퀀스 명령, 내부 연산 처리 명령, 기능 명령으로 나눌 수 있습니다.

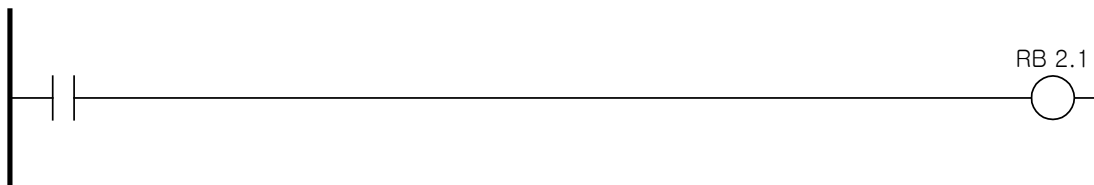
2.1.1 일반 시퀀스 명령

- RD / RDN / WR / SET 과 같이 프로그램 작성시 자주 사용되는 입출력 명령
- 일반 시퀀스 명령은 내부 일괄처리 방식으로 명령 수행 속도가 빠릅니다.

[RD]



[WR]



- 일반 시퀀스 명령은 Ladder의 1 Cell 사용하며 하나의 인자만을 가집니다.
- 다른 일반 명령이나 기능 명령과 조합하여 사용합니다.
- WRUP / WRDN / SET / RESET 명령은 입력 신호와 조합하여 사용해야 합니다.

[WPUP]



[WRDN]



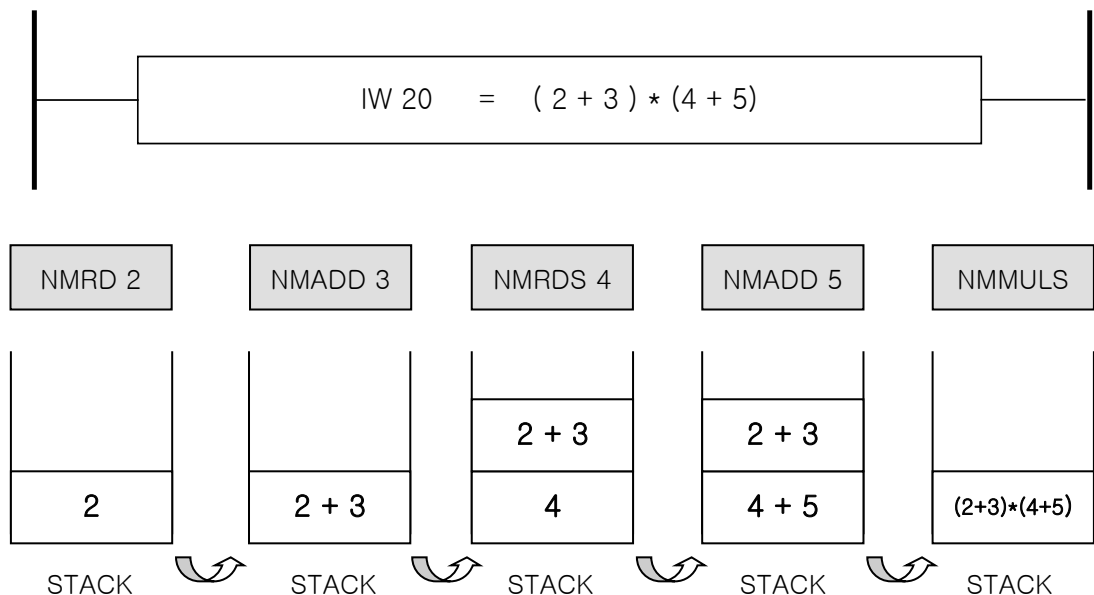
2.1.2 내부 연산 처리 명령

- 연산 데이터 임시 저장소 스택을 사용하여 연산을 처리하는 명령
- 일반 명령이나 기능 명령들의 조합으로 이루어진 명령을 수행할 때 내부 연산 처리에 사용되는 명령입니다.
- 릴레이 회로 스택 연산 명령과 수치 스택 연산 명령
- 내부 명령이기 때문에 사용자가 직접 명령을 입력할 수 없습니다.

■ 릴레이 회로 스택 연산 명령



■ 수치 스택 연산 명령



2.1.3 기능 명령

- 다양한 기능에 사용되는 명령
- 기능 특성에 따라 일반 기능 명령과 고급 기능 명령으로 분류됩니다.
- 명령 특성에 따라 입력부, 출력부를 가지고, 연산자를 필요로 합니다.
- GX PLC 프로그램에서는 다음과 같은 종류의 기능 명령을 제공합니다.

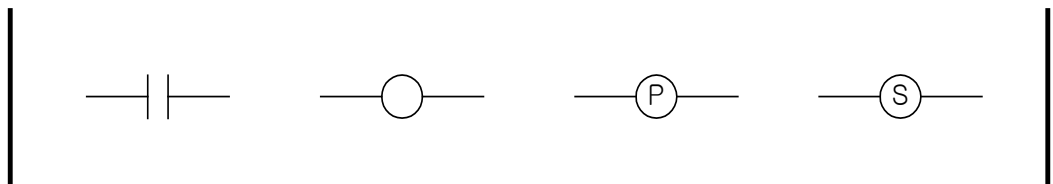
	종 류
일반 기능 명령 고급 기능 명령	◇ 프로그램 제어 명령 ◇ 논리 연산 명령 ◇ 수치 연산 명령 ◇ 수치 비교 명령 ◇ 데이터 변환 명령 ◇ 데이터 전송 명령 ◇ 타이머 / 카운터 명령 ◇ 데이터 조작 명령 ◇ Math Function 명령 ◇ 테이블 데이터 처리 명령 ◇ 데이터 제어 명령 ◇ 통신 명령

2.2 명령 구성

일반 명령은 한 Cell로 명령이 이루어지고 기능 명령은 한 lung 을 기준으로 입력부, 명령부, 출력부로 구성됩니다.

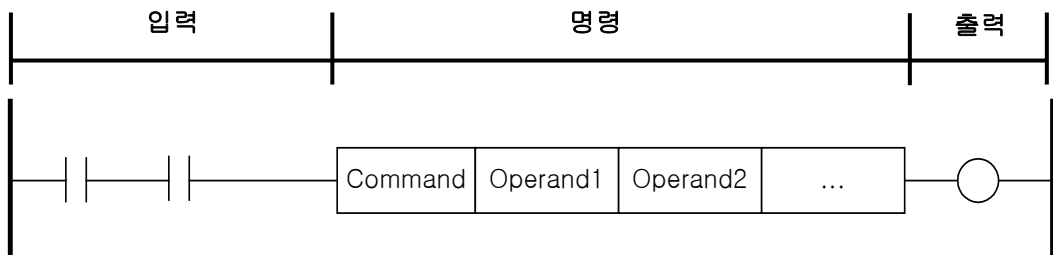
2.2.1 명령 구성

■ 일반 명령



- ◇ 일반 명령은 Ladder의 한 Cell로 명령이 이루어집니다.
- ◇ 다른 일반 명령이나 기능 명령과 조합하여 명령을 구성할 수 있습니다.
- ◇ WRUP / WRDN / SET / RESET 명령은 반드시 입력 신호가 있어야 합니다.

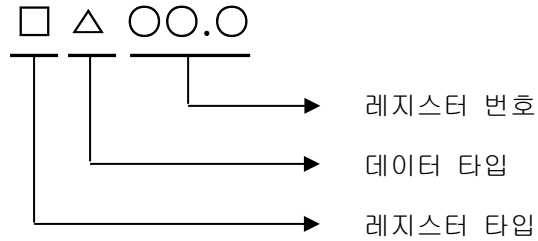
■ 기능 명령



- ◇ 기능 명령은 Ladder의 한 Rung을 기준으로 이루어진 명령입니다.
- ◇ 기능 명령의 기본 구성은 명령부 입니다.
- ◇ 입력부는 입력 명령들의 조합으로 구성할 수 있으며, 입력부를 생략하면 조건 없이 명령이 수행됩니다.
- ◇ 명령에 대한 출력을 사용할 수 있습니다. 명령이 정상 수행되면 HIGH가 출력됩니다.

2.2.2 레지스터 표시 방법

■ 레지스터 표시 구성



■ 데이터 메모리

항 목	내 용	
데이터 메모리	G/F	PLC & System Interface Data
	X/Y	Input/Output
	R	PLC Program Global Variable
	#	PLC Program Local Variable

■ 연산자 타입

	연산자 타입	표시 기호
Register Type	BIT	B
	WORD (32Bit)	W
	FLOAT (32Bit)	F
Constant Type	WORD 형 상수	CW
	FLOAT 형 상수	CF

■ 사용 예

사용 예	설 명
GB 100.10	G 레지스터 100번의 10 Bit
XW 50	X 레지스터의 50번 Word형 데이터
GF 200	G 레지스터의 200번 Float형 데이터

2.3 PLC 명령 목록

GX PLC 명령은 크게 일반 명령과 기능 명령으로 나눌 수 있습니다.

자주 사용되는 일반 명령에는 입출력 관련 명령과 릴레이 회로 스택 연산 명령, 수치 스택 연산 명령 같은 내부 연산 처리 명령이 있고 기능 명령에는 다양한 타입의 명령을 지원하고 있습니다.

■ 입출력 명령

지정한 레지스터에 대한 접점 상태로 입출력을 나타내는 명령으로 가장 자주 사용되는 PLC 기본 명령입니다.

명 칭	심 볼	내 용
READ	RD	비트형 레지스터 값을 스택에 저장
READ NOT	RDN	비트형 레지스터 값을 반전하여 스택에 저장
READ 상승미분	RDUP	지정한 접점이 상승(OFF→ON)시에만 1사이클 동안 HIGH
READ 하강미분	RDDN	지정한 접점이 하강(ON→OFF)시에만 1사이클 동안 LOW
WRITE	WR	논리연산 결과를 지정한 레지스터에 저장
WRITE NOT	WRN	논리연산 결과를 반전하여 레지스터에 저장
상승 미분 (출력계)	WRUP	입력신호의 상승(OFF→ON)시에 지정한 접점을 1사이클 동안 HIGH
하강 미분 (출력계)	WRDN	입력신호의 하강(ON→OFF)시에 지정한 접점을 1사이클 동안 LOW
Set Coil (출력계)	SET	Set 신호가 On 되면 출력 신호는 계속 HIGH 를 유지
Reset Coil (출력계)	RESET	Reset 신호가 On 되면 출력 신호는 계속 LOW 를 유지
KEEP	KEEP	자기 유지 동작

■ 릴레이 회로 스택 연산 명령

둘 혹은 그 이상의 입출력 명령 간의 상호 관계를 나타내는 내부 명령입니다.

연산과 관련하여 데이터에 대한 임시 저장소로 Stack을 사용합니다.

명 칭	심 볼	내 용
AND	AND	지정한 레지스터의 값과 첫번째 스택의 값을 논리곱 연산하여 스택에 저장
AND NOT	ANDN	지정한 레지스터의 값을 반전하여 첫번째 스택의 값과 논리곱 연산하여 스택에 저장
AND 상승 미분	ANDUP	지정한 레지스터의 상승 미분 값과 스택의 값을 논리곱하여 스택에 저장
AND 하강 미분	ANDDN	지정한 레지스터의 하강 미분 값과 스택의 값을 논리곱하여 스택에 저장
OR	OR	지정한 레지스터의 값과 첫번째 스택의 값을 논리합 연산하여 스택에 저장
OR NOT	ORN	지정한 레지스터의 값을 반전하여 첫번째 스택의 값과 논리합 연산하여 스택에 저장
OR 상승 미분	ORUP	지정한 레지스터의 상승 미분 값과 스택의 값을 논리합하여 스택에 저장
OR 하강 미분	ORDN	지정한 레지스터의 하강 미분 값과 스택의 값을 논리합하여 스택에 저장
Read Stack	RDS	스택값을 1비트 좌로 Shift하고, 지정한 레지스터의 값을 스택에 저장
Read Not Stack	RDNS	스택값을 1비트 좌로 Shift하고, 지정한 레지스터의 값을 반전하여 스택에 저장
Read 상승 미분 Stack	RDUPS	스택값을 1비트 좌로 Shift하고, 지정한 레지스터의 상승 미분 값을 스택에 저장
Read 하강 미분 Stack	RDDNS	스택값을 1비트 좌로 Shift하고, 지정한 레지스터의 하강 미분 값을 스택에 저장
And Stack	ANDS	첫번째, 두번째 스택값을 논리곱 연산하여 스택에 저장하고, 스택값을 1비트 우로 Shift
Or Stack	ORS	첫번째, 두번째 스택값을 논리합 연산하여 스택에 저장하고, 스택값을 1비트 우로 Shift

■ 프로그램 제어 명령

조건문, 반복문 등과 같은 프로그램 제어에 관련된 명령입니다. 또한 서브 프로그램과 모션 프로그램을 호출하는 명령을 포함하고 있습니다.

명 칭	심 볼	내 용
서브 프로그램 호출	SCALL	서브 프로그램을 호출
모션 프로그램 호출	PCALL	모션 프로그램을 호출
IF 문	IFON	OP1의 값이 1이면 IF문 실행
	IFOFF	OP1의 값이 0이면 IF문 실행
	ELSE	ELSE
	IEND	IF문 END
Switch-Case 문	SWITCH	Switch 문
	CASE	Case 문
	DEFAULT	Default 문
	SWEND	Switch문 END
FOR 문	FOR	FOR 문
	FEND	FOR문 END
WHILE 문	WHILEON	OP1 값이 1이면 While루프 실행
	WHILEOFF	OP1 값이 0이면 While루프 실행
	WEND	WHILE문 END
BREAK	BREAK	반복 루프를 빠져나감
점프 처리	GOTO	지정한 LABEL 번호로 점프
LABEL표시	LABEL	LABEL 표시
프로그램 끝	PEND	프로그램 끝을 표시
수행 정지	STOP	전체 System 동작을 정지

■ 논리 연산 명령

논리 연산을 수행하는 PLC 명령입니다.

명 칭	심 볼	내 용
논리곱	ANDMOV	논리곱 결과를 지정한 레지스터에 저장
논리합	ORMOV	논리합 결과를 지정한 레지스터에 저장
배타적논리합	XORMOV	배타적논리합 결과를 지정 레지스터에 저장

■ 수치 연산 명령

수치 및 Time에 대한 기본 연산을 수행하는 명령입니다.

명 칭	심 볼	내 용
+	ADD	OP1 + OP2 의 값을 OP3에 저장
-	SUB	OP1 - OP2 의 값을 OP3에 저장
*	MUL	OP1 * OP2 의 값을 OP3에 저장
/	DIV	OP1 / OP2 의 값을 OP3에 저장
나머지	MOD	나머지
INCREMENT	INC	지정된 레지스터에서 1을 더함
DECREMENT	DEC	지정된 레지스터에서 1을 뺌
TIME 가산	TIMEADD	시/분/초 가산
TIME 감산	TIMESUB	시/분/초 감산

■ 수치 스택 연산 명령

둘 혹은 그 이상의 수치 연산 명령 간의 상호 관계를 나타내는 내부 명령입니다

연산과 관련하여 데이터에 대한 임시 저장소로 Stack을 사용합니다.

명 칭	심 볼	내 용
Numeric Read	NMRD	지정한 수치 값을 스택에 저장
Numeric Read Stack	NMRDS	지정한 수치 스택값을 1칸 좌로 Shift하고, 지정한 수치 값을 수치 스택에 저장
Addition	NMADD	지정한 수치 값과 스택의 값을 더하여 스택에 저장
Subtraction	NMSUB	지정한 수치 값과 스택의 값을 빼서 스택에 저장
Multiply	NMMUL	지정한 수치 값과 스택의 값을 곱하여 스택에 저장
Divide	NMDIV	지정한 수치 값과 스택의 값을 나누어 스택에 저장
Addition Stack	NMADDS	첫번째, 두번째 수치 스택값을 더하여 스택에 저장하고, 수치 스택값을 1칸 우로 Shift
Subtraction Stack	NMSUBS	첫번째, 두번째 수치 스택값을 빼서 스택에 저장하고, 수치 스택값을 1칸 우로 Shift
Multiply Stack	NMMULS	첫번째, 두번째 수치 스택값을 곱해서 스택에 저장하고, 수치 스택값을 1칸 우로 Shift
Divide Stack	NMDIVS	첫번째, 두번째 수치 스택값을 나눠서 스택에 저장하고, 수치 스택값을 1칸 우로 Shift
Numeric Write	NMWR	논리연산 결과를 지정 레지스터에 저장

■ 수치 비교 명령

레지스터의 값이나 수치에 대한 비교 연산을 수행하는 명령입니다.

비교 연산에 대한 결과 출력을 저장할 레지스터 지정을 필요로 합니다.

명 칭	심 볼	내 용
<	LT	크기 비교
≤	LE	크기 비교
>	GT	크기 비교
≥	GE	크기 비교
=	EQ	동등 비교
=	EQ2	동등 비교
≠	NE	동등 비교
범위 체크	RNG	레지스터의 값이 범위 내에 있는지 체크.

■ 데이터 변환 명령

데이터의 여러 가지 형 변환 명령입니다.

명 칭	심 볼	내 용
부호 반전	INV	부호를 반대로 변환
비트 반전	INVB	레지스터의 비트 값을 반전 (0→1, 1→0)
2의 보수	COM	2의 보수(Complement)로 변환해 줌
절대치 변환	ABSD	절대치 변환
BCD 변환	BCD	BIN ⇒ BCD 변환
2진 변환	BIN	BCD ⇒ BIN 변환
데이터형 변환 (WORD→FLOAT)	WTOF	WORD형 레지스터를 FLOAT형으로 변환
데이터형 변환 (FLOAT→WORD)	FTOW	FLOAT형 레지스터를 WORD형으로 변환
비트행→ 비트열 변환	BITRTOC	지정된 레지스터의 각 비트 데이터를 별도로 지정한 32개의 레지스터의 지정비트에 Set
비트열→ 비트행 변환	BITCTOR	지정된 32개 레지스터에서 지정비트의 데 이터만을 별도로 지정한 레지스터에 Set

패리티 변환	PARITY	2진수 표현 Bit의 On Bit 수를 계산
문자열 변환 (ASCII → BIN)	ASCTOBIN	문자열을 32Bit의 Binary Data로 변환 (ASCII 코드에 의한 변환)
문자열 변환 (BIN → ASCII)	BINTOASC	32Bit의 Binary Data를 문자열로 변환 (ASCII 코드에 의한 변환)

■ 데이터 전송 명령

데이터에 대한 Move, Change, Set 같은 전송 명령입니다.

명 칭	심 볼	내 용
비트 복사	MOVB	원하는 위치로 비트 값을 복사
레지스터 복사	MOVD	원하는 위치로 레지스터 값을 복사
비트 교환	CHGB	서로의 비트 값을 교환
레지스터 교환	CHGD	서로의 레지스터 값을 교환
Byte Swap	SWAP	워드변수의 상위바이트와 하위바이트를 교환
Data 초기화	SETD	지정된 범위의 레지스터를 초기화
Data 초기화	SETD2	지정된 범위의 레지스터를 초기화
Set Bit Data	SETB	레지스터의 특정 위치에 값을 기록
Set Bit Data	SETB2	레지스터의 특정 위치에 값을 기록
Get Bit Data	GETB	레지스터 비트의 특정 위치에 값을 기록
Data 검색	SRCH	지정된 범위의 레지스터 중에서 Data와 일치하는 레지스터의 위치를 검색
SORT	SORT	지정한 범위의 레지스터를 정렬
=	INL	오른쪽 값을 왼쪽 인자에 대입
→	INR	왼쪽 값을 오른쪽 인자에 대입

■ 타이머 / 카운터 명령

여러 가지 종류의 타이머와 카운터에 대한 명령입니다.

명 칭	심 볼	내 용
On Delay Timer (1 ms)	TMRON	On Delay Timer (1 ms) (0 ~ 2147483초 까지 설정가능)
Off Delay Timer (1 ms)	TMROFF	Off Delay Timer (1 ms) (0 ~ 2147483초 까지 설정가능)
적산식 On Delay Timer (1 ms)	TMRONT	적산식 ON Delay Timer (0 ~ 2147483초 까지 설정가능)
적산식 Off Delay Timer (1 ms)	TMROFFT	적산식 OFF Delay Timer (0 ~ 2147483초 까지 설정가능)
UP 카운터	CNTUP	UP 카운터
DOWN 카운터	CNTDN	DOWN 카운터
UP/DOWN 카운터	CNTUPDN	UP/DOWN 카운터

■ 데이터 조작 명령

Rotation, Shift등과 같은 데이터 조작과 관련된 명령입니다.

명 칭	심 볼	내 용
비트 좌회전	ROTL	지정한 레지스터의 비트 값을 좌로 회전
고급 비트 좌회전	ROTL2	지정한 범위에서 비트 값을 좌로 회전
비트 우회전	ROTR	지정한 레지스터의 비트 값을 우로 회전
고급 비트 우회전	ROTR2	지정한 범위에서 비트 값을 우로 회전
비트 좌 Shift	SFTL	지정한 레지스터의 비트 값을 좌로 Shift
고급 비트 좌 Shift	SFTL2	지정한 범위에서 비트 값을 좌로 Shift
비트 우 Shift	SFTR	지정한 레지스터의 비트 값을 우로 Shift
고급 비트 우 Shift	SFTR2	지정한 범위에서 비트 값을 우로 Shift

■ Math Function 명령

수학 함수 명령입니다.

명 칭	심 볼	내 용
평방근	SQRT	평방근
Degree 변환	RADTODEG	Radian을 Degree로 변환
Radian 변환	DEGTORAD	Degree를 Radian으로 변환
SIN	SIN	SIN
COSIN	COS	COS
TANGENT	TAN	TAN
ARC SINE	ASIN	ASIN
ARC COSINE	ACOS	ACOS
ARC TANGENT	ATAN	ATAN
지수	EXP	밑이 e인 지수 계산
지수	POW	지수 계산
자연로그	LN	LN
상용로그	LOG	LOG
반올림	ROUND	반올림
올림	FUP	올림
버림	FIX	버림

■ 데이터 제어 명령

데이터 제어와 관련된 명령입니다.

명 칭	심 볼	내 용
상/하한 LIMIT 제어	CTRLMT	입력 데이터가 상하한 리미트 데이터 범위의 데이터인지 비교하여 출력 데이터 제어
일차지연	DELAY	일차지연
직선 가속기	LINEACC	직선 가속기

■ 테이블 데이터 처리 명령

테이블 데이터와 관련된 명령입니다.

테이블 데이터를 생성, 설정하고 데이터에 대한 검색을 수행하는 명령들이 있습니다.

명 칭	심 볼	내 용
스택 영역 설정	STSET	스택 영역을 설정하고, 설정된 영역의 값을 0으로 초기화
스택데이터 격납	STPUSH	스택 영역에 데이터를 격납(Push)
선입 선출	STFIFO	처음 저장한 데이터를 가져오는 명령어
후입 선출	STLIFO	마지막 저장한 데이터 가져오는 명령어
테이블 값 읽기	TBRD	테이블에서 지정한 위치의 값을 Read
테이블 값 쓰기	TBWR	테이블에서 지정한 위치에 값을 Write
테이블 데이터 검색	TBSRCH	테이블에서 데이터의 위치를 검색한다.
테이블 최대값 검색	TBMAX	지정한 테이블에서 최대값을 검색
테이블 최소값 검색	TBMIN	지정한 테이블에서 최소값을 검색
테이블 합계 산출	TBSUM	지정한 테이블의 합을 산출

■ 통신 명령

메시지 송수신과 같은 통신과 관련된 명령입니다.

명 칭	심 볼	내 용
원격 레지스터 읽기	RDRMT	통신모듈을 통해 원격지 레지스터 읽기
원격 레지스터 쓰기	WRRMT	통신모듈을 통해 원격지 레지스터 쓰기
레지스터 읽기	GETREG	지정 레지스터에 값 읽기
레지스터 쓰기	SETREG	지정 레지스터에 값 쓰기
사용자 이벤트	SETEVT	사용자 이벤트 설정 명령

3

PLC 프로그램 편집기

GX 시리즈의 PLC 프로그램 편집기의 개요 및 특징에 대해 설명하고, PLC 프로그램 편집기 사용 방법 및 프로그램 편집 방법에 대해 간략히 설명하고 있습니다. 'PLC 프로그램 편집기'에 대한 자세한 설명은 'GX-Builder 매뉴얼'을 참조하십시오.

3.1 GX PLC 프로그램 에디터

3.1.1 PLC 에디터 특징

3.1.2 기능 사양

3.2 GX PLC 프로그램 화면 구성

3.2.1 전체 화면 구성

3.2.2 Menu Bar 주요 항목

3.2.3 Tool Bar 주요 항목

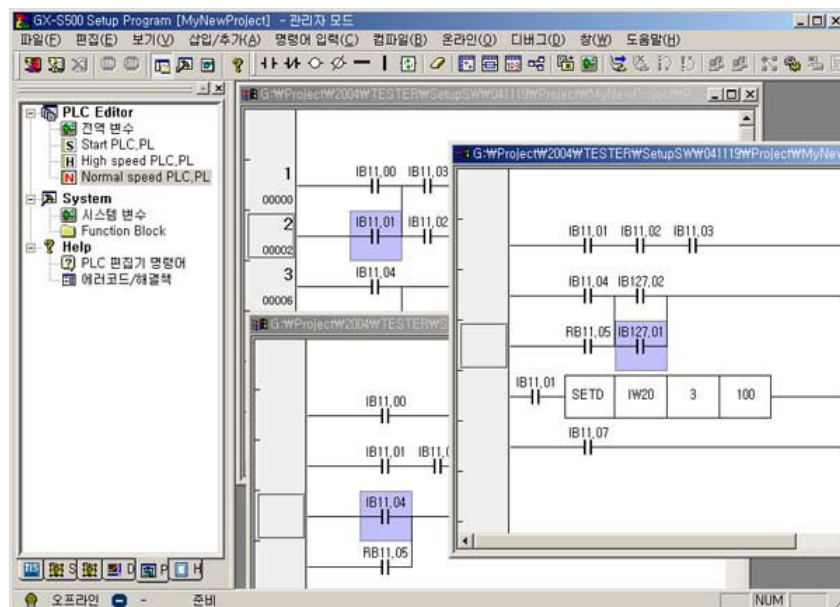
3.3 PLC 프로그램 작성

3.1 GX Series PLC 프로그램 편집기

PLC 편집기는 GX에서 동작하는 PLC 래더 파일을 작성하는 프로그램입니다. PLC 편집기에서 작성하여, 컴파일, 다운로드 한 PLC 래더 파일은 PLC Executor를 통해 실행 됩니다

GX PLC 프로그램 편집기는 Ladder와 Text 형태의 언어를 혼용한 프로그램 방식을 지원합니다.

3.1.1 PLC 에디터 특징



■ GX PLC 프로그램의 특징

- ◇ High-Speed Processing
- ◇ Large Programming Memory
- ◇ 150여 개의 풍부한 명령어 지원
- ◇ 사용자에게 익숙한 Interface 방식의 편집 화면 구성
- ◇ Ladder 방식의 PLC 프로그램 편집 가능
- ◇ Symbol Table 편집 및 표시
- ◇ 다양한 편집 지원 기능
- ◇ 프로그램 실시간 모니터링 / 시뮬레이션 기능
- ◇ Online Editing 기능 지원

3.1.2 기능 사양

■ PLC 프로그램 기능 사양

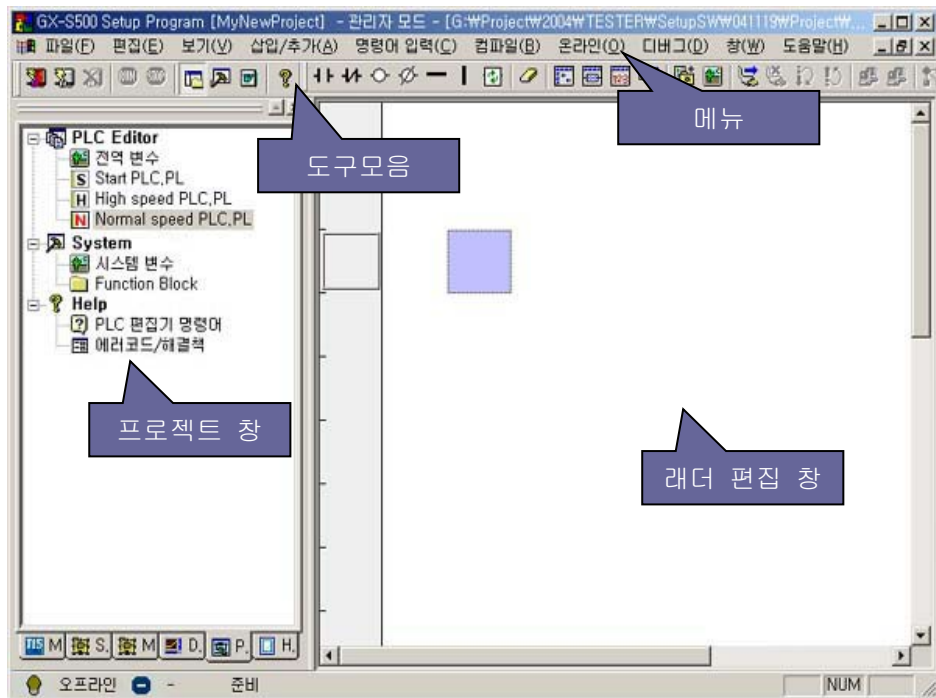
PLC Editor는 다음과 같이 다양한 기능을 지원합니다.

기 능	설 명
명령어 입력 기능	인플레이스 에디팅 (In-place editing)
편집 기능	Redo, Undo, Copy, Cut 등의 기본 편집 기능
변수 관리 기능	시스템에서 사용되는 변수를 정의, 관리하는 기능
Build 기능	PLC 편집 파일을 2진 파일로 변환시키는 기능
모니터링 기능	기계와 통신 연결하여 실제 프로그램의 상태를 모니터링
시뮬레이션 기능	가상으로 프로그램의 상태를 테스트 하는 기능
Online-Editing 기능	실제 실행중에 PLC 파일을 수정하여 적용할 수 있는 기능
수치 연산 블록	레지스터와 상수의 여러 조합을 연산하는 기능
엑셀 호환 기능	데이터 저장 파일을 엑셀과 호환하여 사용 가능
인쇄 기능	PLC 편집 파일 인쇄
도움말 기능	PLC 명령과 에러코드에 대한 도움말 제공

3.2 GX PLC 프로그램 화면 구성

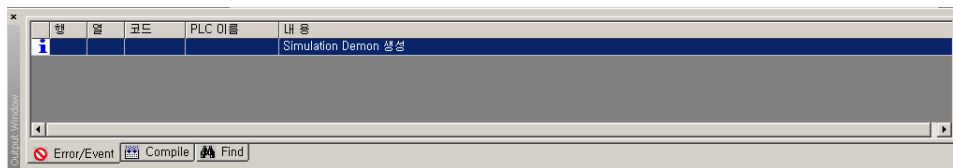
3.2.1 전체 화면 구성

PLC Editor 모듈의 화면 구성은 다음과 같습니다.



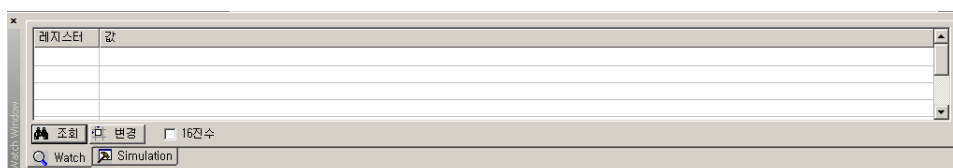
다음은 PLC Editor에서 사용하는 Output Window입니다.

Error/Event, Compile, Find Tab으로 이루어져 있습니다.



아래 그림은 변수 모니터 창입니다.

Quick, Watch Tab으로 이루어져 있습니다.



3.2.2 Menu Bar 주요 항목

PLC 프로그램 편집 시 사용하는 Menu Bar의 주요 항목에 대해 설명합니다.

주로 명령어 편집, 컴파일, 온라인 기능 등에 관한 내용입니다.

명령어 입력	컴파일	온라인
기본 명령어 입력창... 기능 명령어 입력창... 수치연산 블록 입력창... Read Read Not Write Write Not Invert Horizontal Link Vertical Link	컴파일 전체 컴파일 컴파일 중지 이전 컴파일 파일 지우기	On-line Off-Line 모니터링 모드 전환 시뮬레이션 모드 전환 온라인에디트 모드 전환 디버그 모드 전환 Run Mode Stop Mode 다운로드... 업로드...

■ 명령어 입력

기 능	설 명
기본 명령어 입력창...	기본 명령어(RD, RDN, WR, WRN)를 입력합니다.
기능 명령어 입력창...	기능명령어를 입력합니다.
수치연산 블록 입력창...	수치연산 블록을 입력합니다.
Read	기본 명령어 RD를 입력합니다.
Read Not	기본 명령어 RDN을 입력합니다.
Write	기본 명령어 WR를 입력합니다.
Write Not	기본 명령어 WRN을 입력합니다.
Invert	기본 명령어를 반전시킵니다.
Horizontal Link	수평 링크를 입력합니다.
Vertical Link	수직 링크를 입력합니다.

■ 컴파일

기 능	설 명
컴파일	현재 선택된 PLC 편집파일을 컴파일 합니다.
전체 컴파일	프로젝트에 등록된 PLC 파일 전체를 컴파일 합니다.
컴파일 중지	컴파일을 중지합니다.
이전 컴파일 파일 지우기	이전 컴파일된 파일을 지웁니다.

■ 온라인

기 능	설 명
온라인	온라인으로 전환합니다.
오프라인	온라인에서 오프라인으로 전환합니다.
모니터링 모드 전환	모니터링 모드로 전환합니다.
시뮬레이션 모드 전환	시뮬레이션 모드로 전환합니다.
온라인에디트 모드 전환	온라인에디트 모드로 전환합니다.
Run 모드	기계를 Run 상태로 만듭니다.
Stop 모드	기계를 Stop 상태로 만듭니다.
다운로드...	컴파일된 PLC 파일을 다운로드 합니다.
파일 시스템...	GX의 PLC 파일 시스템을 보여줍니다.

3.2.3 Tool Bar 주요 항목

PLC 프로그램 편집 시 자주 사용되는 Tool Bar의 주요 항목에 대해 설명합니다.
주로 명령어 편집, 컴파일, 온라인 기능 등에 관한 내용입니다.

■ GX-Builder 공통 도구



도 구	기 능	도 구	기 능
	통신 설정		통신 연결
	통신 종료		RUN 모드로 전환
	STOP 모드로 전환		워크 스페이스 윈도우
	프로그램 정보		

■ 표준 도구



도 구	기 능	도 구	기 능
	파일 저장		전체 저장
	실행 취소		재실행
	잘라내기		복사
	붙여넣기		찾기/바꾸기
	다중 바꾸기		화면 확대
	화면 축소		현재 파일 컴파일
	전체 파일 컴파일		컴파일 중지
	컴파일된 파일 지우기		

■ 래더 명령어 도구

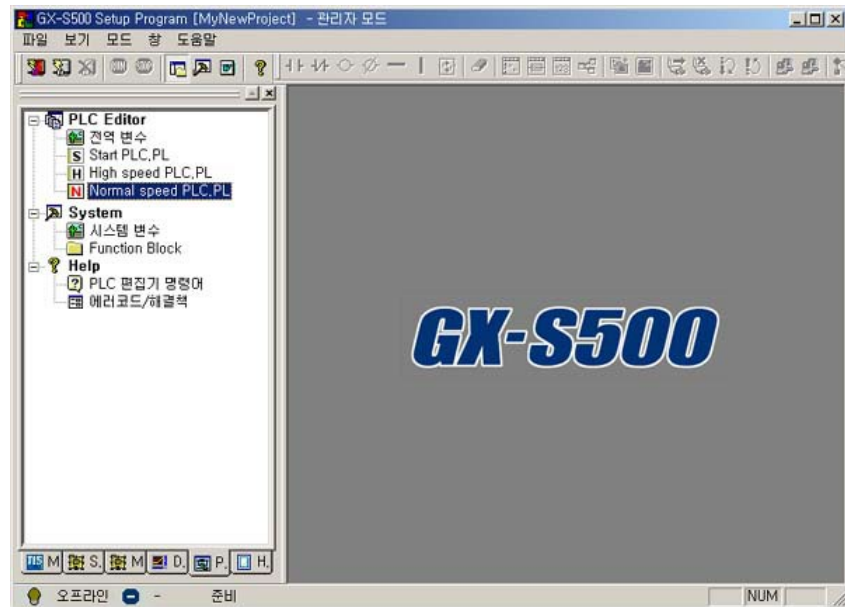


도 구	기 능	도 구	기 능
	Read		Read Not
	Write		Write Not
	Horizontal Link		Vertical Link
	Invert		셀/명령어 지우기
	기본 명령어 입력창		기능 명령어 입력창
	수치연산 블록 입력창		사용한 체크리스트/기능명령 보기
	변수 관리창		다운로드
	모니터링 모드 전환		시뮬레이션 모드 전환
	온라인에디트 모드 전환		속성
	프로그램 관리		

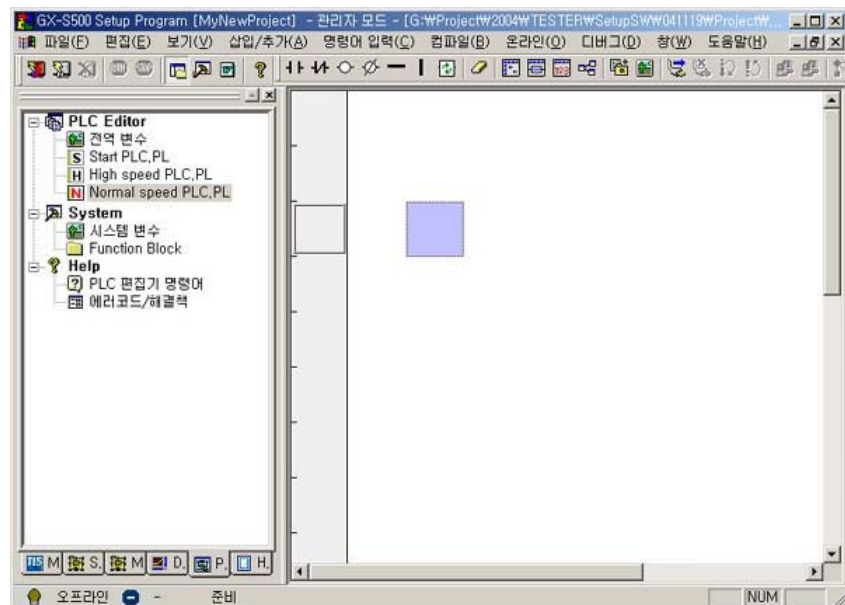
3.3 PLC 프로그램 작성

■ PLC 파일 열기

Setup Program의 Tree Window에서 PLC Editor로 이동하면 다음과 같은 Editor 화면이 나옵니다.



PLC Editor의 Tree Window의 항목을 살펴보면 두개 채널의 PLC로 구성되어 있는 것을 알 수 있습니다. 이중 편집하고자 하는 채널을 선택하면 해당 파일이 열립니다.



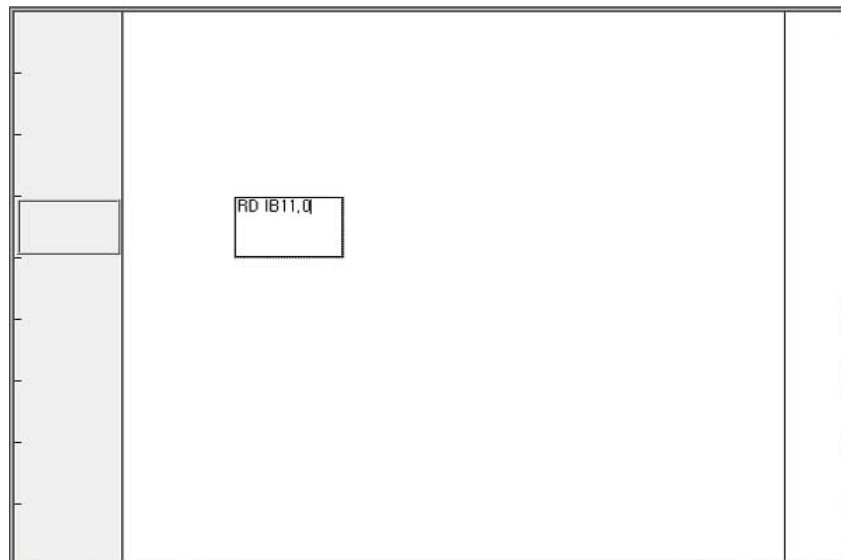
■ PLC 명령 입력

PLC 명령을 입력하는 방법은 직접 명령어를 입력하는 방식과 명령어 입력 창을 사용하여 입력하는 방식이 있습니다. 이 중에서 편집하기 편한 방식을 선택하여 원하는 프로그램을 작성합니다.

◇ 직접 명령 입력

명령을 입력하고자 하는 **Cell**을 선택한 후 키보드를 이용하여 명령을 바로 입력합니다. 기본 명령어와 기능 명령어 모두 입력이 가능 합니다.

일반적으로 명령 이름과 형식을 알고 있을 경우 빠른 입력을 원할 때 사용됩니다. 기본 명령의 경우 기본 명령어와 레지스터 지정을 포함하여 입력하면 입력한 기본 명령에 대한 **Ladder**가 생성됩니다.

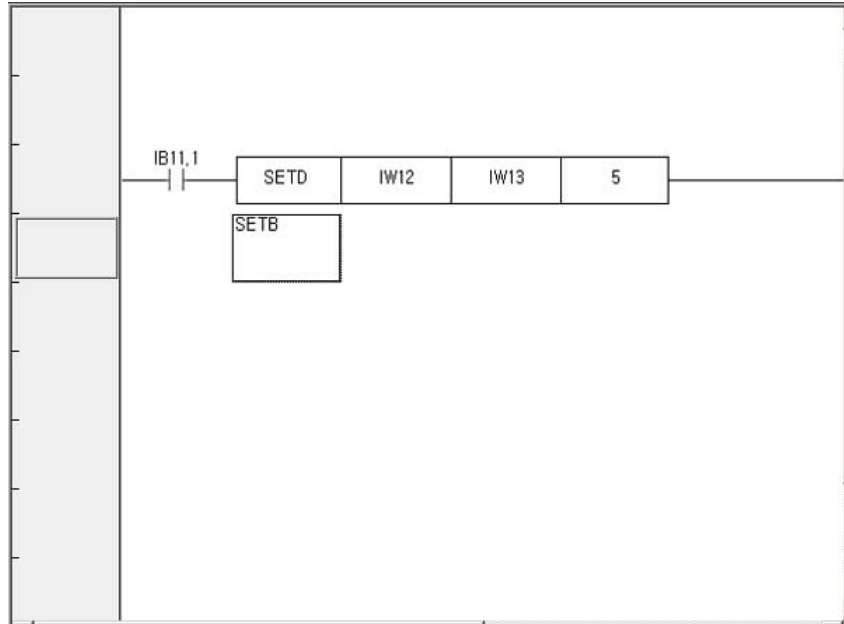


명령에 대한 레지스터 지정이 잘못되었거나 연산자의 데이터 타입이 다르면 입력되지 않습니다. 또한 생성된 **Ladder**를 클릭하면 입력한 명령어를 수정할 수 있습니다.

인플레이스 에디트 시, 입력 형식과 구분자는 다음과 같습니다.

입력 형식과 구분자	
입력 형식	명령어 인자1 , 인자2 , ..., 인자n ; 주석
구분자	공백 () : 명령어(Command)와 인자(Argument)를 구분
	Comma (,) : 인자와 인자를 구분
	Semicolon (;) : 인자와 주석(Comment)을 구분

기능 명령어를 입력하는 경우에는 명령어만 입력하면 해당 명령에 대한 Ladder가 생성됩니다. 그러면 생성된 Ladder에 필요한 인자를 입력합니다.

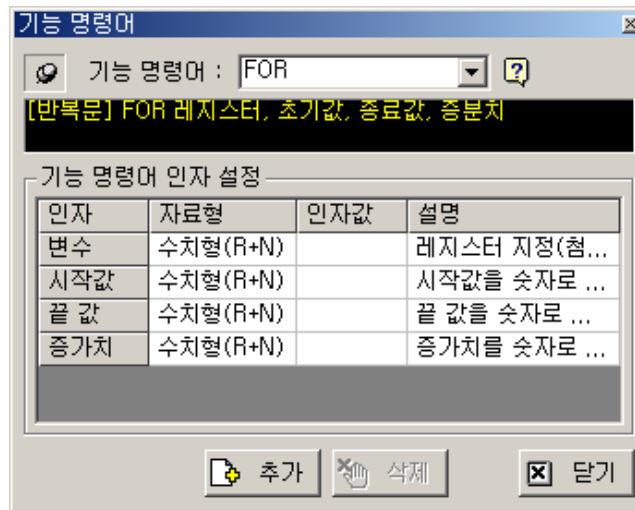


연산자의 데이터 타입이나 지정이 잘못되면 입력되지 않습니다. 또한 기본 명령과 마찬가지로 생성된 Ladder를 클릭하면 입력한 명령어를 수정할 수 있습니다.

◇ 명령어 입력 창 사용

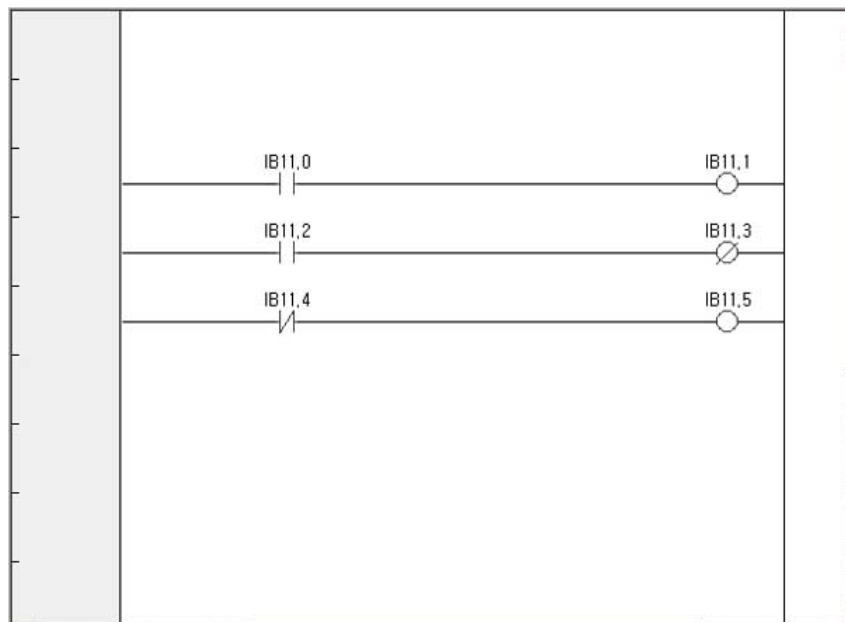
Menu Bar의 [명령어 입력]의 [기본 명령어 입력창]과 [기능 명령어 입력창]을 선택하면 다음과 같은 명령어 입력 창이 열립니다.

또한 래더 명령어 도구에서 해당 아이콘을 클릭해도 동일한 명령어 입력 창이 열립니다.



명령어 입력 창을 사용하면 명령어 목록과 명령어에 대한 정보를 알 수 있기 때문에 쉽게 명령을 입력할 수 있습니다. 그러나 명령어에 대한 레지스터 지정과 해당 연산자 지정을 하여야만 명령이 입력됩니다.

다음은 직접 입력 및 명령 입력 창을 통하여 프로그램을 작성한 기본 예입니다.

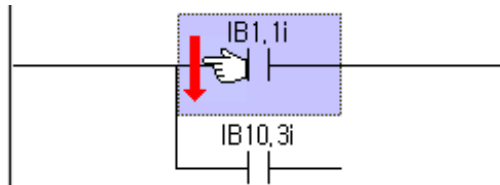


■ 명령 링크 설정 방법

링크를 추가하는 방법은 기능키[F11, F12]를 이용하는 방법과 마우스를 이용하는 방법이 있습니다. 기능키를 이용하여 링크를 추가할 경우는 현재의 셀의 좌측에 링크가 추가됩니다.

다음 그림은 명령 좌측에 링크를 추가하는 그림입니다.

현재의 셀에서 기능키 F12를 누르면 동일한 결과를 얻습니다.



메뉴/단축키	기능키	아이콘	설 명
Horizontal Link	F11	—	수평 연결 링크를 추가
Vertical Link (Up)	F12	⏏	수직 연결 링크를 셀의 왼쪽에 추가

■ 주석 기입 방법

PLC 편집기에는 주석 (설명)이 다음과 같이 4가지가 있습니다. 주석은 항상 녹색으로 표시하여 구별합니다.

◇ 명령어 주석 입력

레지스터 입력 시 주석을 함께 넣을 수 있습니다. 기본 명령어의 경우만 가능하며, 입력 형식은 다음과 같습니다.

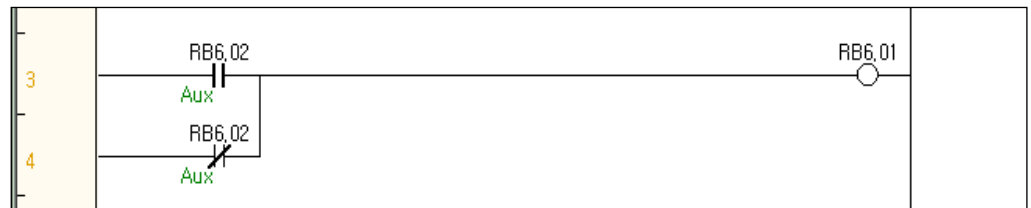
명령어 **인자** ; **주석**

주석과 함께 입력한 레지스터는, 다음과 같이 자동으로 전역 변수 관리창에 등록이 됩니다.

전역 변수			
ID	레지스터	변수명	설명
1	RB6.02		Aux

한번 등록이 된 레지스터는, 다음부터는 주석을 함께 입력하지 않아도 자동으로 주석과 함께 표시됩니다.

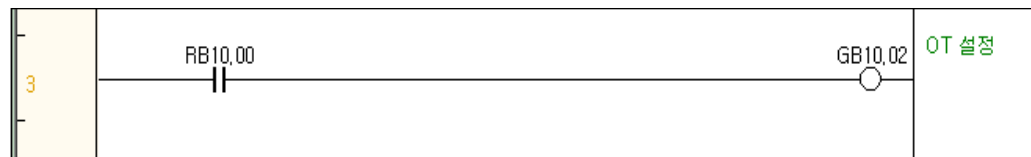
주석을 변경하려면 전역 변수 관리창을 열어 수정하거나, 인플레이스 에디트로 변경할 주석을 넣어주면 됩니다.



◇ 행 주석문 입력

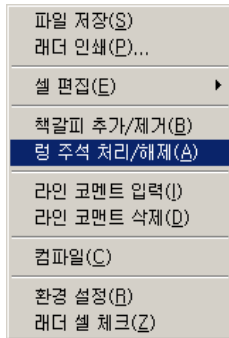
래더 마지막 열은 주석 열입니다.

각 라인의 마지막 셀을 더블클릭 하거나, **Enter** 키를 사용해서 입력 상태로 전환한 후, 설명문을 입력합니다.



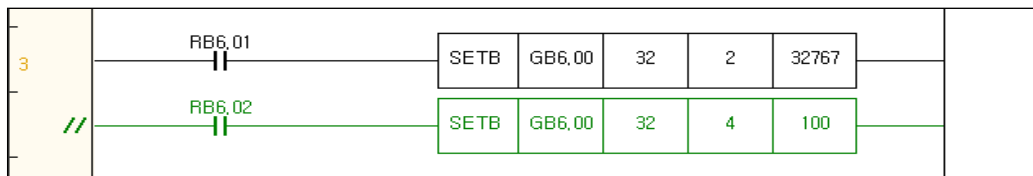
◇ 링 주석 설정/해제

먼저, PLC 편집파일에서 주석으로 처리하고 싶은 링을 선택합니다.



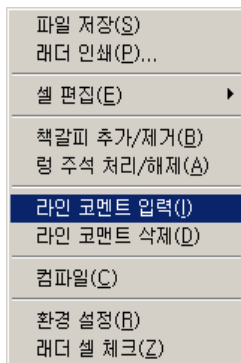
주석으로 처리할 링을 선택한 후, 마우스 오른쪽 버튼을 누르면 왼쪽과 같은 메뉴가 나타납니다. **링 주석 처리/해제**를 선택합니다.

주석으로 처리된 링은 녹색으로 표시가 되며, 컴파일 해도 아무런 영향을 미치지 않습니다. 주석을 해제하려면, 주석 처리할 때와 같이 다시 **링 주석 처리/해제**를 선택하면 토글 되어 복구됩니다.



◇ 라인 코멘트 입력/삭제

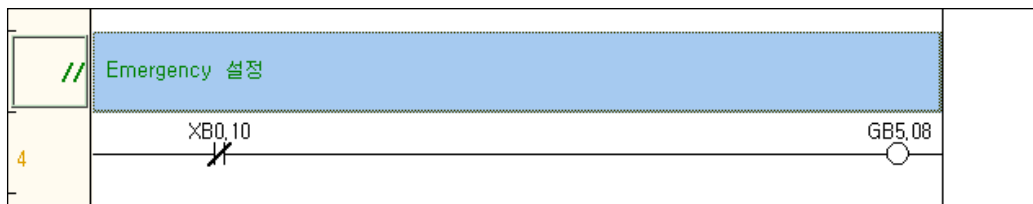
라인 코멘트는 PLC 편집 파일에서 한 행 전체를 사용하여, 설명문으로 사용하기 위한 기능입니다. 작성한 래더에 중요한 설명을 덧붙이거나, 각 라인이나 링 단위로 설명을 덧붙일 수 있습니다.



먼저, PLC 편집파일에서 라인 코멘트를 입력하고 싶은 행을 선택합니다.

라인 코멘트를 입력할 행을 선택한 후, 마우스 오른쪽 버튼을 누르면 왼쪽과 같은 메뉴가 나타납니다. **라인 코멘트 입력**을 선택합니다.

라인 코멘트를 삭제하려면, 라인 코멘트가 입력된 행을 선택하고, 마우스 오른쪽 버튼을 누릅니다. **라인 코멘트 삭제**를 누르면 삭제됩니다



4

일반 명령

GX PLC의 일반 명령에 대해 설명하고 있습니다. 일반 명령에는 일반 시퀀스 명령, 내부 연산 처리 명령이 포함되어 있습니다.

4.1 입출력 명령

4.2 릴레이 회로 스택 연산 명령

4.3 수치 스택 연산 명령

4.1 입출력 명령

4.1.1 RD / RDN (Read / Read Not)

■ [기능]

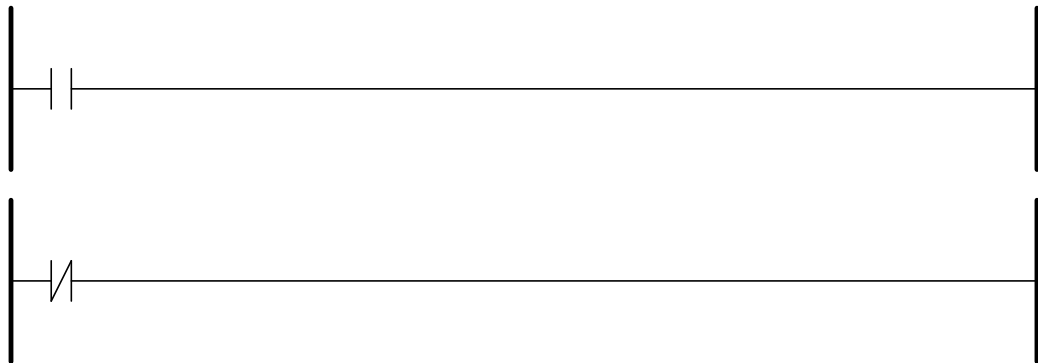
RD : 지정된 비트 레지스터의 값을 읽어서 **Stack**의 첫번째 공간(**Stack 0**)에 기록

RDN : 지정된 비트 레지스터의 값을 읽은 후에 반전해서 **Stack**의 첫번째 공간 (**Stack 0**)에 기록

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	RD/RDN 레지스터 (Bit형)

■ [Ladder]



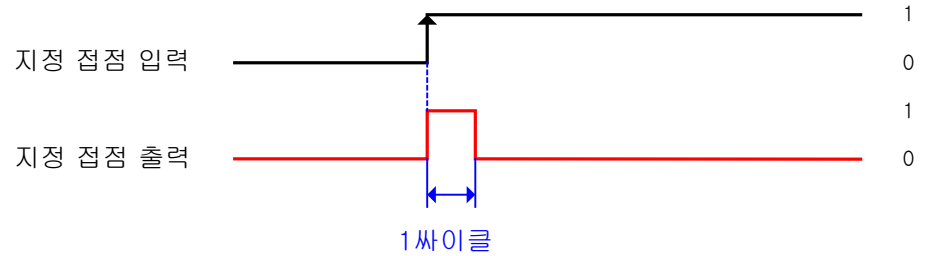
■ [설명]

Operand 1	RD/RDN 할 BIT형 레지스터를 지정합니다.
-----------	----------------------------

4.1.2 RDUP (입력 상승 미분)

■ [기능]

지정한 접점의 상승(Off→On)시에 한 사이클 동안 On 합니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	RDUP 레지스터 (Bit형)

■ [Ladder]



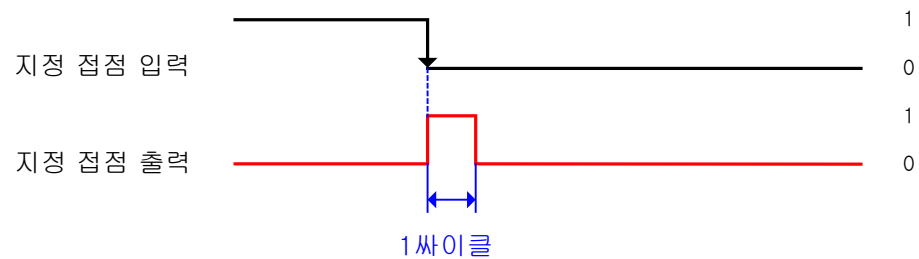
■ [설명]

Operand 1	입력 상승 미분 신호로 사용될 비트 레지스터를 지정
-----------	------------------------------

4.1.3 RDDN (입력 하강 미분)

■ [기능]

지정한 접점의 하강(On→Off)시에 한 사이클 동안 On 합니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	RDDN 레지스터 (Bit형)

■ [Ladder]



■ [설명]

Operand 1	입력 하강 미분 신호로 사용될 비트 레지스터를 지정
-----------	------------------------------

4.1.4 WR / WRN (Write / Write Not)

■ [기능]

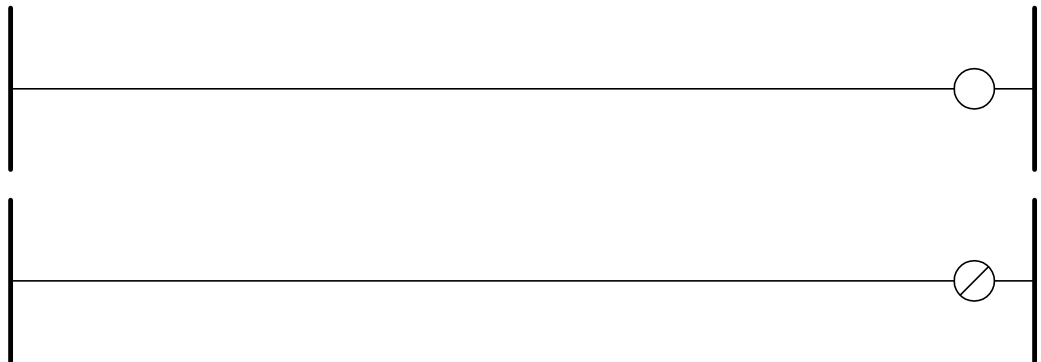
WR : Stack의 첫번째 공간에 기록된 값을 읽어서 지정된 비트 레지스터에 기록

WRN : Stack의 첫번째 공간에 기록된 값을 읽은 후에 반전해서 지정된 비트 레지스터에 기록

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	WR/WRN 레지스터 (Bit형)

■ [Ladder]



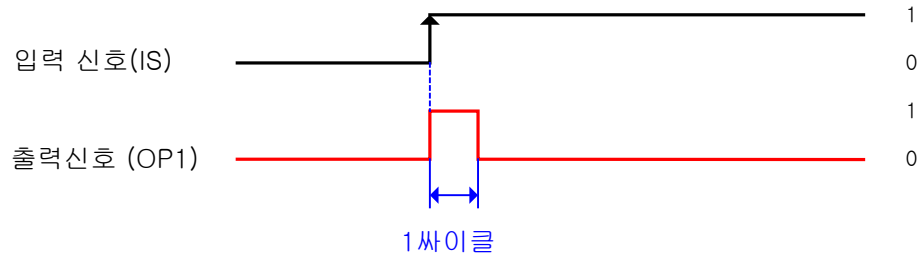
■ [설명]

Operand 1	WR/WRN 할 BIT형 레지스터를 지정합니다.
-----------	----------------------------

4.1.5 WRUP (출력 상승 미분)

■ [기능]

입력신호의 상승(Off→On)시에 지정된 접점을 한 사이클 동안 On 합니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	WRUP 레지스터 (Bit형)

■ [Ladder]



■ [설명]

Operand 1	결과를 출력할 비트 레지스터(출력 접점)를 지정
-----------	----------------------------

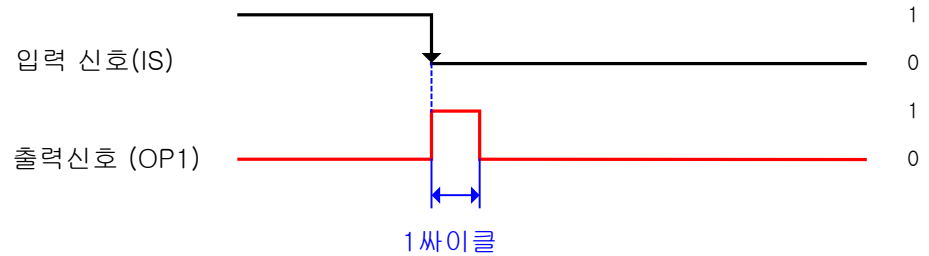
입력신호(입력 접점)를 BIT형 레지스터로 지정해야 합니다.

입력신호가 상승(Off→On)하면, 출력은 1 사이클 동안 On

4.1.6 WRDN (출력 하강 미분)

■ [기능]

입력신호의 하강(On→Off)시에 지정된 접점을 한 사이클 동안 On 합니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	WRDN 레지스터 (Bit형)

■ [Ladder]



■ [설명]

Operand 1	결과를 출력할 비트 레지스터(출력 접점)를 지정
-----------	----------------------------

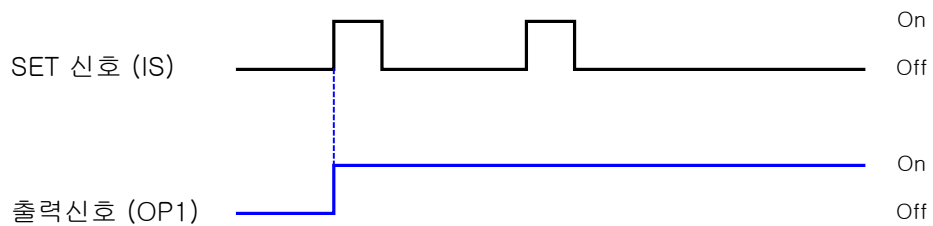
입력신호(입력 접점)를 BIT형 레지스터로 지정해야 합니다.

입력신호가 하강(On→Off)하면, 출력은 1 사이클 동안 On

4.1.7 SET (Set Coil)

■ [기능]

Set 신호가 On 되면 출력 신호는 계속 On으로 유지됩니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	Set 입력 레지스터 (Bit형)

■ [Ladder]



■ [설명]

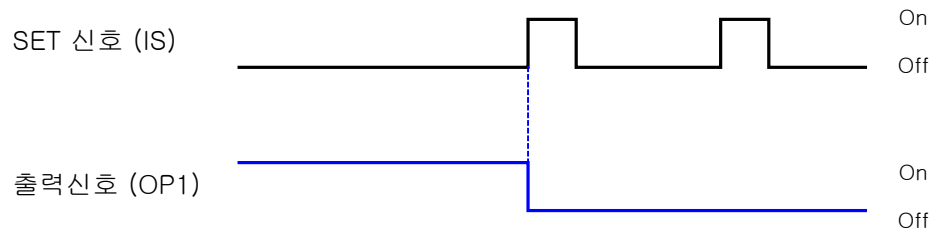
Operand 1	결과를 출력할 비트 레지스터(출력 접점)를 지정
-----------	----------------------------

입력신호(입력 접점)를 BIT형 레지스터로 지정해야 합니다.

4.1.8 RESET (Reset Coil)

■ [기능]

Reset 신호가 On 되면 출력 신호는 계속 Off으로 유지됩니다.



■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	Reset 입력 레지스터 (Bit형)

■ [Ladder]



■ [설명]

Operand 1	결과를 출력할 비트 레지스터(출력 접점)를 지정
-----------	----------------------------

입력신호(입력 접점)를 BIT형 레지스터로 지정해야 합니다.

4.1.9 KEEP (자기 유지)

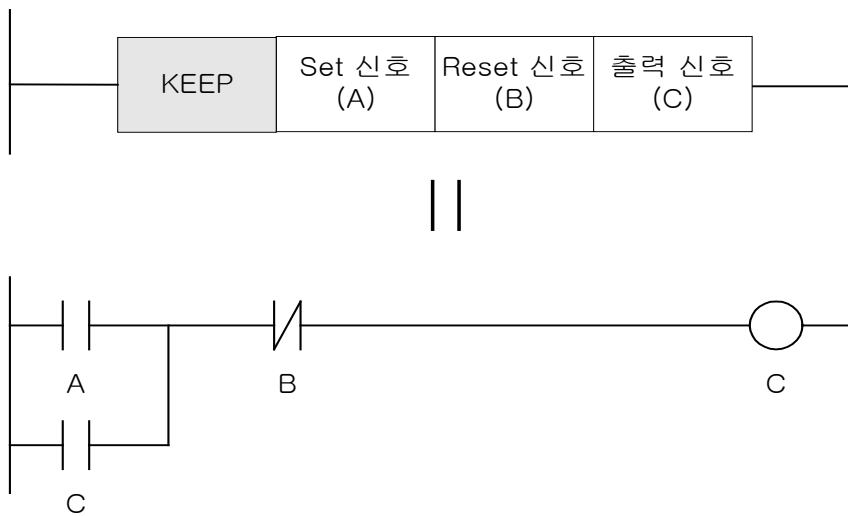
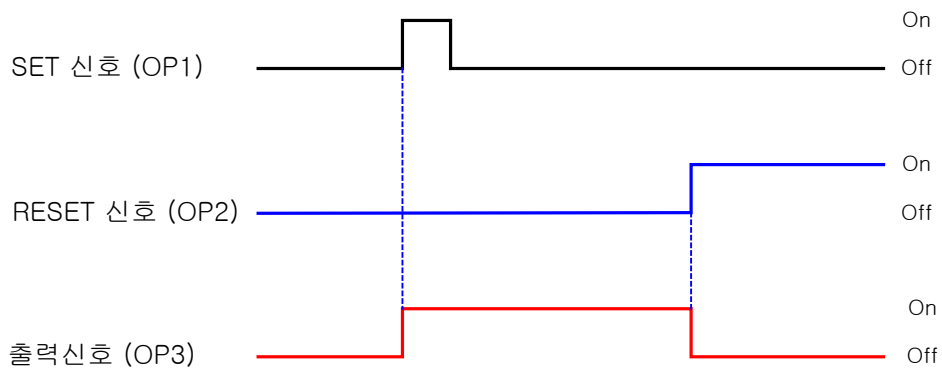
■ [기능]

자기 유지 동작을 합니다.

Set 신호가 한번 입력되면 출력은 계속 On 됩니다.

Reset 신호가 입력되면 출력은 계속 Off 됩니다.

Set과 Reset 신호가 동시에 입력되면 출력은 Off 입니다.



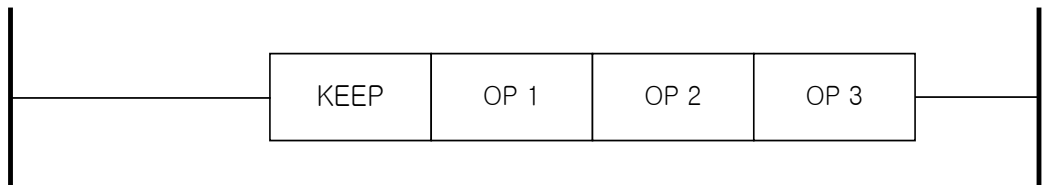
위의 두 Ladder는 완전히 동일합니다

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	Set 신호
OP2	○	-	-	-	-	Reset 신호
OP3	○	-	-	-	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

Operand 1	비트 레지스터(Set 신호)를 지정
Operand 2	비트 레지스터(Reset 신호)를 지정
Operand 3	결과를 출력할 레지스터 (출력 접점)을 지정

4.2 릴레이 회로 스택 연산 명령

4.2.1 AND

- [RD] 명령과 [RD] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.2 ANDN

- [RD] 명령과 [RDN] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.3 ANDUP

- [RD] 명령과 [RDUP] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.4 ANDDN

- [RD] 명령과 [RDDN] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.5 OR

- [RD] 명령과 [RD] 명령의 논리곱 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.6 ORN

- [RD] 명령과 [RDN] 명령의 논리곱 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.2.7 ORUP

- [RD] 명령과 [RDUP] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



4.2.8 ORDN

- [RD] 명령과 [RDDN] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



4.2.9 RDS

- 스택의 내용을 좌로 1 Shift하고 [RD] 명령을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



4.2.10 RDNS

- 스택의 내용을 좌로 1 Shift하고 [RD] 명령을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



4.2.11 RDUPS

- 스택의 내용을 좌로 1 Shift하고 [RDUP] 명령을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



4.2.12 RDDNS

- 스택의 내용을 좌로 1 Shift하고 [RDDN] 명령을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

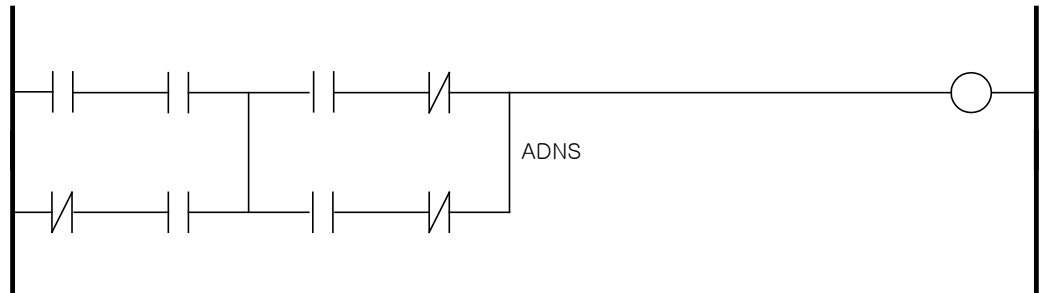
■ [Ladder]



4.2.13 ANDS

- [RD] 명령과 [RD] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

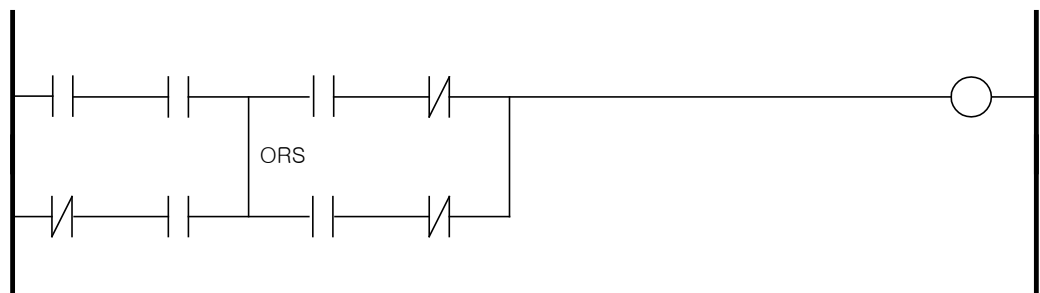
■ [Ladder]



4.2.14 ORS

- [RD] 명령과 [RD] 명령의 논리합 연산을 수행합니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.

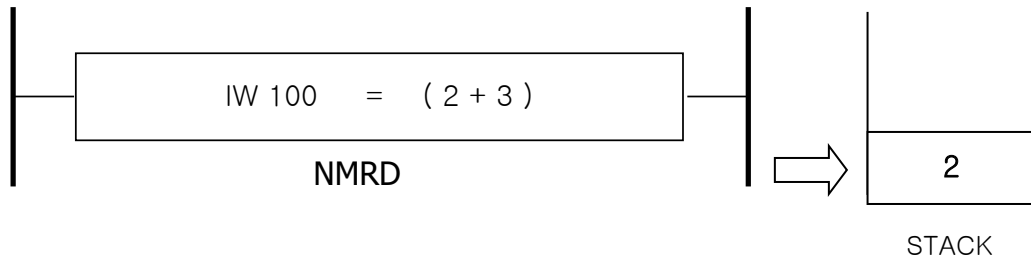
■ [Ladder]



4.3 수치 스택 연산 명령

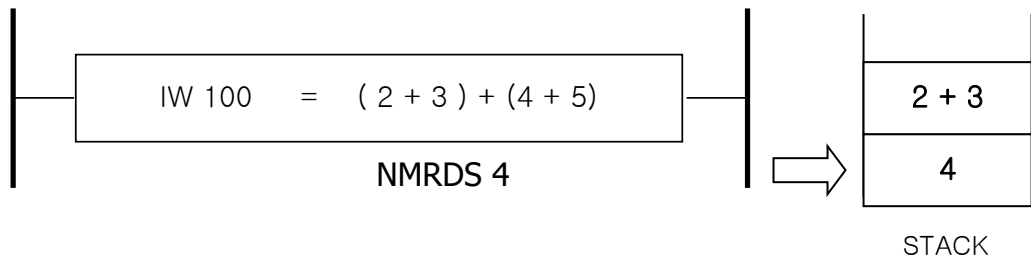
4.3.1 NMRD

- Stack에 처음 수치 값을 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



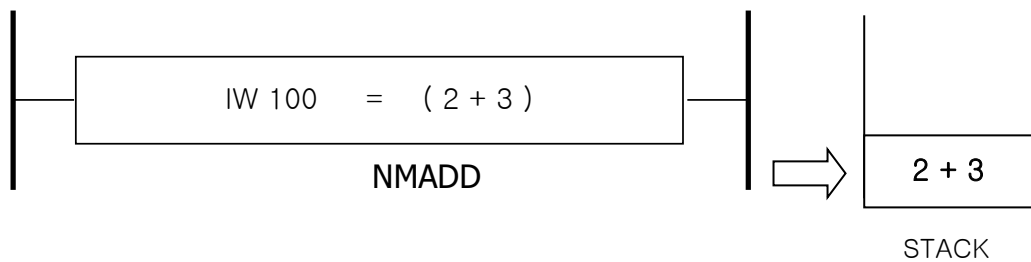
4.3.2 NMRDS

- Stack값을 1 Shift하고 새로운 수치 값을 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.3.3 NMADD

- Stack값과 새로운 수치 값을 더하여 Stack에 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]

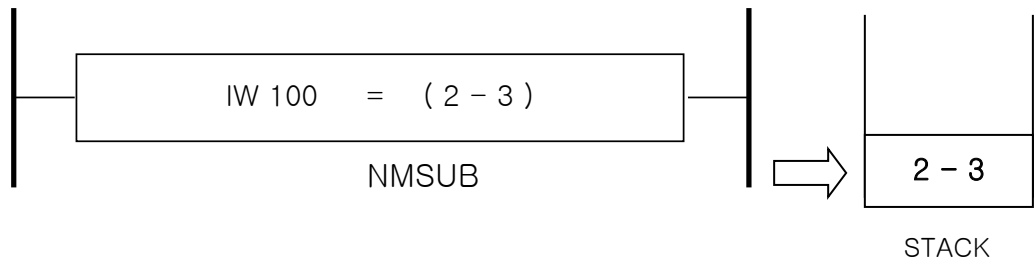


4.3.4 NMSUB

■ Stack값과 새로운 수치 값을 빼서 Stack에 저장하는 명령입니다.

■ 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]

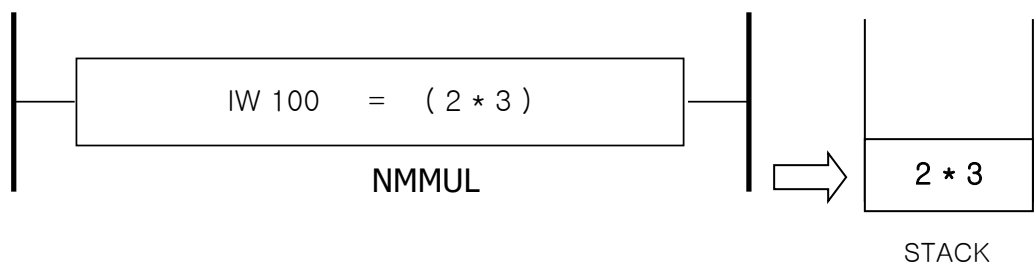


4.3.5 NMMUL

■ Stack값과 새로운 수치 값을 곱해서 Stack에 저장하는 명령입니다.

■ 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]

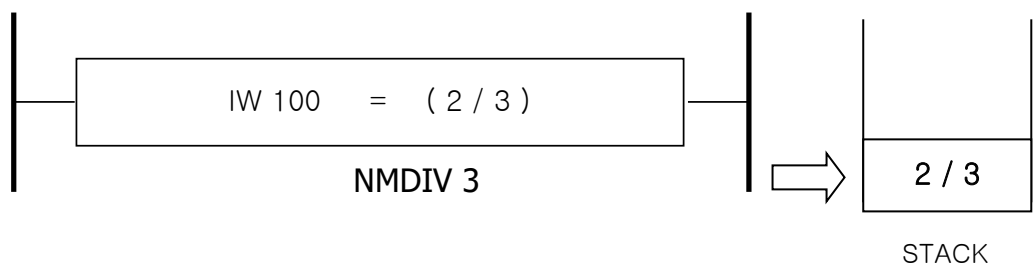


4.3.6 NMDIV

■ Stack값과 새로운 수치 값을 나눠서 Stack에 저장하는 명령입니다.

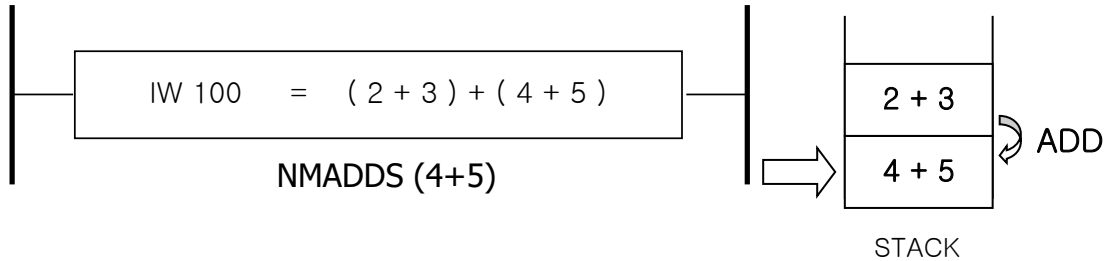
■ 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



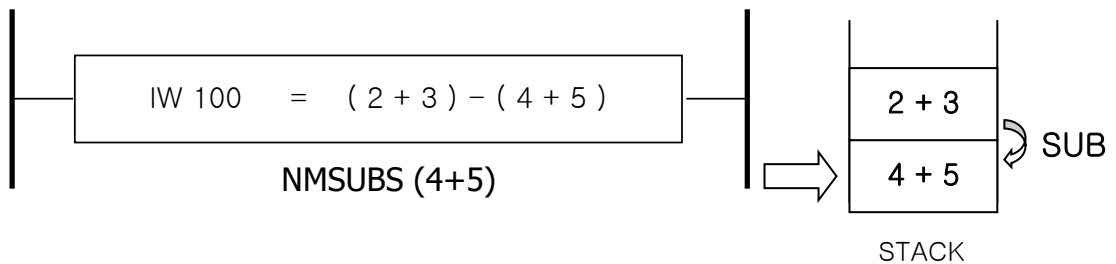
4.3.7 NMADDS

- Stack의 첫번째 값과 두번째 값을 더하여 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



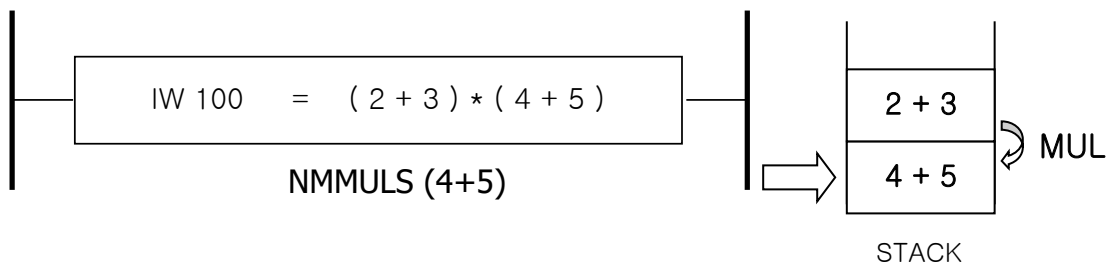
4.3.8 NMSUBS

- Stack의 첫번째 값과 두번째 값을 빼서 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]



4.3.9 NMMULS

- Stack의 첫번째 값과 두번째 값을 곱하여 저장하는 명령입니다.
- 내부 연산 처리 명령이므로 자동 수행됩니다.
- [Ladder]

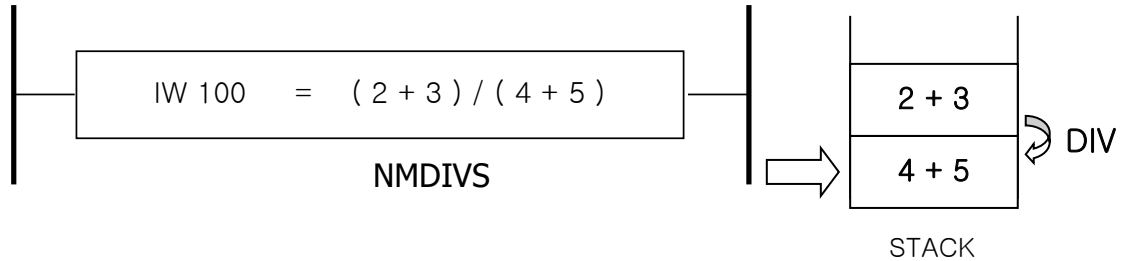


4.3.10 NMDIVS

■ Stack의 첫번째 값과 두번째 값을 나눠서 저장하는 명령입니다.

■ 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]

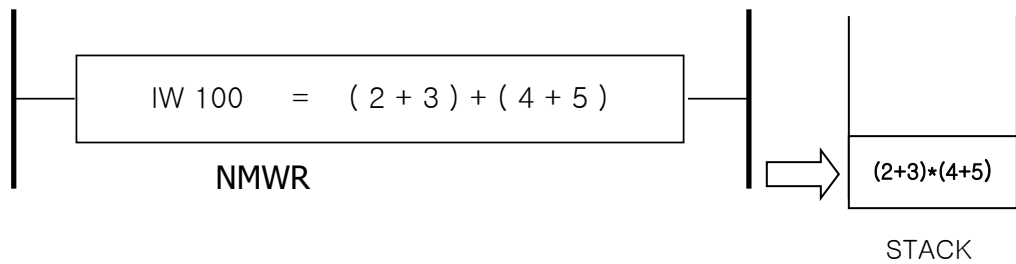


4.3.11 NMWR

■ Stack의 값을 출력하는 명령입니다.

■ 내부 연산 처리 명령이므로 자동 수행됩니다.

■ [Ladder]



5

기능 명령

GX PLC의 다양한 고급 기능을 수행할 수 있는 명령에 대하여 설명하고 있습니다.

5.1 프로그램 제어 명령

5.2 논리 연산 명령

5.3 수치 연산 명령

5.4 수치 비교 명령

5.5 데이터 변환 명령

5.6 데이터 전송 명령

5.7 타이머 / 카운트 명령

5.8 데이터 조작 명령

5.9 Math Function 명령

5.10 테이블 데이터 처리 명령

5.11 데이터 제어 명령

5.12 통신 명령

5.1 프로그램 제어 명령

5.1.1 SCALL (서브 프로그램 호출)

■ [기능]

서브프로그램을 호출하는 명령입니다.

서브프로그램 호출은 2단계 호출까지 가능합니다.

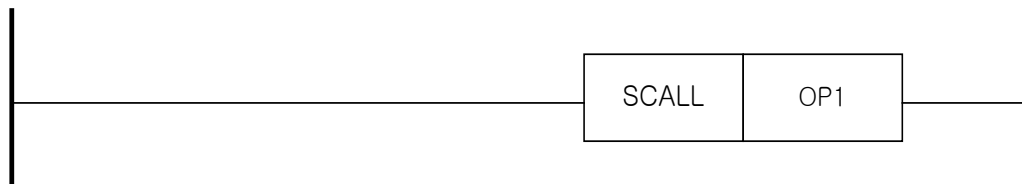
즉, Main 프로그램에서는 1단계로 서브프로그램을 호출할 수 있고, 호출된 서브프로그램에서는 2단계로 서브프로그램을 호출할 수 있습니다. 하지만, 2단계로 호출된 서브프로그램에서는 더 이상 서브 프로그램을 호출 할 수 없습니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	서브 프로그램 이름

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	호출할 서브 프로그램의 이름을 Word형 상수로 입력
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

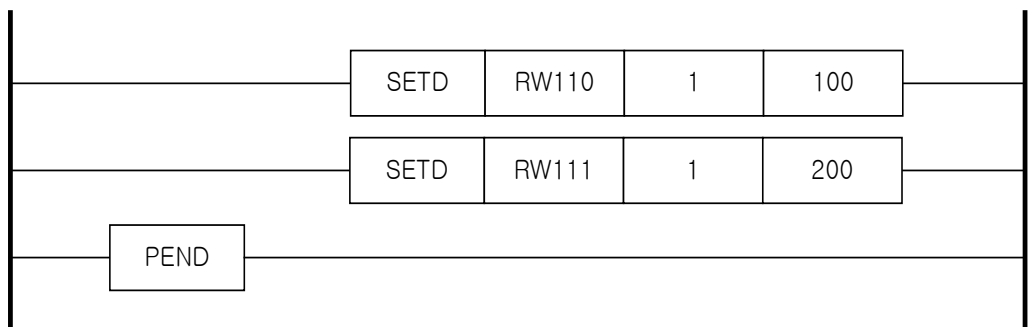
■ [예제]

Normal Main PLC파일에서 서브 프로그램 [Sub-02-01.Plc] 파일을 호출합니다.

[Normal Main PLC]



[Sub-02-01.PLC]



5.1.2 PCALL (모션 프로그램 호출)

■ [기능]

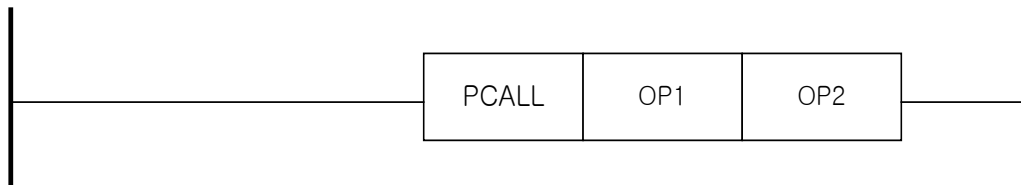
Motion 프로그램을 호출하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	모션 프로그램 채널 번호
OP2	-	-	○	-	-	모션 프로그램 이름

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]

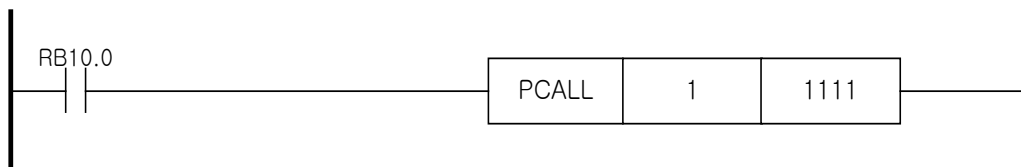


■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	호출할 Motion 프로그램의 Channel 번호를 지정
Operand 2	호출할 Motion 프로그램의 이름을 Word형 상수로 입력
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

1Channel의 [1111] 모션 파일을 호출합니다.



5.1.3 IFON/ IFOFF (조건문)

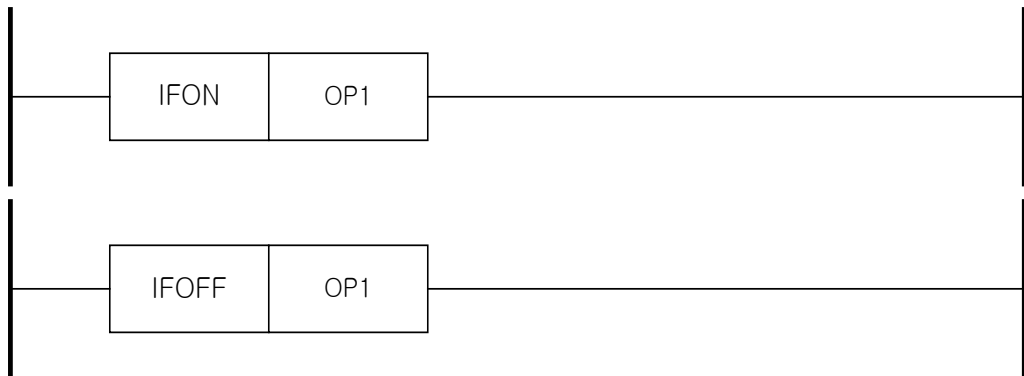
■ [기능]

IFON[IFOFF]는 반드시 IEND(또는 ELSE)와 짝을 이루어야 하며, IFON[IFOFF]와 IEND 사이의 명령어들은 IFON(IFOFF)의 실행조건에 따라서 실행되거나 실행되지 않습니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	조건 비트 입력

■ [Ladder]



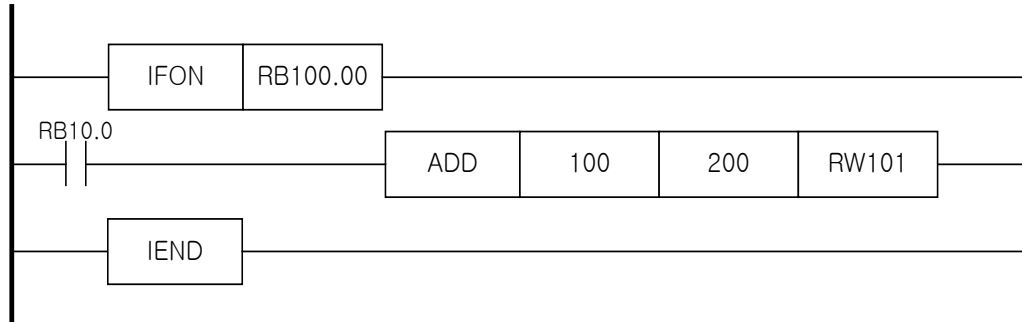
■ [설명]

Operand 1	BIT형 레지스터를 입력
IFON	OP1의 값이 1이면, IFON과 IEND 사이의 명령어들을 실행 OP1의 값이 0이면, IFON과 IEND 사이의 명령어들을 실행하지 않고 IEND 다음으로 건너뛰어 IEND 이후의 명령어를 실행합니다. 짝을 이루는 ELSE가 있을 경우에는 ELSE 다음으로 건너뛰어 ELSE 이후의 명령어를 실행합니다
IFOFF	OP1의 값이 0이면, IFOFF와 IEND 사이의 명령어들을 실행 OP1의 값이 1이면, IFOFF와 IEND 사이의 명령어들을 실행하지 않고 IEND 다음으로 건너뛰어 IEND 이후의 명령어를 실행합니다. 짝을 이루는 ELSE가 있을 경우에는 ELSE 다음으로 건너뛰어 ELSE 이후의 명령어를 실행합니다

■ [예제]

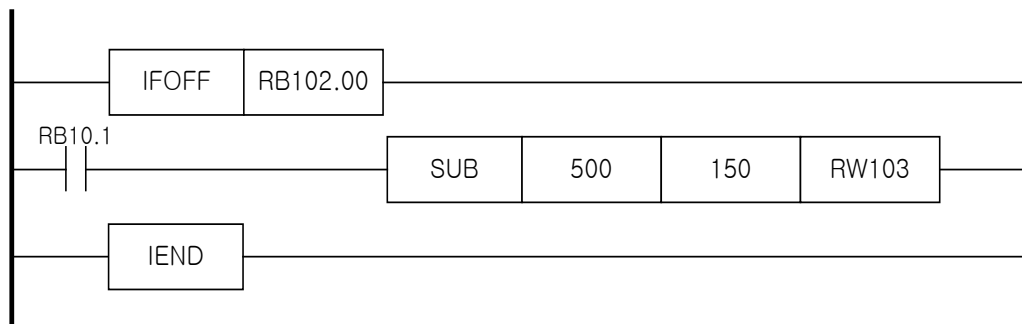
[RB100.00]의 값이 1이면 [ADD] 명령을 수행합니다.

[RB100.00]의 값이 0이면 [ADD] 명령을 수행하지 않고, [IEND] 다음 명령을 수행



[RB102.00]의 값이 0이면 [SUB] 명령 수행합니다.

[RB102.00]의 값이 1이면 [SUB] 명령을 수행하지 않고, [IEND] 다음 명령을 수행

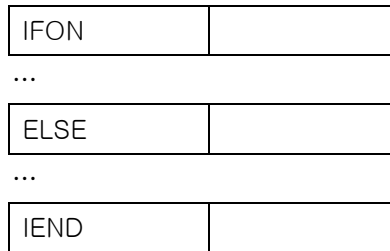


5.1.4 ELSE (조건문)

■ [기능]

반드시 IFOF[IFOFF]와 IEND 사이에 사용되어야 합니다.

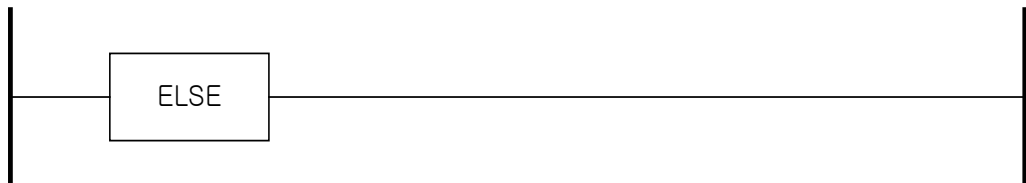
IFON의 OP1의 0일 때[IFOFF의 OP1이 1일 때] ELSE와 IEND 사이의 명령어가 실행됩니다



■ [형식]

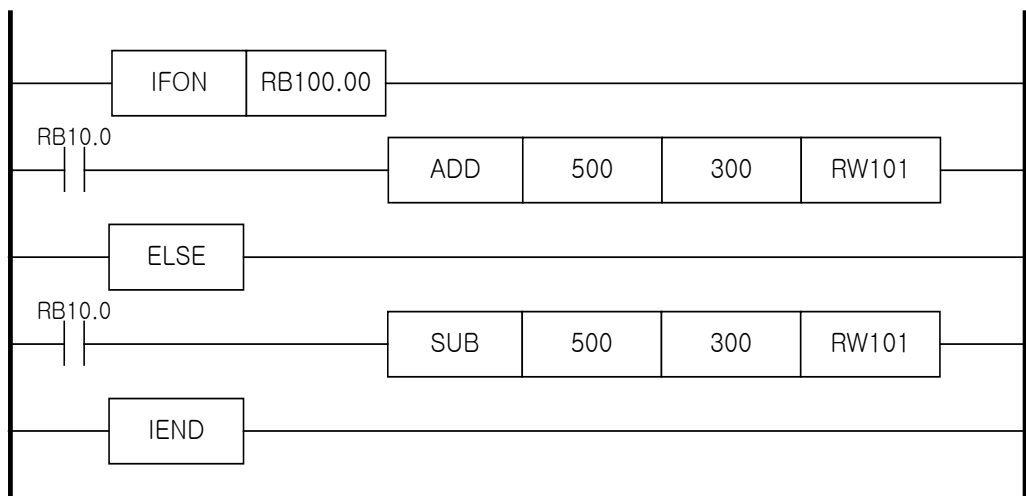
	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



■ [예제]

[RB100.00]의 값이 1이면 [ADD] 명령을 수행, 0이면 [SUB] 명령을 수행합니다.



5.1.5 IEND (조건문)

■ [기능]

IFON[IFOFF] 문의 끝을 표시합니다.

반드시 IFON[IFOFF]와 짝을 이루어야 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



5.1.6 FOR (반복문)

■ [기능]

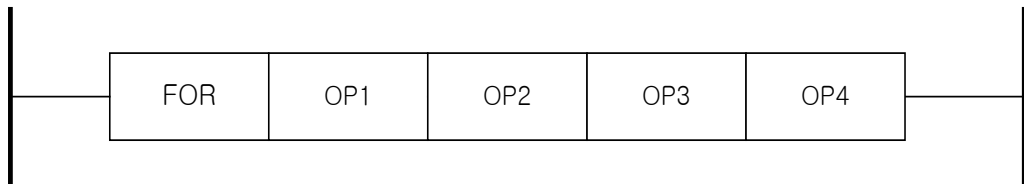
FOR 루프의 시작을 표시합니다.

지정한 레지스터(OP1)의 값이 최대값(OP3)보다 클 때까지 FOR와 FEND 사이의 명령어들을 반복 실행 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	현재 반복값 저장 레지스터
OP2	-	○	○	-	-	FOR문 초기값
OP3	-	○	○	-	-	FOR문 최대값
OP4	-	○	○	-	-	FOR문 증가값

■ [Ladder]



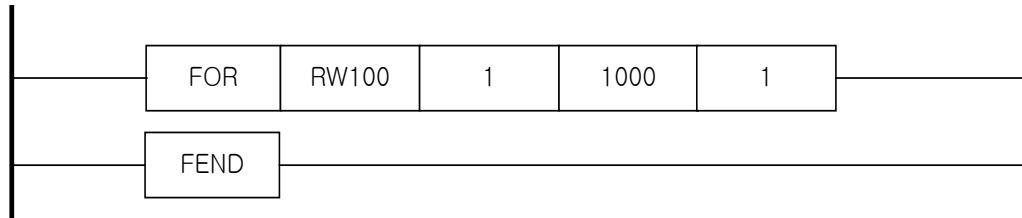
■ [설명]

Operand 1	FOR의 값을 저장할 Word형 레지스터를 지정합니다. 이 값이 지정한 최대값보다 크면(OP1 > OP3 이면) FOR 루프를 종료하고, FEND 이후의 명령어들을 실행합니다. OP1 <= OP3 이면, FOR와 FEND 사이의 명령어들을 실행하고, OP1을 OP4만큼 증가시킨 후에 다시 OP1과 OP3의 크기를 비교합니다.
Operand 2	FOR가 최초로 실행될 때 FOR의 값은 초기화 됩니다. 이때의 초기값을 설정합니다.
Operand 3	FOR의 최대값을 설정합니다. FOR의 값이 OP3보다 크면 (OP1 > OP3) FOR 루프는 종료됩니다.
Operand 4	FOR가 한 루프 회전을 마치면, FOR의 값(OP1)은 일정 값 만큼 증가합니다. OP4는 이 증가 값을 설정합니다.

■ [예제]

[FOR]문부터 [FEND]문까지 1000번 수행합니다.

[RW100]의 값이 1부터 1000까지 1만큼씩 증가될 때 까지 수행합니다.



5.1.7 FEND (반복문)

■ [기능]

FOR 루프의 끝을 표시합니다.

반드시 FOR와 짝을 이루어야 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



5.1.8 WHILEON/ WHILEOFF (반복문)

■ [기능]

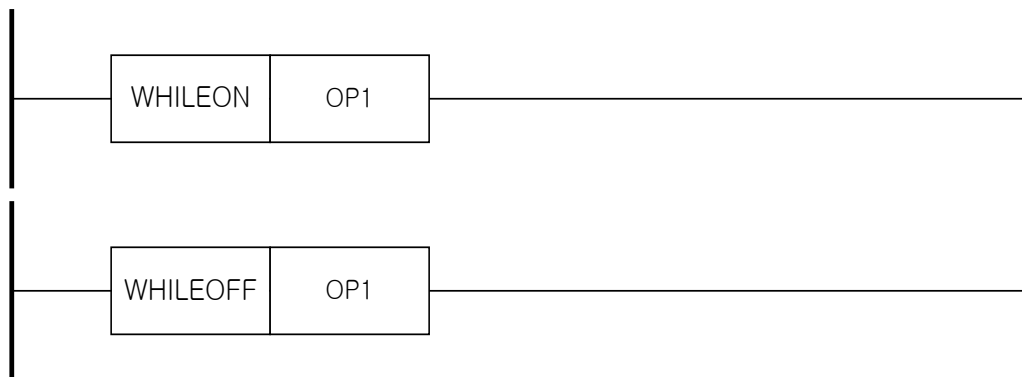
WHILEON: 반복루프. OP1의 값이 1 이면, WHILE 루프를 반복 실행합니다.

WHILEOFF: 반복루프. OP1의 값이 0 이면, WHILE 루프를 반복 실행합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	조건 비트 입력

■ [Ladder]



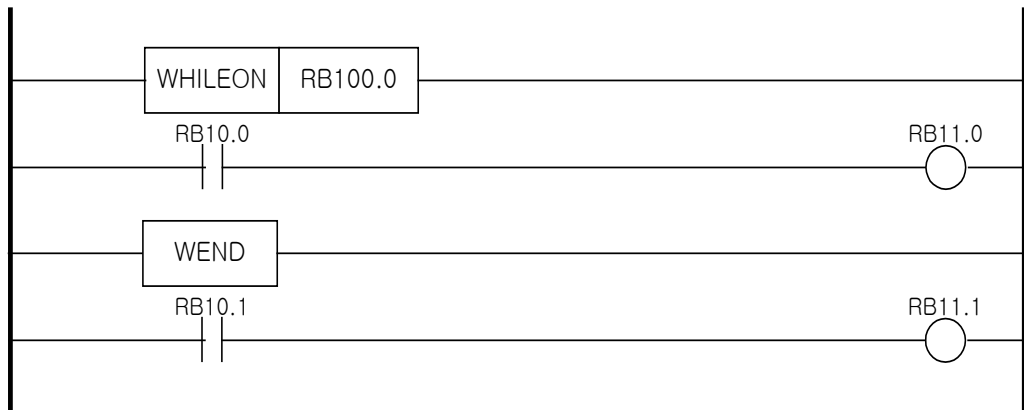
■ [설명]

Operand 1	비교조건
WHILEON	OP1의 값이 0이면 WHILE 루프를 종료하고, WEND 이후의 명령어를 실행. OP1의 값이 1이면 WHILE ON과 WEND 사이의 명령어들을 실행한 후에 다시 OP1의 값을 비교
WHILEOFF	OP1의 값이 1이면 WHILE 루프를 종료하고 WEND 이후의 명령어를 실행. OP1의 값이 0이면 WHILE OFF와 WEND 사이의 명령어들을 실행한 후에 다시 OP1의 값을 비교

■ [예제]

[RB100.0]의 값이 0인 경우, [WHILEON] 명령부터 [WEND] 명령까지 수행하지 않고 [WEND] 다음 명령을 수행합니다.

[RB100.0]의 값이 1인 경우, [WHILEON] 명령부터 [WEND] 명령만 반복 수행합니다.



5.1.9 WEND (반복문)

■ [기능]

WHILEON[WHILEOFF]의 끝을 표시합니다.

반드시 WHILEON[WHILEOFF]와 짝을 이루어야 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



5.1.10 BREAK (BREAK 문)

■ [기능]

반복 루프를 빠져나가는 명령어 입니다.

1. FOR와 FEND 사이에 BREAK를 사용할 경우

- FEND로 점프하여, FEND 이후의 명령어를 실행 합니다. 이로써, 반복 루프는 종료됩니다.

2. WHILE ON[WHILE OFF]와 WEND 사이에 BREAK를 사용할 경우

- WEND로 점프하여, WEND 이후의 명령어를 실행 합니다. 이로써, 반복 루프는 종료됩니다.

3. SWITCH-CASE 에서 BREAK를 사용할 경우

- SWEND로 점프하여, SWEND 이후의 명령어를 실행 합니다.

■ [형식]

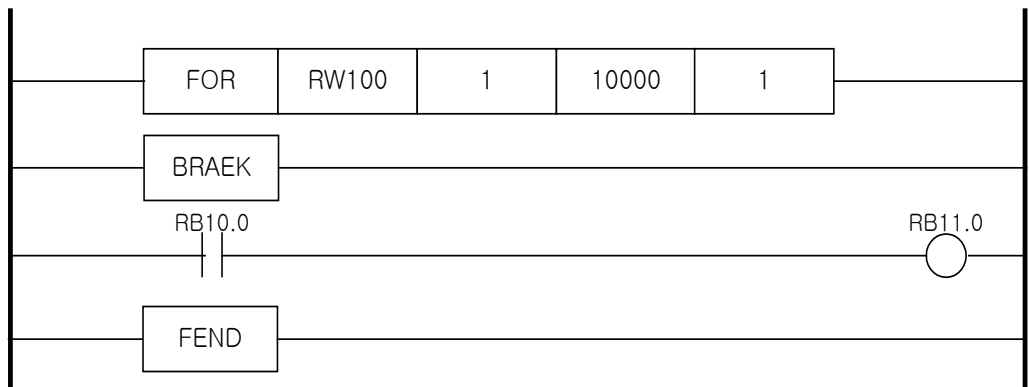
	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



■ [예제]

[FOR]문 수행하는 도중 [RB10.0]이 ON되면 [FEND] 다음 명령으로 이동합니다.



5.1.11 GOTO (점프 명령)

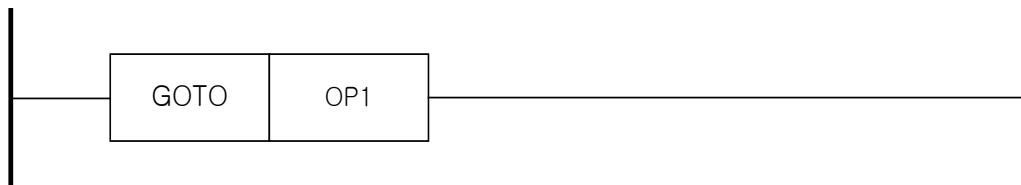
■ [기능]

특정 위치로 점프하는 명령어 입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	점프할 Label 번호

■ [Ladder]



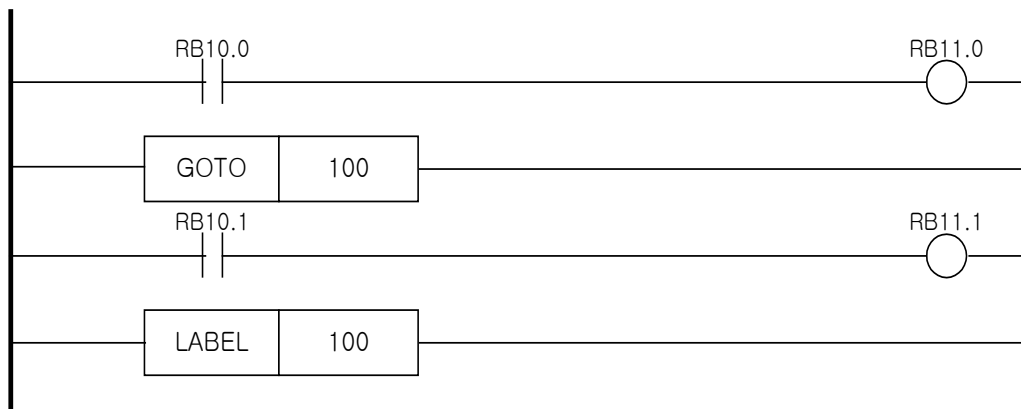
■ [설명]

Operand 1	점프할 Label의 번호를 입력합니다.
-----------	-----------------------

■ [예제]

[RB10.2]가 OFF 이면 [RD RB10.1] 명령을 수행합니다.

[RB10.2]가 ON 이면 [GOTO] 명령을 수행하여 [LABEL] 다음 명령으로 이동합니다.



5.1.12 LABEL (라벨 표시)

■ [기능]

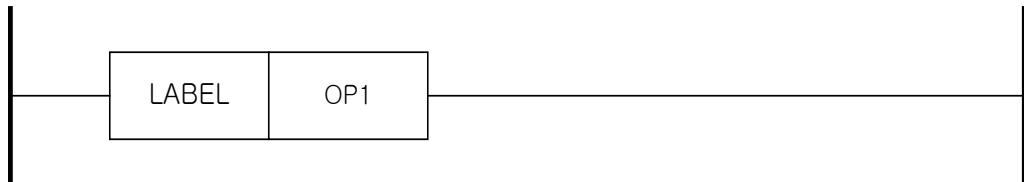
특정 위치를 표시하는 명령어 입니다.

주로, GOTO와 함께 사용합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	Label 번호

■ [Ladder]



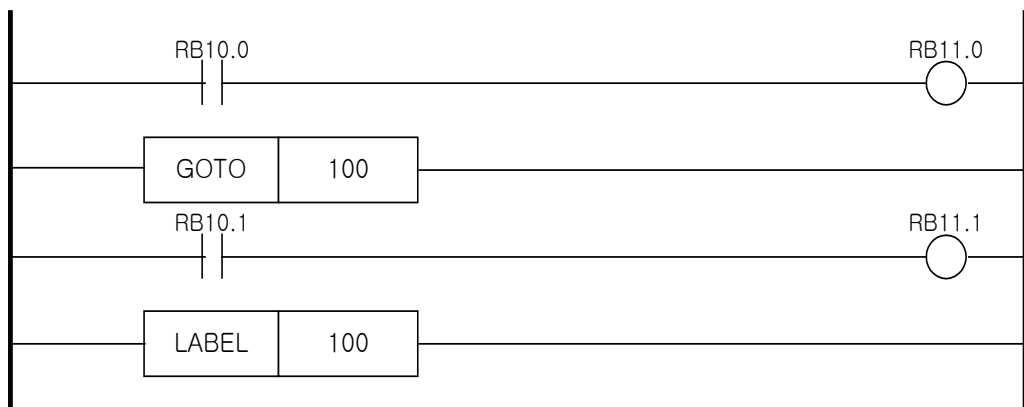
■ [설명]

Operand 1	LABEL의 번호를 지정합니다.
-----------	-------------------

■ [예제]

[RB10.2]가 OFF 이면 [RD RB10.1] 명령을 수행합니다.

[RB10.2]가 ON 이면 [GOTO] 명령을 수행하여 [LABEL] 다음 명령으로 이동합니다.



5.1.13 PEND (프로그램 끝)

■ [기능]

프로그램의 끝을 표시합니다. 모든 PLC 파일은 PEND로 끝나야 합니다.

PEND 이후의 명령어들은 모두 무시됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



■ [예제]

프로그램의 끝을 나타냅니다. [PEND] 이후의 명령은 무시합니다.



5.1.14 STOP (프로그램 정지)

■ [기능]

프로그램을 정지합니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



■ [예제]

[RB10.0]이 ON이면 프로그램을 정지합니다.



5.1.15 SWITCH (SWITCH-CASE문)

■ [기능]

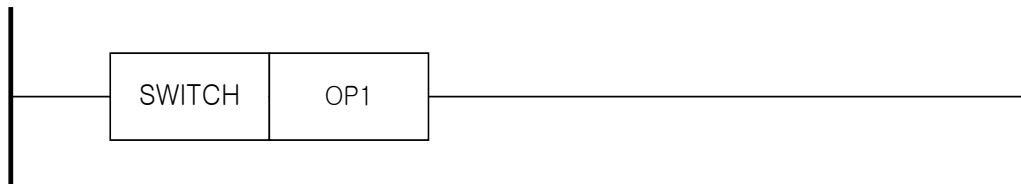
지정한 조건 값에 맞는 CASE문만 수행하는 조건 명령어 입니다.

SWITCH, CASE, DEFAULT, BREAK, SWEND문으로 구성하여 특정한 조건을 조합하여 사용할 수 있습니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	조건 레지스터

■ [Ladder]

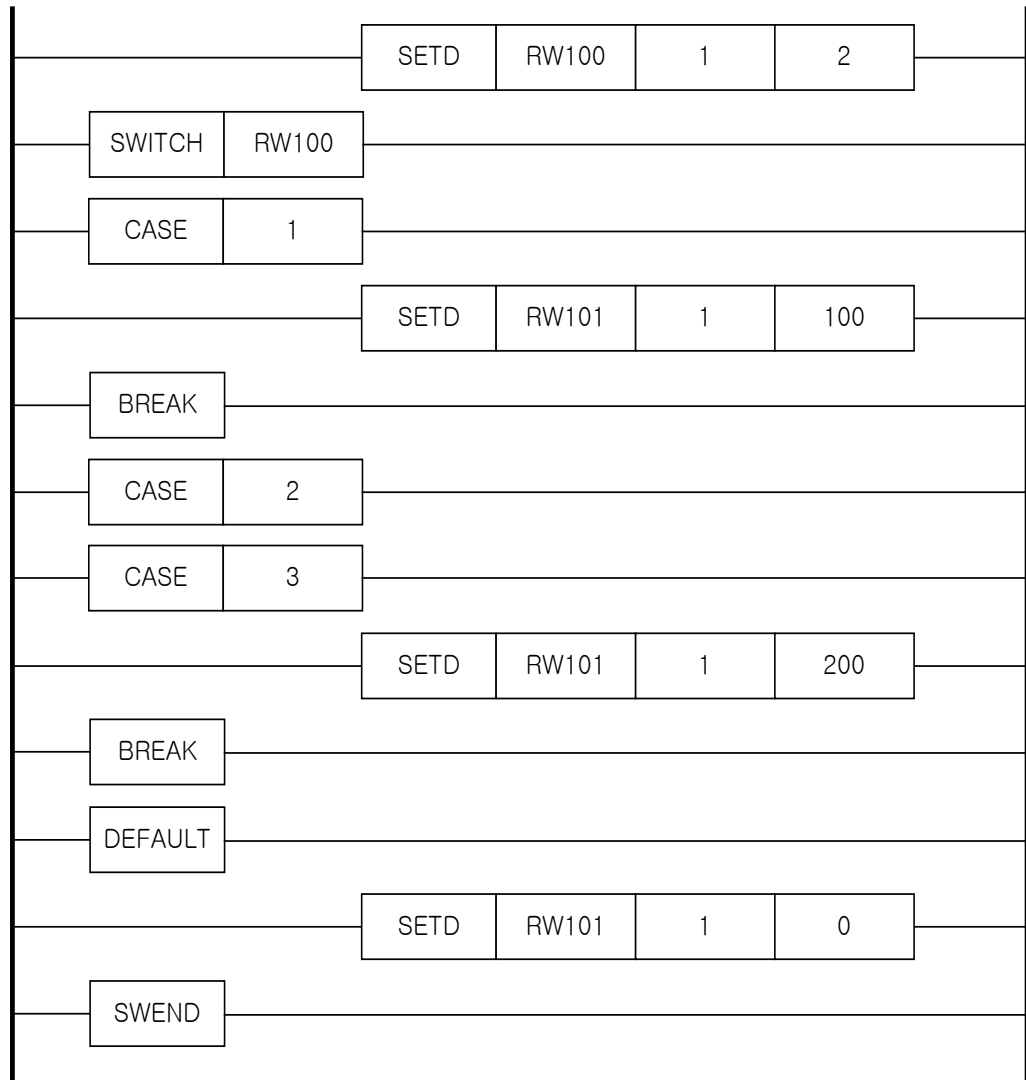


■ [설명]

Operand 1	Case문과 비교할 조건 값이 저장된 레지스터를 입력합니다.
-----------	-----------------------------------

■ [예제]

[RW100]에 2 가 저장되므로 CASE문 중에 조건 2 와 같은 명령만 수행되어 [RW101]에는 200 이 저장됩니다.



5.1.16 CASE (SWITCH-CASE문)

■ [기능]

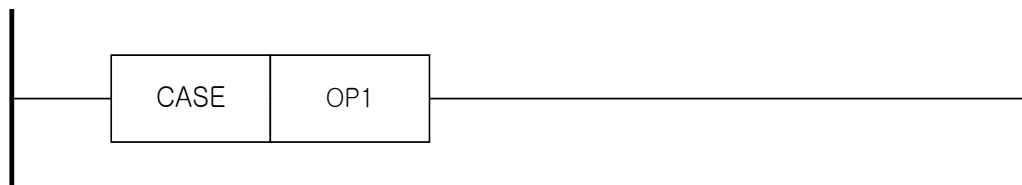
지정한 조건 값에 맞는 **CASE**문만 수행하는 조건 명령어 입니다.

SWITCH – SWEND 명령 안에서만 사용할 수 있습니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	—	—	○	—	—	비교값

■ [Ladder]



■ [설명]

Operand 1	Switch 조건과 비교할 값을 입력합니다.
-----------	--------------------------

5.1.17 DEFAULT (SWITCH-CASE문)

■ [기능]

지정한 조건 값에 맞는 **CASE**문이 없는 경우 수행하는 조건 명령어 입니다.

SWITCH – SWEND 명령 안에서만 사용할 수 있습니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



5.1.18 SWEND (SWITCH-CASE문)

■ [기능]

SWITCH-CASE문의 끝을 표시합니다.

반드시 SWITCH와 짝을 이루어야 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
No Operand	-	-	-	-	-	

■ [Ladder]



5.2 논리 연산 명령

5.2.1 ANDMOV (논리곱 이동)

■ [기능]

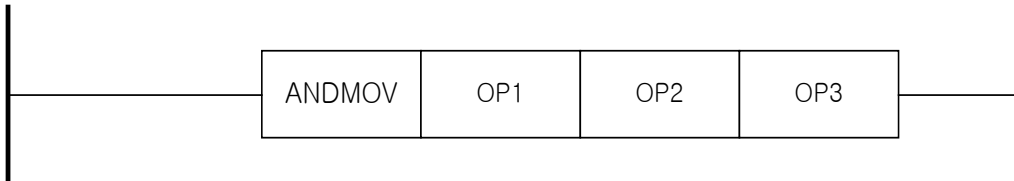
두 값을 논리곱(AND)하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	논리곱 연산 인자 1
OP2	-	○	○	-	-	논리곱 연산 인자 2
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

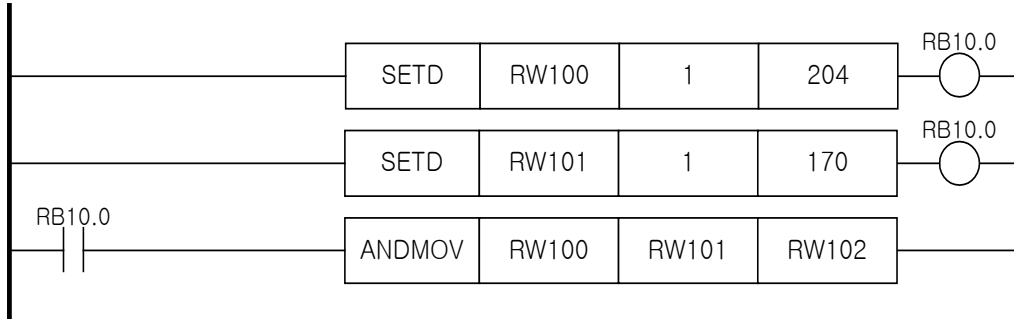
실행 조건	명령 실행 조건을 삼입 가능
Operand 1	논리곱 연산을 할 첫번째 값을 지정
Operand 2	논리곱 연산을 할 두번째 값을 지정
Operand 3	OP1 과 OP2의 논리곱 연산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

<진리표>

입력1	입력2	출력
0	0	0
0	1	0
1	0	0
1	1	1

■ [예제]

[RW100]과 [RW101] 데이터 값의 [ANDMOV] 명령 수행 결과를 [RW102]에 저장
[RW102]의 결과값은 136입니다.



5.2.2 ORMOV (논리합 이동)

■ [기능]

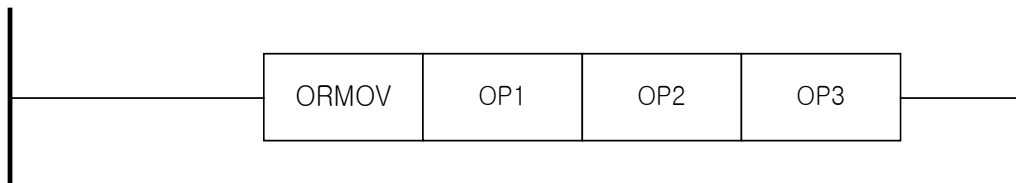
두 값을 논리합(OR)하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	논리합 연산 인자 1
OP2	-	○	○	-	-	논리합 연산 인자 2
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

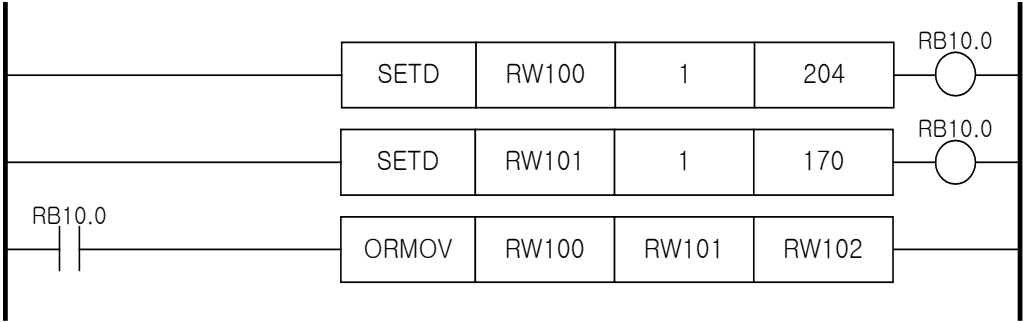
실행 조건	명령 실행 조건을 삼입 가능
Operand 1	논리합 연산을 할 첫번째 값을 지정
Operand 2	논리합 연산을 할 두번째 값을 지정
Operand 3	OP1 과 OP2의 논리합 연산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

<진리표>

입력1	입력2	출력
0	0	0
0	1	1
1	0	1
1	1	1

■ [예제]

[RW100]과 [RW101] 데이터 값의 [ORMOV] 명령 수행 결과를 [RW102]에 저장
[RW102]의 결과값은 238입니다.



5.2.3 XORMOV (배타적 논리합 이동)

■ [기능]

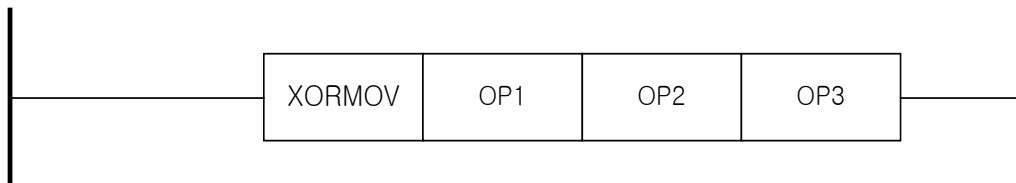
두 값을 배타적논리합(XOR)하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	배타적 논리합 연산 인자 1
OP2	-	○	○	-	-	배타적 논리합 연산 인자 2
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	배타적논리합 연산을 할 첫번째 값을 지정
Operand 2	배타적논리합 연산을 할 두번째 값을 지정
Operand 3	OP1과 OP2의 배타적논리합 연산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

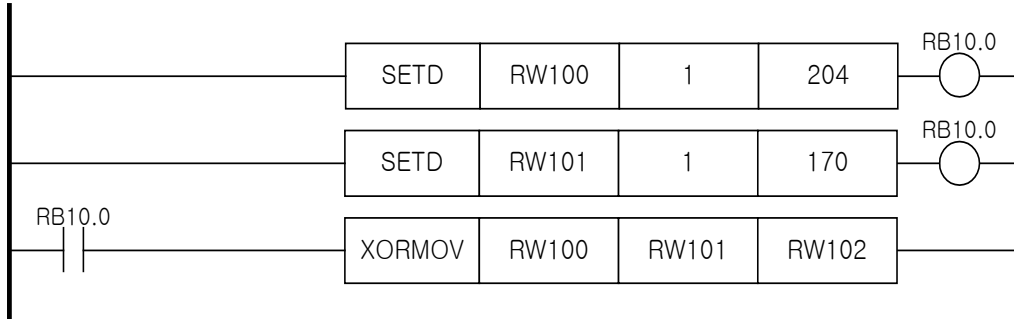
<진리표>

두개의 입력들이 서로 다르면 출력은 1이 되고, 두 개의 입력이 서로 같으면 출력은 0이 됩니다.

입력1	입력2	출력
0	0	0
0	1	1
1	0	1
1	1	0

■ [예제]

[RW100]과 [RW101] 데이터 값의 [XORMOV] 명령 수행 결과를 [RW102]에 저장
[RW102]의 결과값은 90입니다.



5.3 수치 연산 명령

5.3.1 ADD (더하기)

■ [기능]

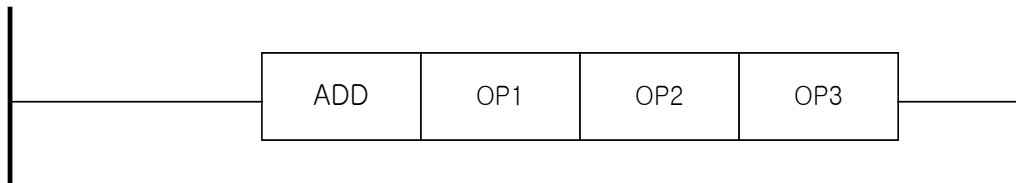
두 값을 합하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	덧셈 할 인자 1
OP2	-	○	○	○	○	덧셈 할 인자 2
OP3	-	○	-	○	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



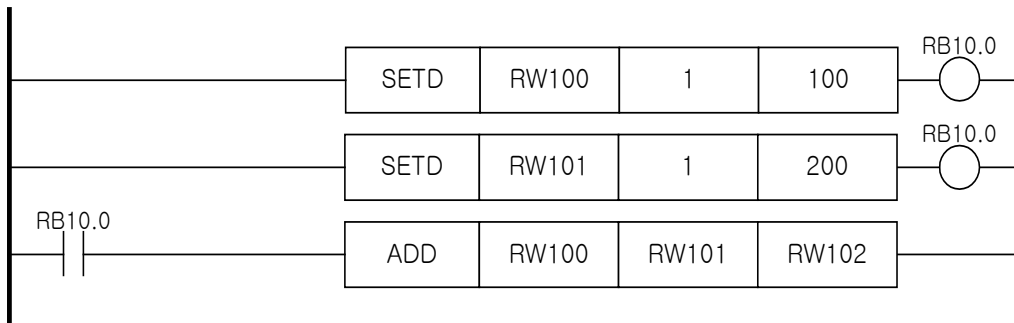
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	덧셈 할 첫 번째 값을 지정
Operand 2	덧셈 할 두 번째 값을 지정
Operand 3	연산 결과를 저장할 레지스터를 지정 (OP1 + OP2) 의 값을 OP3에 저장
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터와 [RW101]의 데이터를 합하여 [RW102]에 저장합니다.

[RW102] 레지스터에 300이 저장됩니다.



5.3.2 SUB (빼기)

■ [기능]

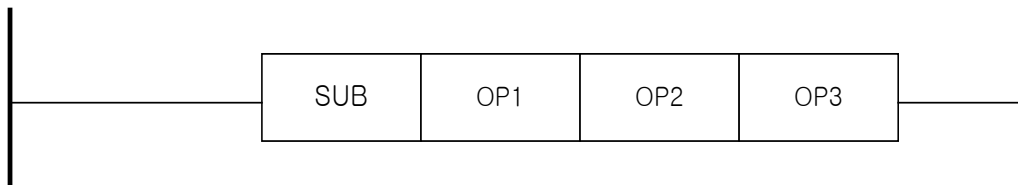
두 값을 뺄셈하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

		B	W	CW	F	CF	내 용
OP1		-	○	○	○	○	뺄셈 할 인자 1
OP2		-	○	○	○	○	뺄셈 할 인자 2
OP3		-	○	-	○	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



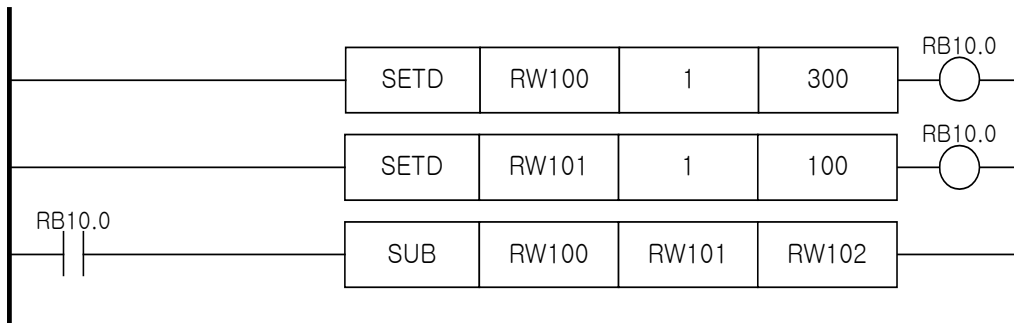
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	뺄셈 할 첫 번째 값을 지정
Operand 2	뺄셈 할 두 번째 값을 지정
Operand 3	연산 결과를 저장할 레지스터를 지정 (OP1 - OP2) 의 값을 OP3에 저장
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터에서 [RW101]의 데이터를 빼서 [RW102]에 저장합니다.

[RW102] 레지스터에 200이 저장됩니다.



5.3.3 MUL (곱하기)

■ [기능]

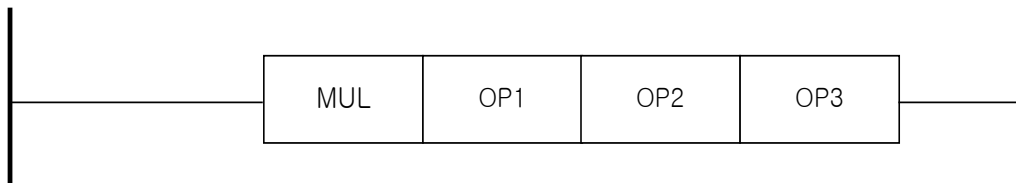
두 값을 곱하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	곱셈 할 인자 1
OP2	-	○	○	○	○	곱셈 할 인자 2
OP3	-	○	-	○	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



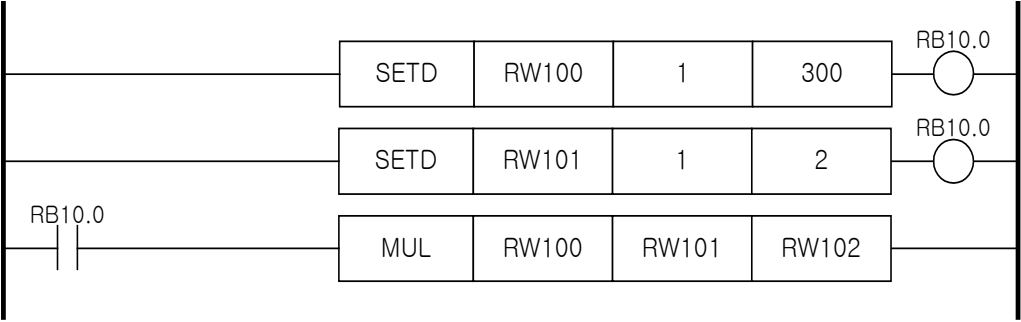
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	곱셈 할 첫 번째 값을 지정
Operand 2	곱셈 할 두 번째 값을 지정
Operand 3	연산 결과를 저장할 레지스터를 지정 (OP1 * OP2) 의 값을 OP3에 저장
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터와 [RW101]의 데이터를 곱하여 [RW102]에 저장합니다.

[RW102] 레지스터에 600이 저장됩니다.



5.3.4 DIV (나누기)

■ [기능]

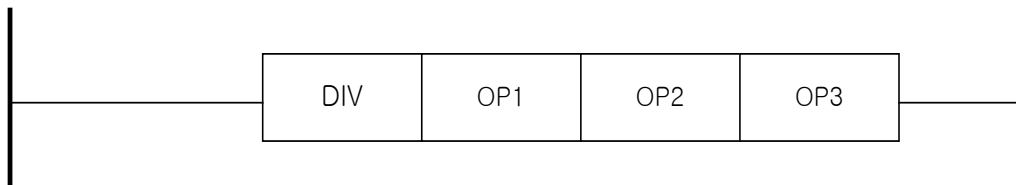
두 값을 나눴셈하여 그 결과를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	나눴셈 할 인자 1
OP2	-	○	○	○	○	나눴셈 할 인자 2
OP3	-	○	-	○	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



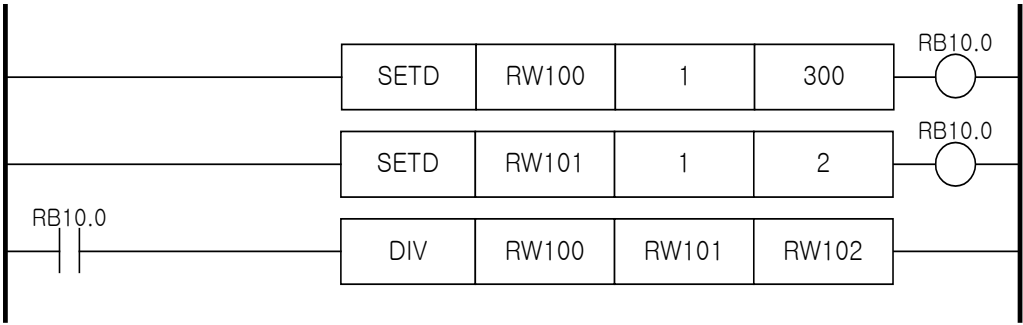
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	나눴셈 할 첫 번째 값을 지정 (피제수 지정)
Operand 2	나눴셈 할 두 번째 값을 지정 (제수 지정)
Operand 3	연산 결과를 저장할 레지스터를 지정 (OP1 / OP2) 의 값을 OP3에 저장 OP3에 지정한 레지스터의 타입이 정수이면, 나눴셈 결과의 정수 부분만 저장이 됩니다. OP3에 지정한 레지스터의 타입이 실수이면, 나눴셈 결과가 실수로 저장
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터를 [RW101]의 데이터로 나누어 [RW102]에 저장합니다.

[RW102] 레지스터에 150이 저장됩니다.



5.3.5 MOD (나머지)

■ [기능]

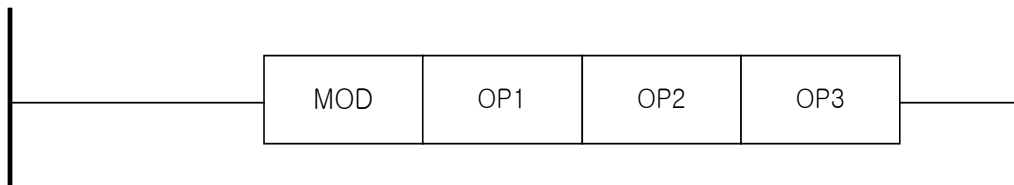
두 값을 나눴셈하여 그 나머지를 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	피제수
OP2	-	○	○	-	-	제수
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



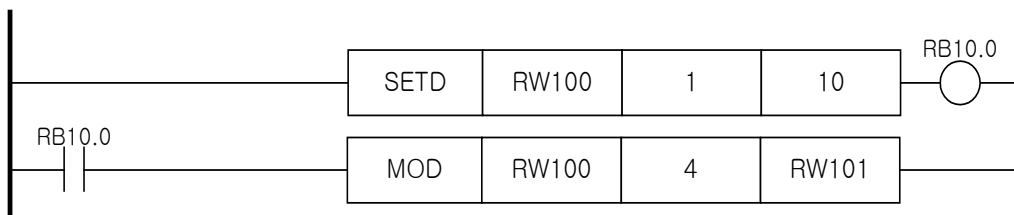
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	피제수 지정
Operand 1	제수 지정
Operand 1	나머지를 저장할 레지스터를 지정 (OP1 / OP2) 의 나머지를 OP3에 저장
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 값을 4로 나눈 나머지를 [RW101] 레지스터에 저장합니다.

[RW102] 레지스터에 2가 저장됩니다.



5.3.6 INC (INCREMENT 명령)

■ [기능]

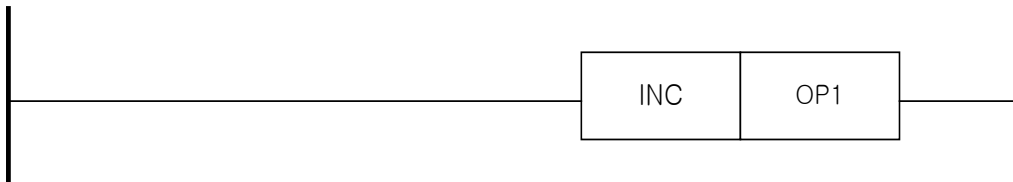
지정한 레지스터의 값을 1만큼 증가시킵니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	증가 값을 저장할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	값을 1만큼 증가시킬 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON이면 [RW100]이 1만큼 증가합니다.



5.3.7 DEC (DECREMENT 명령)

■ [기능]

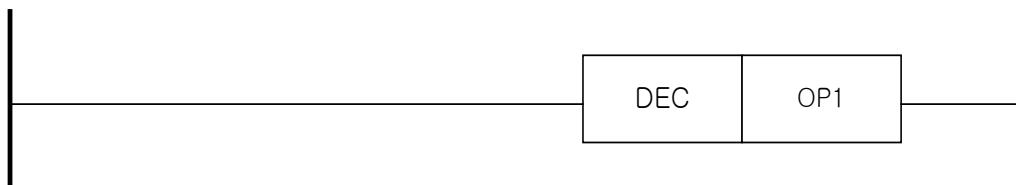
지정한 레지스터의 값을 1만큼 감소시킵니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	감소 값을 저장할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 1만큼 감소시킬 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON이면 [RW100]이 1만큼 감소합니다.



5.3.8 TIMEADD (시간 가산)

■ [기능]

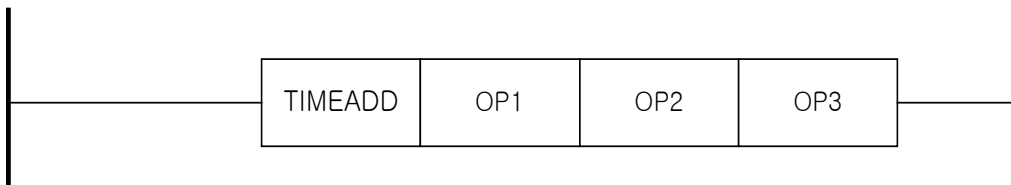
두 시간 데이터를 더하여 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	가산될 시간 데이터
OP2	-	○	○	-	-	가산할 시간 데이터
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



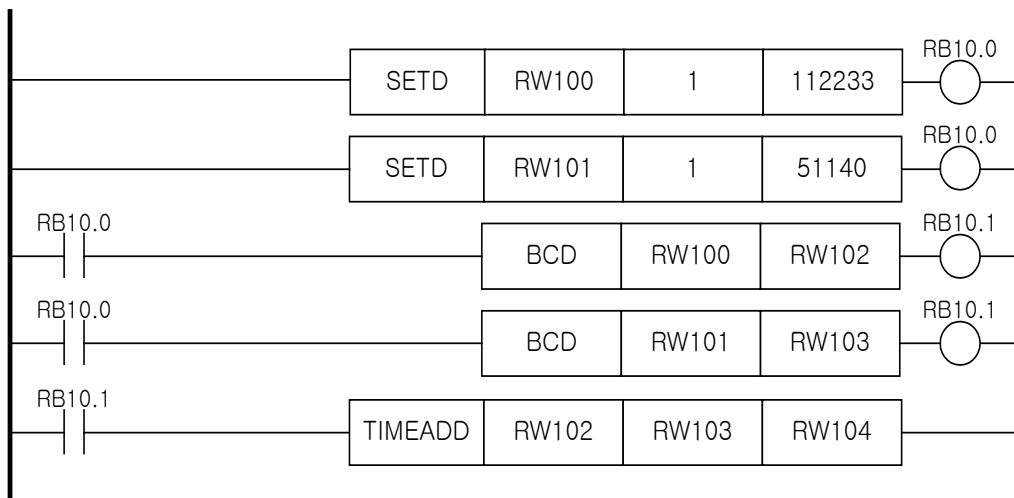
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	더해질 시간 데이터(BCD)를 지정
Operand 2	더할 시간 데이터(BCD)를 지정
Operand 3	계산된 시간 데이터(BCD)를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100], [RW101] 레지스터에 Binary형 시간 데이터를 입력하면, BCD형 시간데이터로 변환후, [TIMEADD] 명령을 수행합니다.

[RW104] 레지스터에 14551230이 저장됩니다.



5.3.9 TIMESUB (시간 감산)

■ [기능]

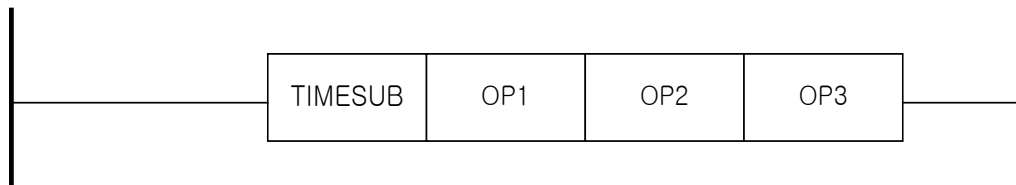
두 시간 데이터를 빼서 지정한 레지스터에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	감산될 시간 데이터
OP2	-	○	○	-	-	감산할 시간 데이터
OP3	-	○	-	-	-	연산 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



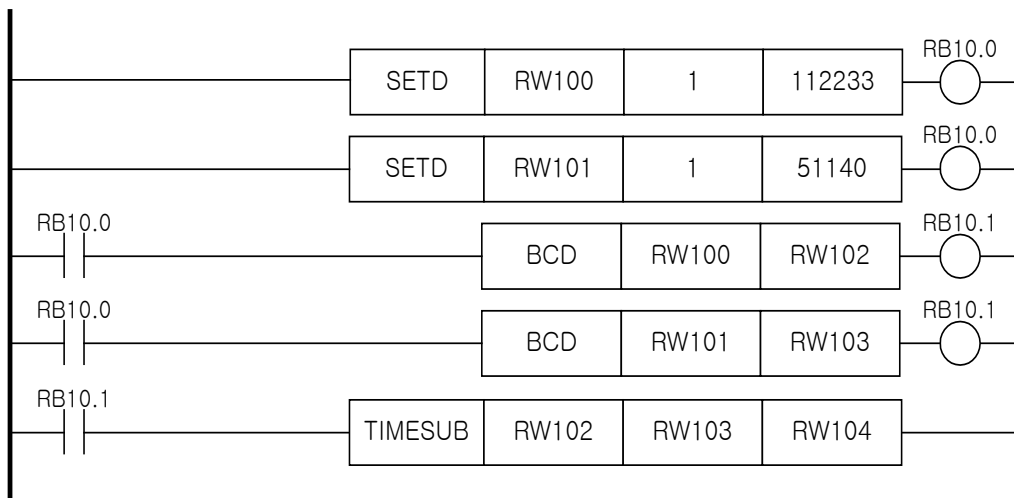
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	감산 되어질 시간 데이터(BCD)를 지정
Operand 2	감산 할 시간 데이터(BCD)를 지정
Operand 3	계산된 시간 데이터(BCD)를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100], [RW101] 레지스터에 Binary형 시간 데이터를 입력하면, BCD형 시간데이터로 변환후, [TIMESUB] 명령을 수행합니다.

[RW104] 레지스터에 3973950이 저장됩니다.



5.4 수치 비교 명령

5.4.1 LT (Less Than)

■ [기능]

두 값의 크기를 비교합니다.

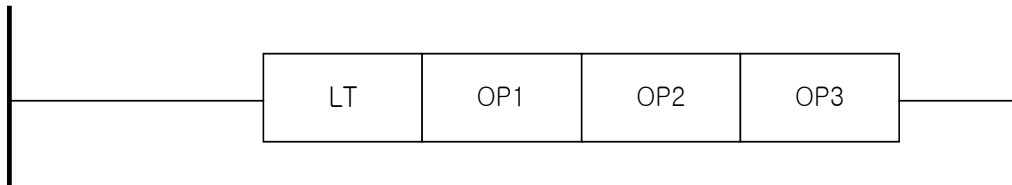
OP1의 값이 OP2의 값보다 작으면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



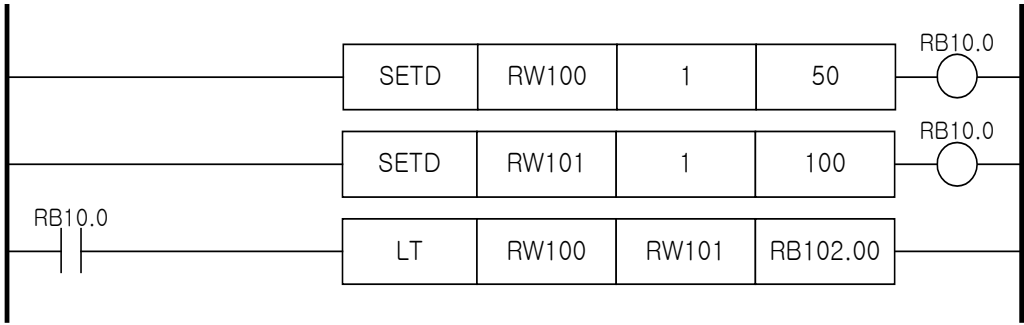
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1 >= OP2 일 경우 결과 = 1 : OP1 < OP2 일 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터 크기를 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] < [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.2 LE (Less Than / Equal)

■ [기능]

두 값의 크기를 비교합니다.

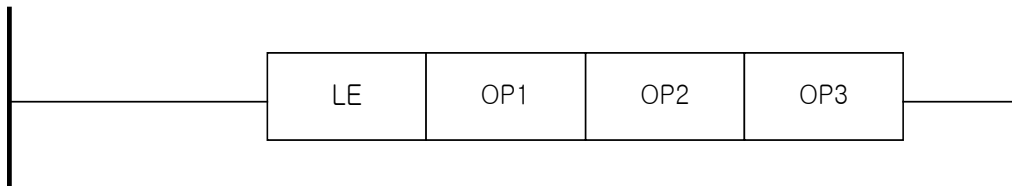
OP1의 값이 OP2의 값보다 작거나 같으면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



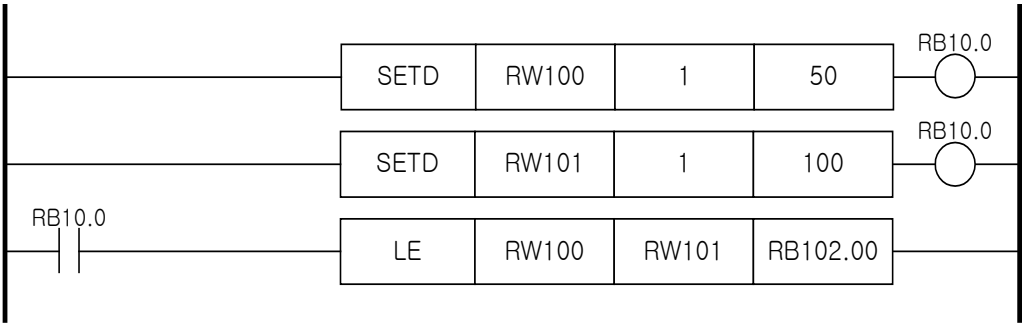
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1 > OP2 일 경우 결과 = 1 : OP1 <= OP2 일 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터 크기를 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] <= [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.3 GT (Greater Than)

■ [기능]

두 값의 크기를 비교합니다.

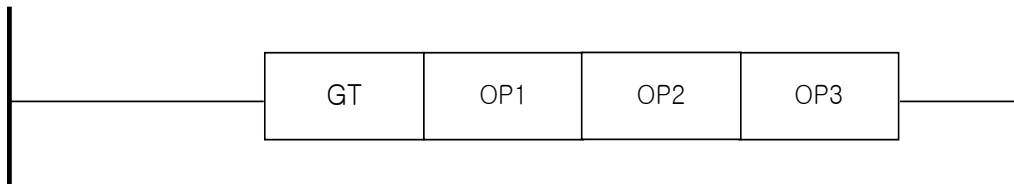
OP1의 값이 OP2의 값보다 크면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



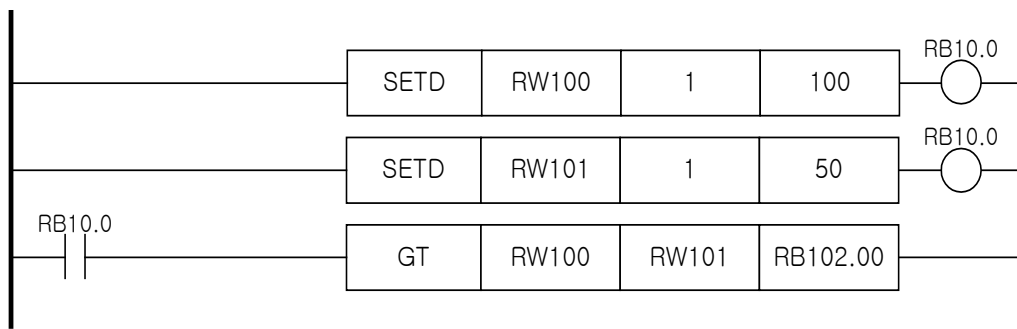
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 : 0 : OP1 ≤ OP2 일 경우 결과 : 1 : OP1 > OP2 일 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터 크기를 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] > [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.4 GE (Greater Than / Equal)

■ [기능]

두 값의 크기를 비교합니다.

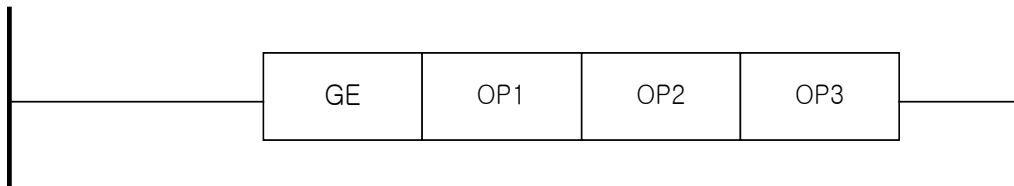
OP1의 값이 OP2의 값보다 크거나 같으면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



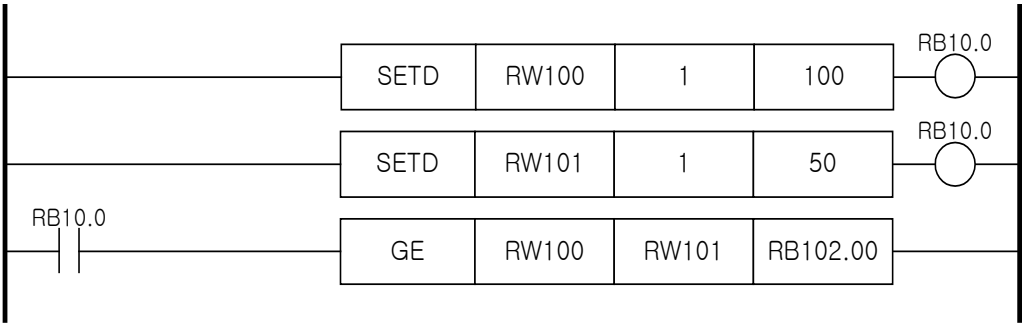
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1 < OP2 일 경우 결과 = 1 : OP1 >= OP2 일 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터 크기를 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] >= [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.5 EQ (Equal)

■ [기능]

두 값이 같은 지를 비교합니다.

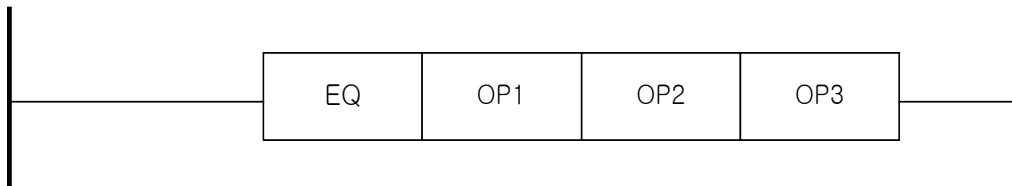
OP1의 값과 OP2의 값이 같으면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



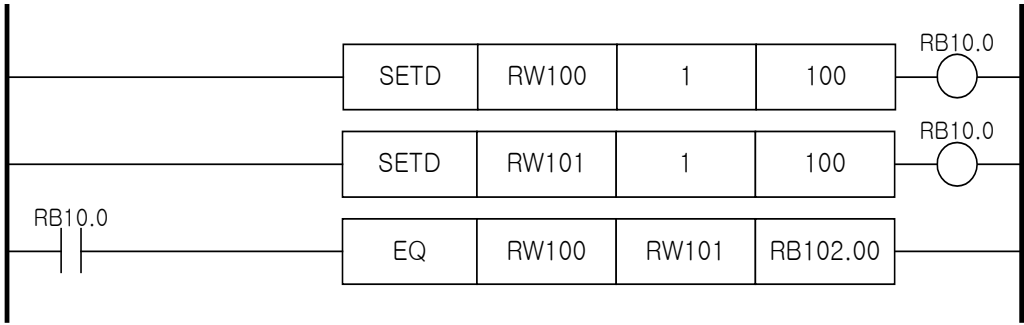
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1가 OP2의 값이 다를 경우 결과 = 1 : OP1과 OP2의 값이 같을 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터가 같은지 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] = [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.6 EQ2 (Equal 2)

■ [기능]

두 값이 같은 지를 비교합니다. OP1의 값과 OP2의 값이 같으면 1을 출력합니다.

EQ 명령과 동일한 기능을 수행합니다.

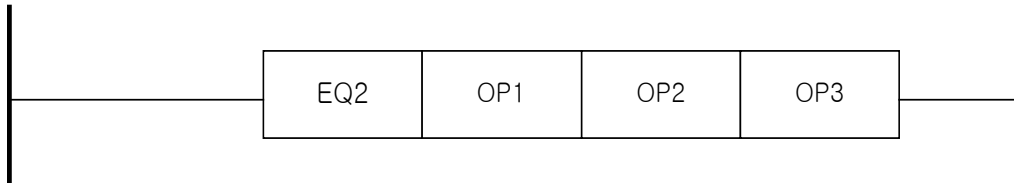
다른 점은 실행 조건이 0이면 항상 0을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

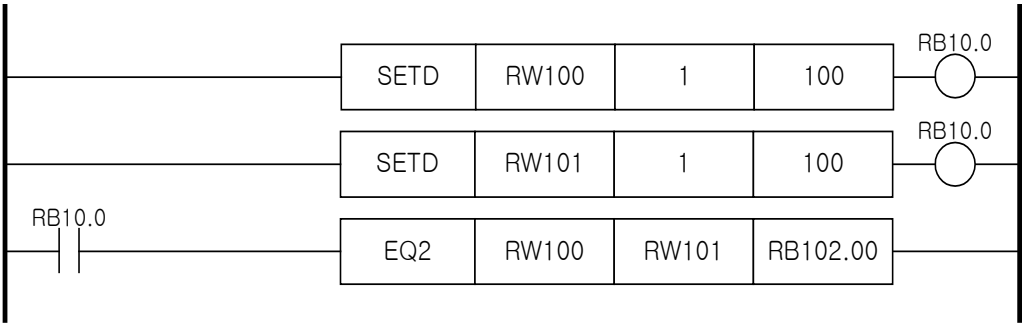
실행 조건	CC = 0 : OP3=0 출력 CC = 1 : EQ2 명령을 실행
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1가 OP2의 값이 다를 경우 or 실행조건 0 결과 = 1 : OP1과 OP2의 값이 같을 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터가 같은지 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] = [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.

[RB10.1]이 0이면 [RB102.00] 레지스터에 0이 출력됩니다.



5.4.7 NE (Not Equal)

■ [기능]

두 값이 다른 지를 비교합니다.

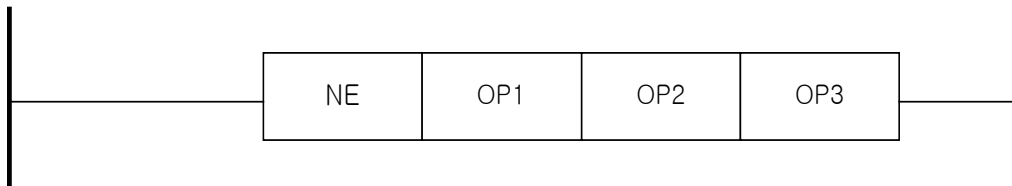
OP1의 값과 OP2의 값이 다르면 1을 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	크기 비교할 인자 1
OP2	-	○	○	○	○	크기 비교할 인자 2
OP3	○	-	-	-	-	연산 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



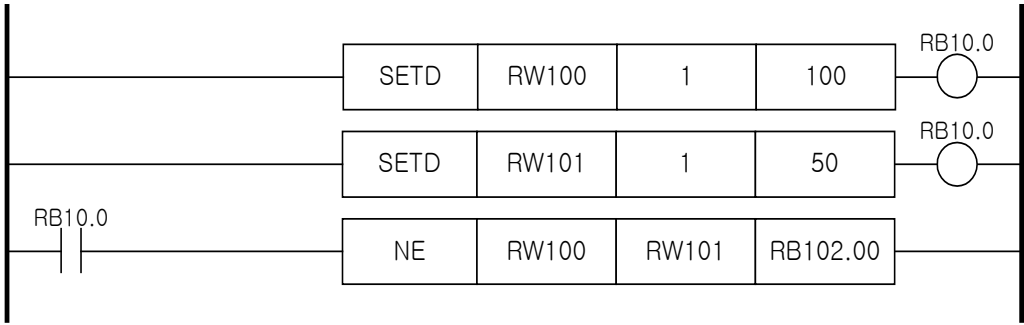
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	크기를 비교할 값을 지정
Operand 2	크기를 비교할 값을 지정
Operand 3	결과 = 0 : OP1가 OP2의 값이 같을 경우 결과 = 1 : OP1과 OP2의 값이 다를 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]와 [RW101]의 데이터가 다른지 비교하여 [RB102.00]에 결과를 출력합니다.

[RW100] != [RW101] 이므로 [RB102.00] 레지스터에 1이 출력됩니다.



5.4.8 RNG (범위 체크)

■ [기능]

지정한 레지스터의 값이 설정한 범위 내에 있는지 체크 합니다.

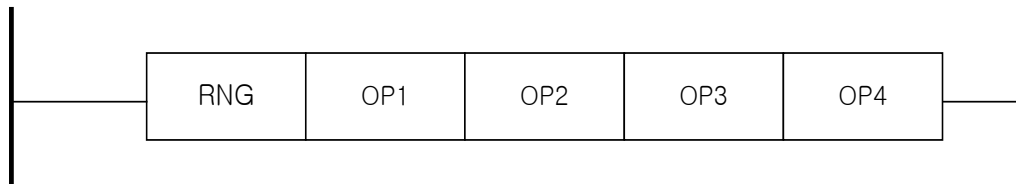
지정한 레지스터의 값이 설정한 범위 내에 있으면, 1을 출력 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	범위 체크할 레지스터
OP2	-	○	○	○	○	설정 범위 하한값
OP3	-	○	○	○	○	설정 범위 상한값
OP4	○	-	-	-	-	범위 체크 결과

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



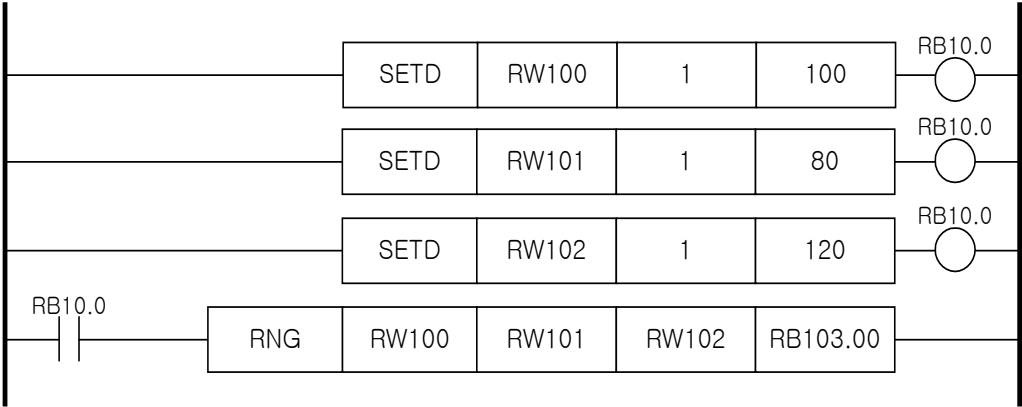
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	범위를 체크할 레지스터를 지정
Operand 2	설정 범위의 최소값을 입력
Operand 3	설정 범위의 최대값을 입력
Operand 4	결과 = 0 : 설정한 범위 내에 없을 경우 결과 = 1 : 설정한 범위 내에 있을 경우
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터가 [RW101], [RW102]의 데이터 범위 내의 데이터인지 비교하여 [RB103.00]에 결과를 출력합니다.

80 < [RW100]=100 < 120 이므로 [RB103.00] 레지스터에 1이 출력됩니다.



5.5 데이터 변환 명령

5.5.1 INV (부호 반전)

■ [기능]

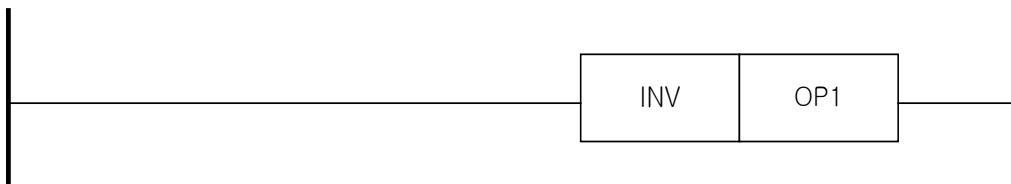
지정한 레지스터 값의 부호를 반대로 만드는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	부호 반전할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



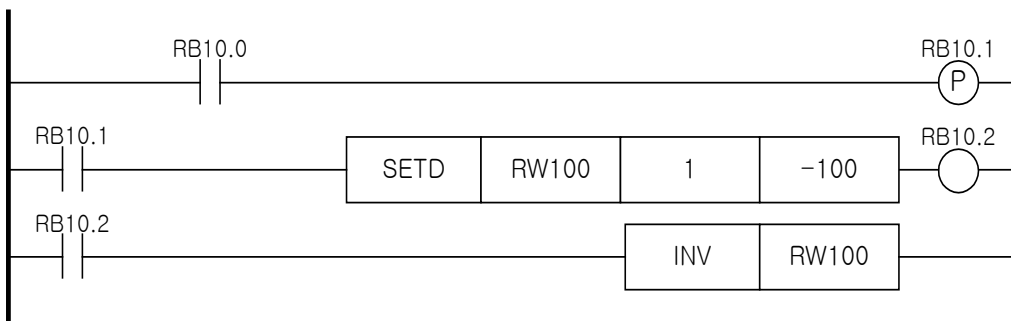
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	부호를 반대로 만들 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100] 데이터의 부호를 반전하여 저장합니다.

[RW100]에 100이 저장됩니다.



5.5.2 INVB (비트 반전)

■ [기능]

지정한 레지스터의 비트 값을 반전 시킵니다(0->1, 1->0)

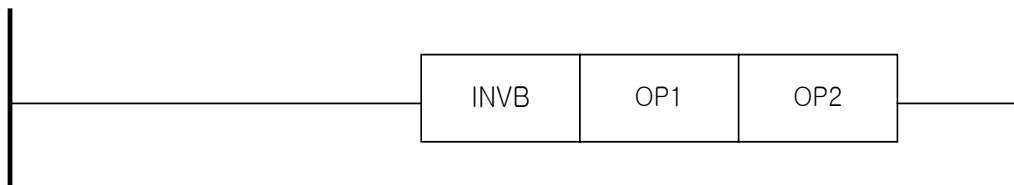
반전 시작 BIT위치와 반전 종료 BIT위치를 지정함으로써, 반전시킬 범위를 설정합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	비트 반전시킬 레지스터
OP2	-	○	○	-	-	반전시킬 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



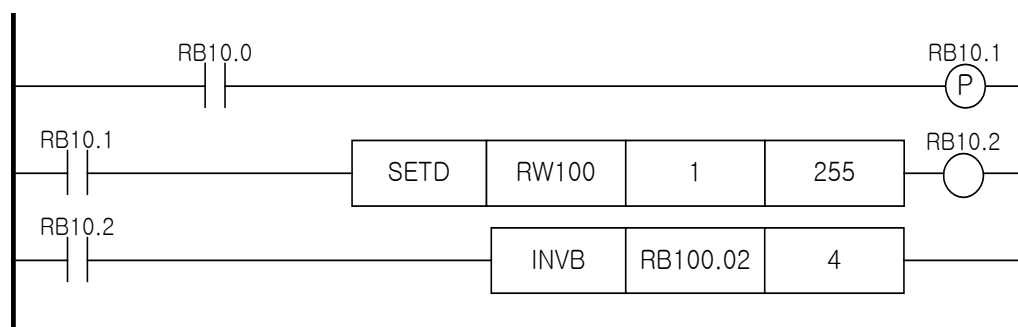
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	비트 값을 반전시킬 레지스터를 지정합니다 값을 기록하기 시작할 BIT의 위치도 함께 지정되어야 하기 때문에, BIT형으로 지정합니다
Operand 2	폭(Width)을 지정합니다. OP1에 지정한 시작 BIT부터 Width 만큼의 BIT가 값을 기록할 영역이 됩니다.
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.2]부터 4개 Bit의 비트값을 반전합니다.

[INVB] 명령 수행결과 [RW100]에는 195가 저장됩니다.



5.5.3 COM (2의 보수 변환)

■ [기능]

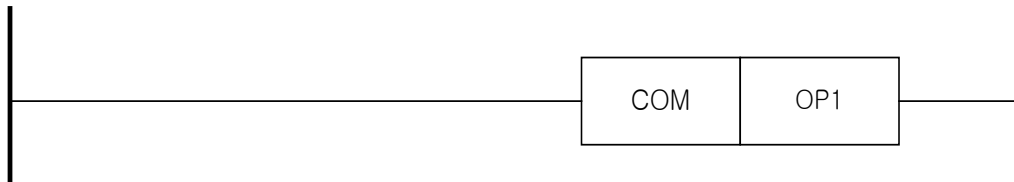
지정한 레지스터 값을 2의 보수로 변환합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	2의 보수로 변환할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



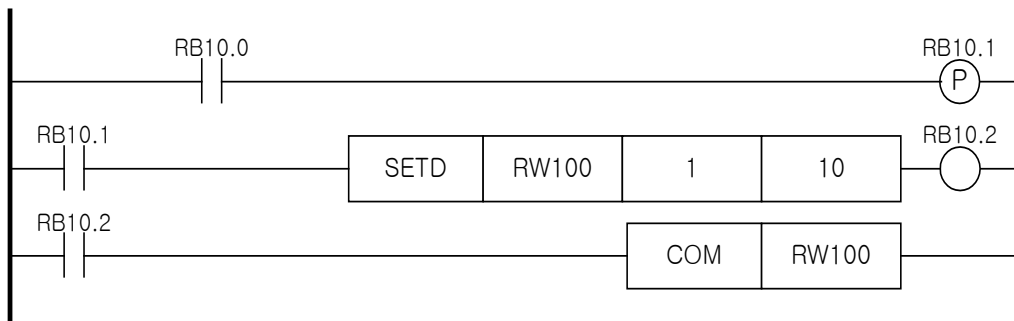
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	2의 보수로 변환할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100] 데이터를 2의 보수로 변환합니다.

[COM] 명령 수행결과 [RW100]에는 -100이 저장됩니다.



5.5.4 ABSD (절대치 변환)

■ [기능]

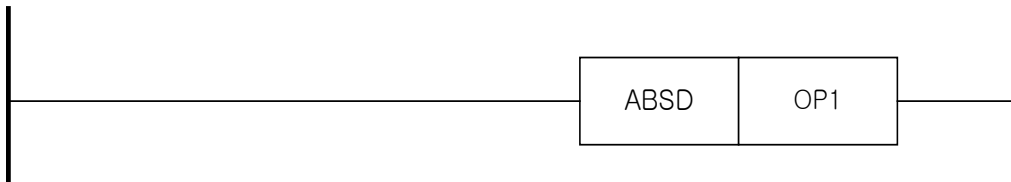
지정한 레지스터 값을 절대값으로 변환합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	절대값으로 변환할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



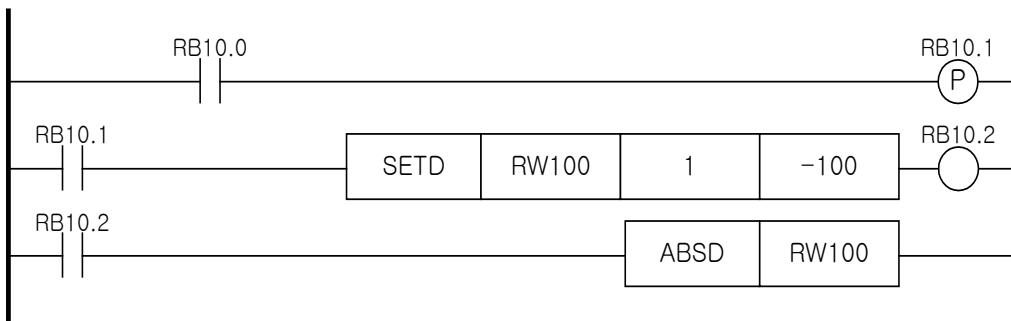
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	절대값으로 변환할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100] 데이터를 절대값으로 변환합니다.

[ABSD] 명령 수행결과 [RW100]에는 100이 저장됩니다.



5.5.5 BCD (BIN ⇒BCD 변환)

■ [기능]

Binary 데이터를 지정한 레지스터에 BCD 변환 합니다.

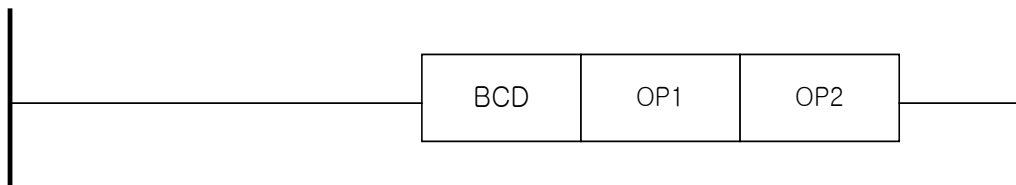
Ex) 1234(10진수) -> 1234H(16진수)

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	BCD 변환할 BIN 데이터의 레지스터
OP2	-	○	-	-	-	BCD 변환된 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



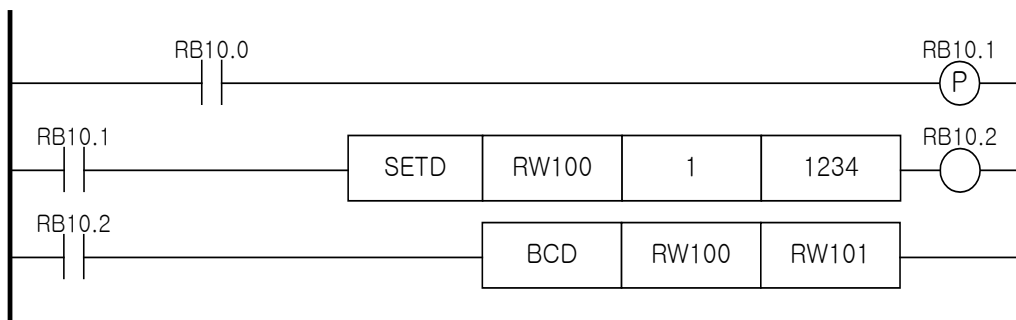
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	BCD 변환할 BIN 데이터의 레지스터를 지정
Operand 2	변환된 BCD 데이터의 결과 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 BIN 데이터를 [RW101]에 BCD 데이터로 변환하여 저장합니다.

[BCD] 명령 수행결과 [RW101]에는 4660이 저장됩니다.



5.5.6 BIN (BCD ⇒ BIN 변환)

■ [기능]

BCD 데이터를 지정한 레지스터에 BIN 변환 합니다

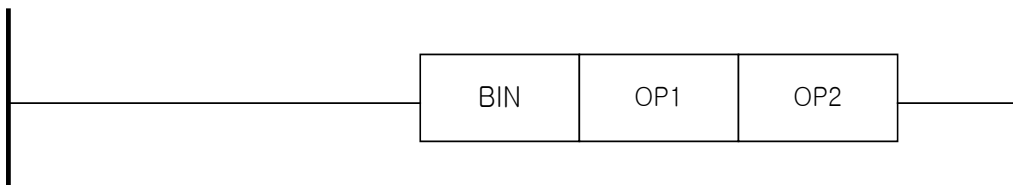
Ex) 1234H(16진수) -> 1234(10진수)

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	BIN 변환할 BCD 데이터의 레지스터
OP2	-	○	-	-	-	BIN 변환된 결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



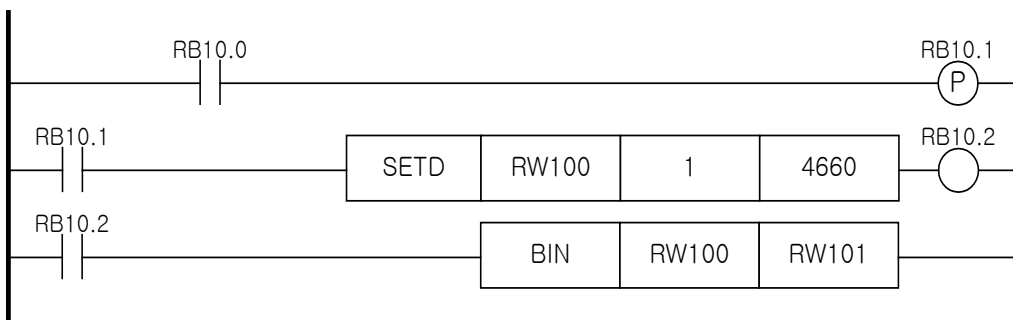
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	BIN 변환할 BCD 데이터의 레지스터를 지정
Operand 2	변환된 BIN 데이터의 결과 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 BCD 데이터를 [RW101]에 BIN 데이터로 변환하여 저장합니다.

[BIN] 명령 수행결과 [RW101]에는 1234가 저장됩니다



5.5.7 WTOF (WORD⇒FLOAT 변환)

■ [기능]

WORD형 레지스터를 FLOAT형 레지스터로 변환하여 저장합니다.

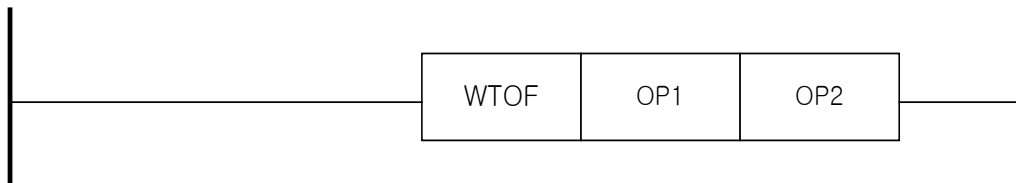
WORD형으로 사용한 레지스터를 변환 없이 FLOAT형으로 사용한다면, 그 값이 완전히 달라집니다. 따라서, 한번 WORD형으로 사용한 레지스터는 계속 WORD형으로만 사용하는 것이 실수를 막을 수 있는 방법입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	변환할 WORD형 레지스터
OP2	-	-	-	○	-	FLOAT형으로 저장할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



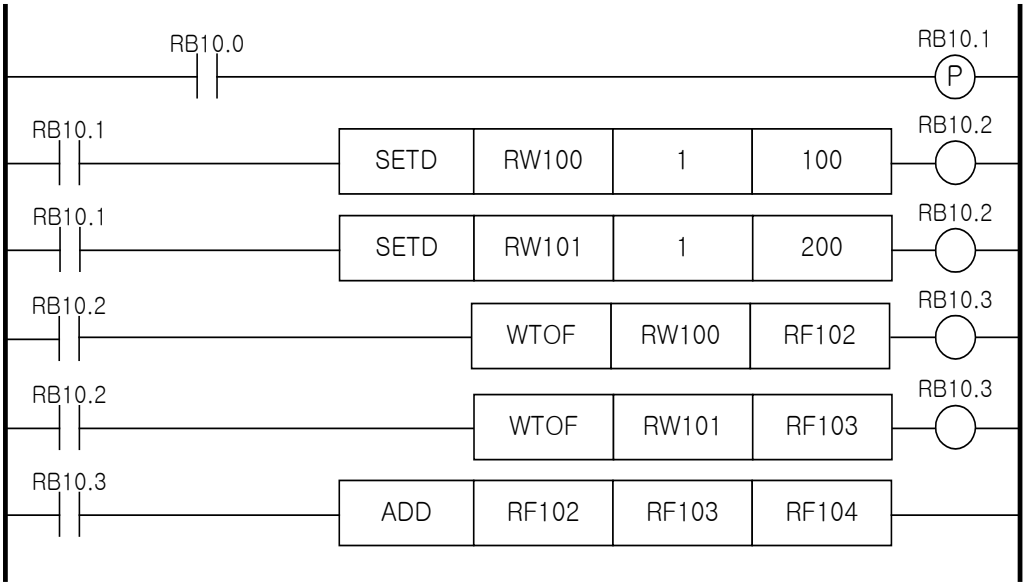
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	WORD형 데이터가 저장되어 있는 레지스터를 지정
Operand 2	FLOAT형으로 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100],[RW101]의 WORD형 데이터를 FLOAT형 데이터로 변환하여 FLOAT 데이터 [ADD] 연산을 수행합니다.

[ADD] 명령 수행결과 [RF104]에는 300.00이 저장됩니다.



5.5.8 FTOW (FLOAT ⇒WORD 변환)

■ [기능]

FLOAT형 레지스터를 WORD형 레지스터로 변환하여 저장합니다.

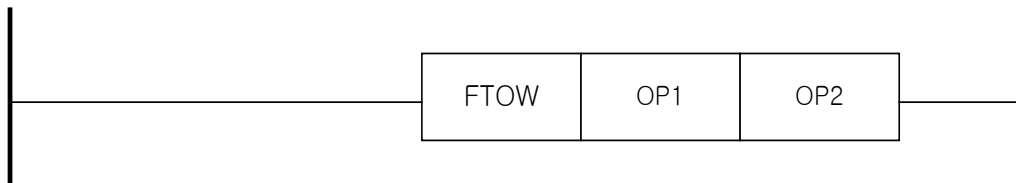
FLOAT형으로 사용한 레지스터를 변환 없이 WORD형으로 사용한다면, 그 값이 완전히 달라집니다. 따라서, 한번 FLOAT형으로 사용한 레지스터는 계속 FLOAT형으로만 사용하는 것이 실수를 막을 수 있는 방법입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	-	○	-	변환할 FLOAT형 레지스터
OP2	-	○	-	-	-	WORD형으로 저장할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



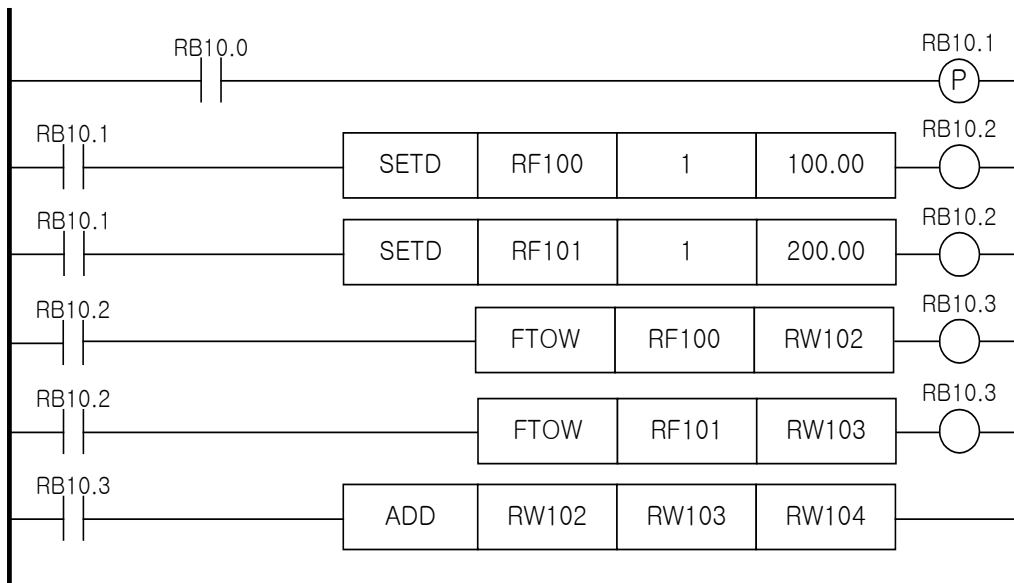
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	FLOAT형 데이터가 저장되어 있는 레지스터를 지정
Operand 2	WORD형으로 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100],[RF101]의 FLOAT형 데이터를 WORD형 데이터로 변환하여 WORD 데이터 [ADD] 연산을 수행합니다.

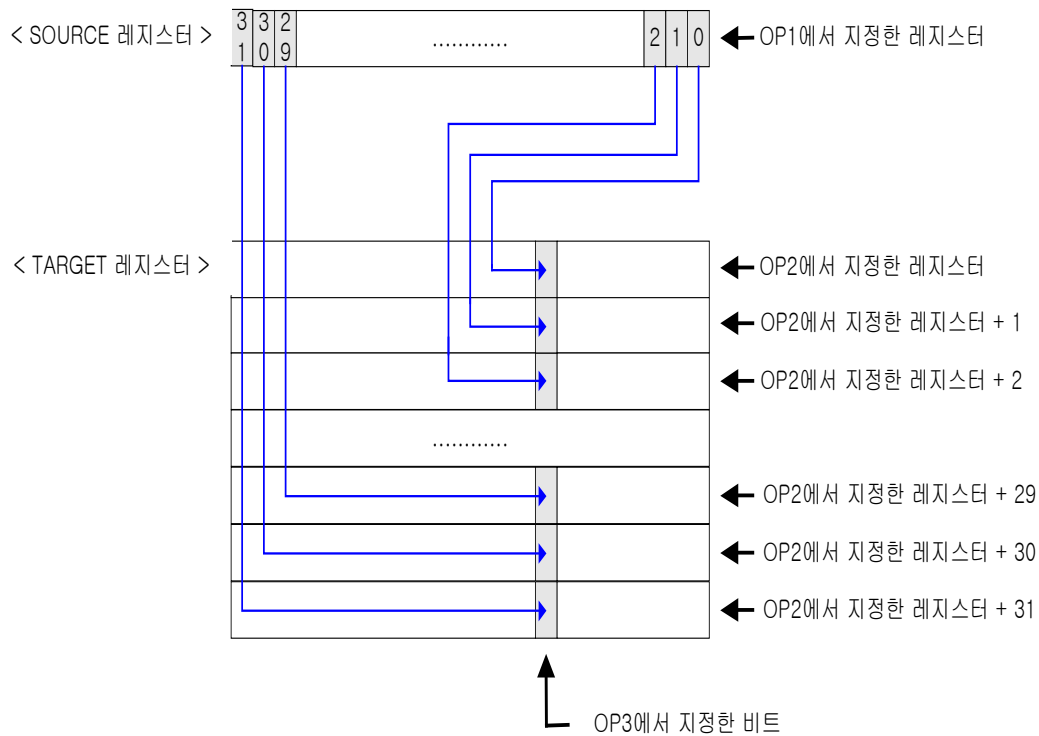
[ADD] 명령 수행결과 [RW104]에는 600이 저장됩니다.



5.5.9 BITRTOC (비트행 → 비트열 변환)

■ [기능]

지정된 레지스터(OP1)의 각 비트 데이터를 별도로 지정한 32개의 레지스터(OP2)의 지정한 비트(OP3)에 세트(Set) 합니다.

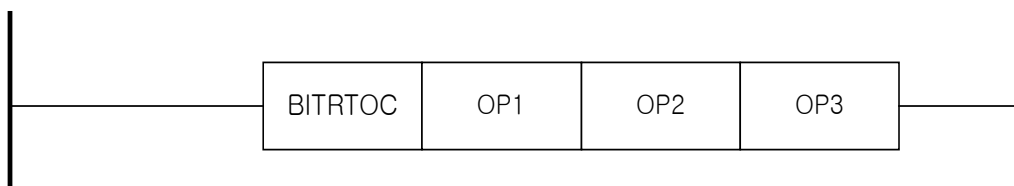


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	비트열로 변환할 레지스터
OP2	-	○	-	-	-	세트할 첫번째 레지스터 지정
OP3	-	○	○	-	-	세트할 비트를 지정

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

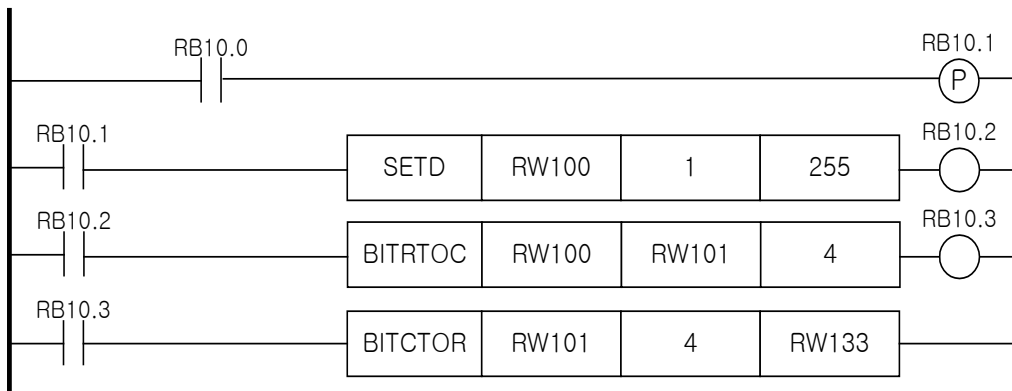


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	비트열로 변환할 레지스터를 지정 (Source)
Operand 2	세트할 첫번째 레지스터 지정 (Target) 이후 연속된 31개의 레지스터는 자동으로 지정됨
Operand 3	세트할 비트를 지정 (Target)
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

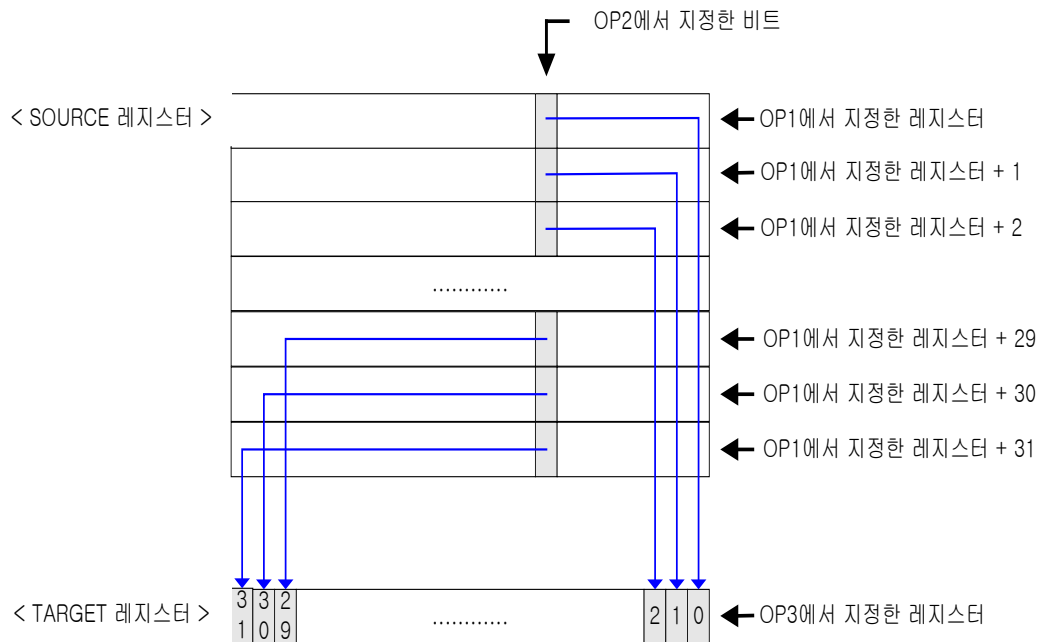
[RW100]의 32Bit 값이 [RW101]부터 32개 레지스터의 4번째 비트에 저장됩니다.
[BITRTOC] 명령 수행결과 [RW101]부터 [RW132]까지 16이 저장됩니다.



5.5.10 BITCTOR (비트열 ⇒ 비트행 변환)

■ [기능]

지정된 32개 레지스터(OP1)에서 지정비트(OP2)의 데이터만을 별도로 지정한 레지스터(OP3)에 세트(Set) 합니다.

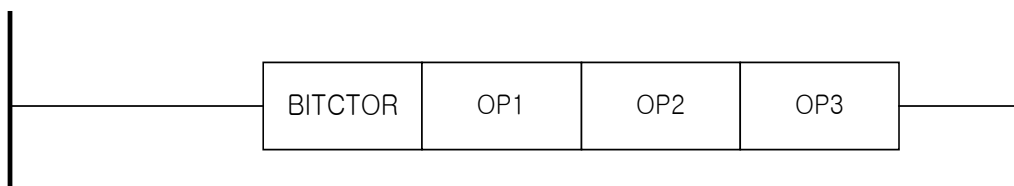


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	값을 읽을 첫번째 레지스터
OP2	-	○	○	-	-	비트 지정
OP3	-	○	-	-	-	결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



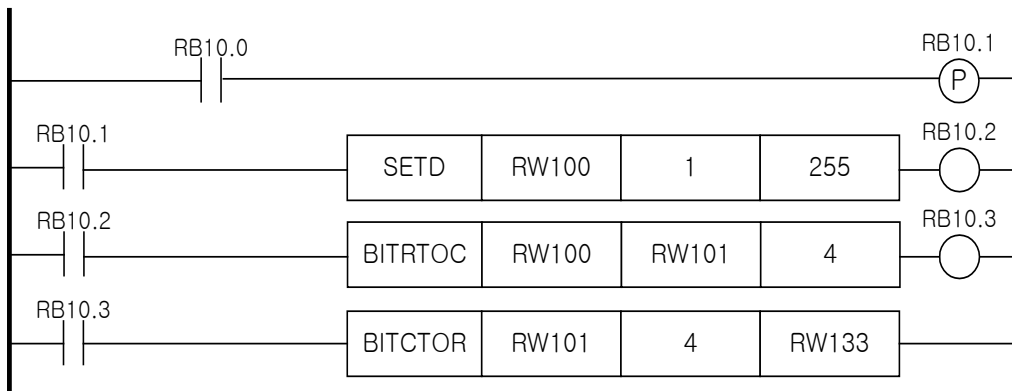
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 읽을 첫번째 레지스터 지정 (Source) 이후 연속된 31개의 레지스터는 자동으로 지정됨
Operand 2	비트 지정 (Source)
Operand 3	결과를 저장할 레지스터 (Target)
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW101]부터 32개 레지스터의 4번째 비트 값이 [RW133]에 저장됩니다.

[BITCTOR] 명령 수행결과 [RW133]에 255가 저장됩니다.



5.5.11 PARITY (패리티 변환)

■ [기능]

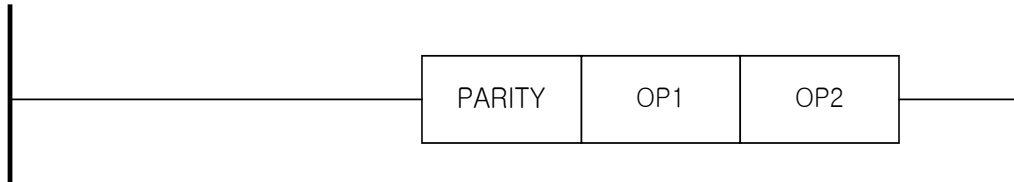
2진수 표현 Bit의 On Bit 수를 계산합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	On 비트 수를 계산할 레지스터
OP2	-	○	-	-	-	결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



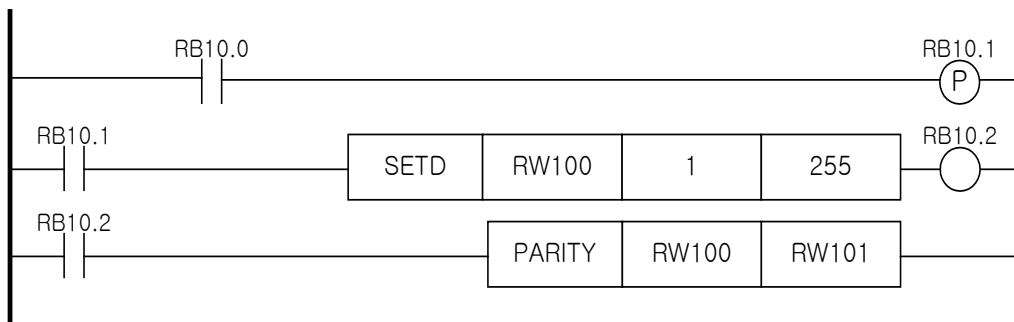
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	On Bit 수를 계산할 레지스터
Operand 2	계산 결과를 저장할 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 ON Bit 수를 계산하여 [RW101]에 저장합니다.

[PARITY] 명령 수행결과 [RW101]에는 8이 저장됩니다.



5.5.12 BINTOASC (BIN⇒ASC 변환)

■ [기능]

32Bit의 Binary Data를 문자열로 변환 (ASCII 코드에 의한 변환)

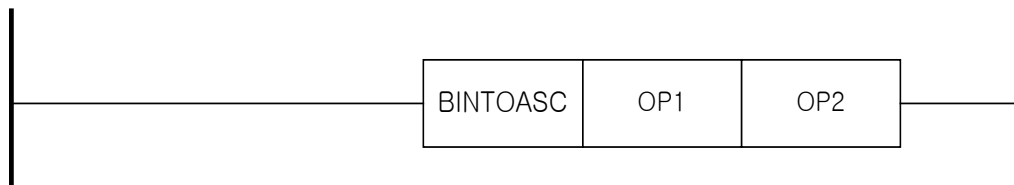
ASCII 데이터 변환시 각 레지스터마다 4개의 ASCII 코드가 저장되어, 지정한 시작 레지스터 위치부터 3개의 레지스터에 저장됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	ASCII 코드로 변환할 데이터
OP2	-	○	-	-	-	결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	ASCII 코드로 변환할 Binary Data
Operand 2	변환 결과를 저장할 시작 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

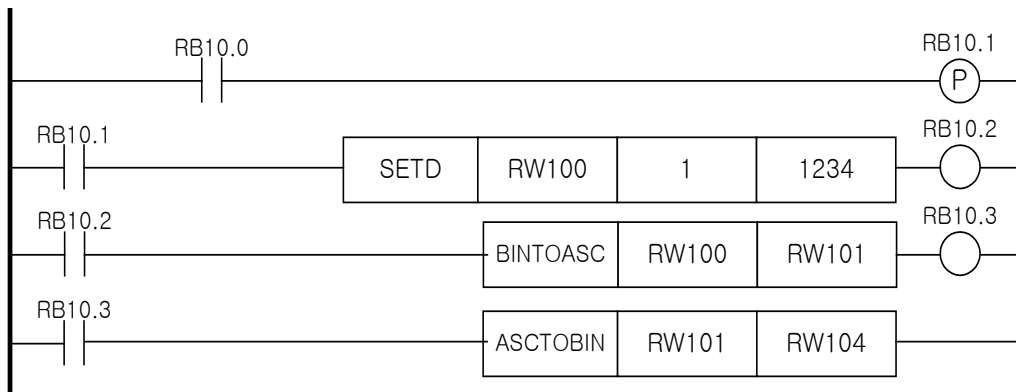
[RW100]의 데이터를 ASCII로 변환합니다.

[BINTOASC] 명령 수행결과 [RW101]부터 3개 레지스터에 ASCII가 저장됩니다.

[RW101]=842216491

[RW102]=49

[RW103]=0



5.5.13 ASCTOBIN (ASC ⇒ BIN 변환)

■ [기능]

ASCII 코드 데이터를 32Bit의 Binary Data로 변환 (ASCII 코드에 의한 변환)

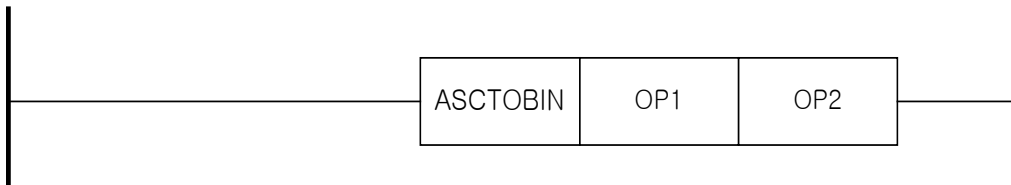
지정한 시작 레지스터부터 3개 레지스터의 ASCII 코드 데이터 값이 하나의 Binary Data로 변환됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	BIN 데이터로 변환할 레지스터
OP2	-	○	-	-	-	결과 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



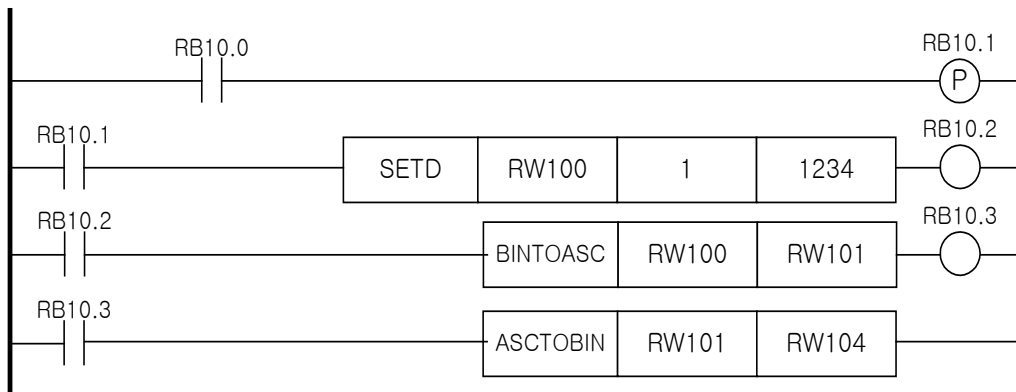
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	BIN 데이터로 변환할 ASCII 코드 데이터의 시작 레지스터
Operand 2	변환 값을 저장할 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW101]부터 3개 레지스터의 ASCII 데이터를 Binary 데이터로 변환합니다.

[ASCTOBIN] 명령 수행결과 [RW104]에 1234가 저장됩니다.



5.6 데이터 전송 명령

5.6.1 MOVB (비트 복사)

■ [기능]

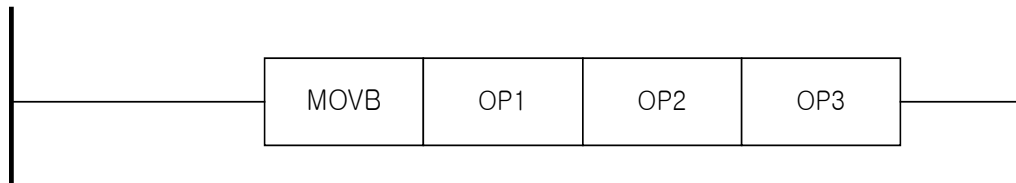
어떤 비트 값 한 개를 원하는 위치로 복사합니다. 또는 어떤 범위의 비트 값들을 원하는 위치로 복사합니다. 원본(Source)이 되는 비트와 목표(Target)가 되는 비트를 지정하고 폭(Width)을 지정하면, Source에서 Target으로 지정한 범위에 해당하는 비트 값들이 복사됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	복사할 원본 레지스터
OP2	○	-	-	-	-	복사될 레지스터
OP3	-	○	○	-	-	복사할 데이터 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



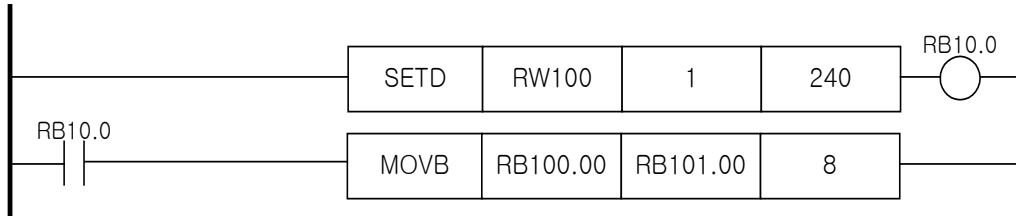
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	복사할 원본(Source)이 되는 레지스터를 지정합니다. 복사 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 2	복사할 목표(Target)가 되는 레지스터를 지정합니다. 복사 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 3	복사할 폭(Width)을 지정합니다. OP1, OP2에서 지정한 시작 BIT 위치와 OP3에서 지정한 Width를 사용하면 복사할 범위가 결정됩니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.00]부터 8개 Bit의 값을 [RB101.00] 레지스터에 복사합니다

[MOVB] 명령 수행결과 [RW101]에는 240이 저장됩니다.



5.6.2 MOVD (레지스터 복사)

■ [기능]

어떤 레지스터 값 한 개를 원하는 위치로 복사합니다.

또는 어떤 범위의 레지스터 값들을 원하는 위치로 복사합니다.

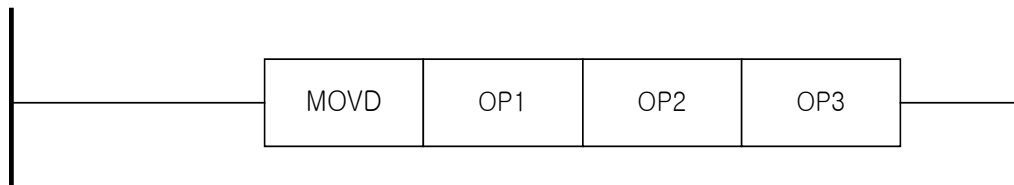
원본(Source)이 되는 레지스터와 목표(Target)가 되는 레지스터를 지정하고 폭(Width)을 지정하면, Source에서 Target으로 지정한 범위에 해당하는 레지스터 값들이 복사됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	복사할 원본 레지스터
OP2	-	○	-	○	-	복사될 레지스터
OP3	-	○	○	-	-	복사할 데이터 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



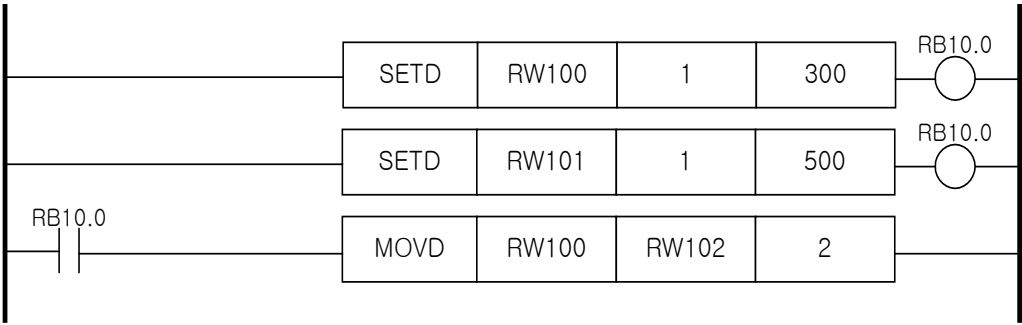
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	복사할 원본(Source)이 되는 레지스터를 지정
Operand 2	복사할 목표(Target)가 되는 레지스터를 지정
Operand 3	복사할 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 2개의 레지스터 값을 [RW102]부터 2개의 레지스터에 복사합니다.

[MOVD] 명령 수행결과 [RW102]에 300, [RW103]에 500이 저장됩니다.



5.6.3 CHGB (비트 교환, Change Bit)

■ [기능]

어떤 두 레지스터 사이에서 특정 범위의 비트 값들을 교환합니다.

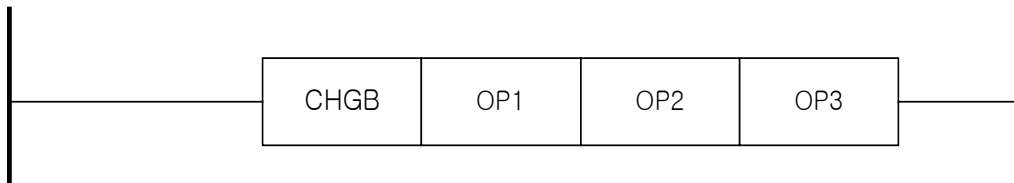
두 개의 레지스터를 지정하고 교환할 폭(Width)을 지정하면, 지정된 범위의 비트 값들이 서로 교환됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	교환할 레지스터 1
OP2	○	-	-	-	-	교환할 레지스터 2
OP3	-	○	○	-	-	교환할 데이터 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



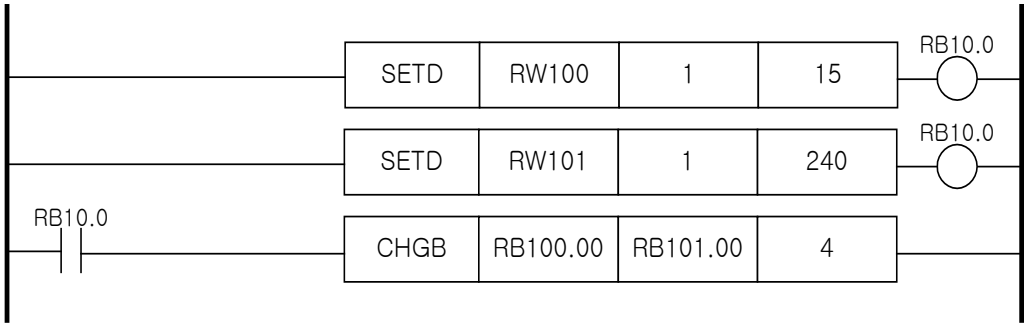
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	비트 값을 교환할 첫 번째 레지스터를 지정
Operand 2	비트 값을 교환할 두 번째 레지스터를 지정
Operand 3	교환이 이루어질 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.00]부터 4Bit 값과 [RB101.00]부터 4Bit 값을 교환합니다.

[CHGB] 명령 수행결과 [RW100]에는 0, [RW101]에는 255가 저장됩니다.



5.6.4 CHGD (레지스터 교환, Change Register)

■ [기능]

어떤 두 레지스터의 값을 서로 교환합니다.

또는 여러 개의 레지스터 사이에서 값을 서로 교환 합니다.

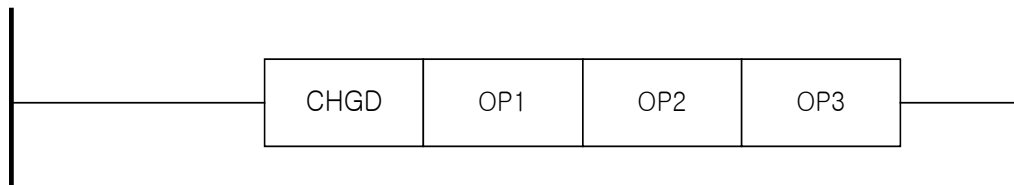
교환이 시작될 두 레지스터를 지정하고 교환할 폭(Width)을 지정하면, 지정된 범위의 레지스터 값들이 서로 교환됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	교환할 레지스터 1
OP2	-	○	-	○	-	교환할 레지스터 2
OP3	-	○	○	-	-	교환할 데이터 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



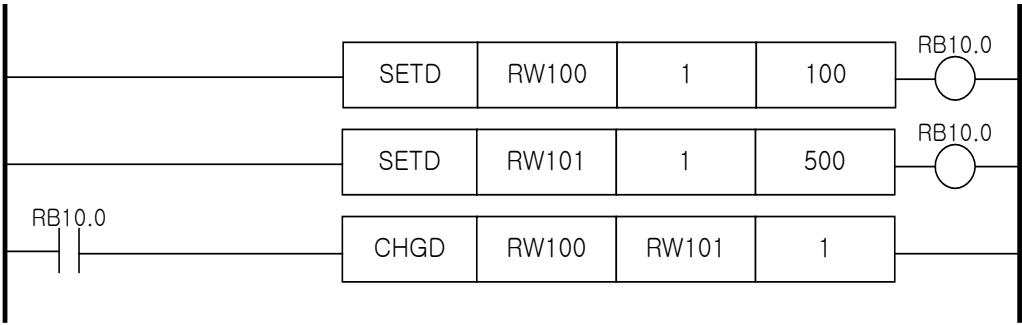
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	레지스터 값을 교환할 첫 번째 시작 위치 지정
Operand 2	레지스터 값을 교환할 두 번째 시작 위치 지정
Operand 3	교환이 이루어질 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100] 값과 [RW101] 값을 교환합니다.

[CHGD] 명령 수행결과 [RW100]에는 500, [RW101]에는 100이 저장됩니다.



5.6.5 SWAP (Byte Swap)

■ [기능]

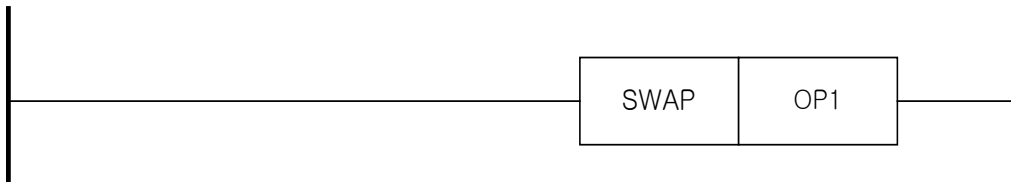
지정된 WORD형 레지스터의 상위 바이트와 하위 바이트를 교환합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	교환할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



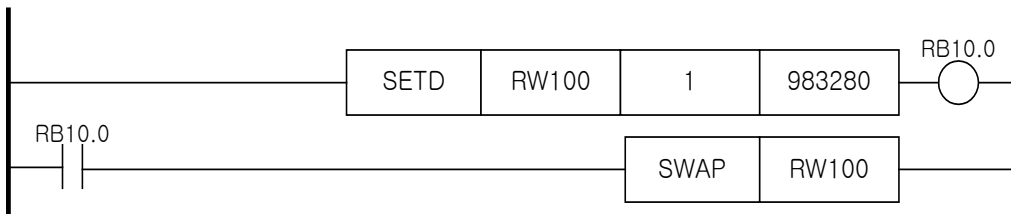
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	상위 바이트와 하위 바이트를 교환할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100] 데이터의 상위 바이트와 하위 바이트를 교환합니다.

[SWAP] 명령 수행결과 [RW100]에는 15728655가 저장됩니다.



5.6.6 SETD (레지스터 초기화)

■ [기능]

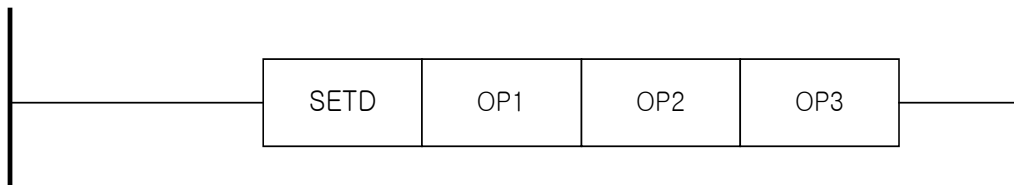
지정된 범위의 레지스터를 지정한 값으로 초기화 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	초기화 할 레지스터
OP2	-	○	○	-	-	초기화 할 데이터 폭(Width)
OP3	-	○	○	○	○	초기값

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



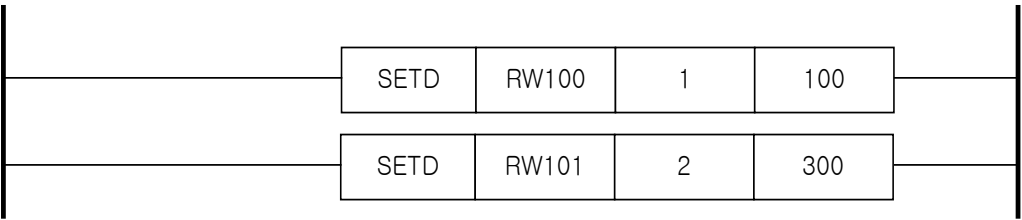
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	초기화 할 레지스터를 지정
Operand 2	폭(Width)을 지정합니다. OP1에 지정한 레지스터를 포함하여 그 이후 Width 만큼의 레지스터들이 모두 초기화 됩니다
Operand 3	초기값을 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100], [RW101]에 다음 데이터를 설정합니다.

[SETD] 명령 수행결과 [RW100]에는 100, [RW101]에는 300이 저장됩니다.



5.6.7 SETD2 (레지스터 초기화 2)

■ [기능]

지정된 범위의 레지스터를 지정한 값으로 초기화 합니다.

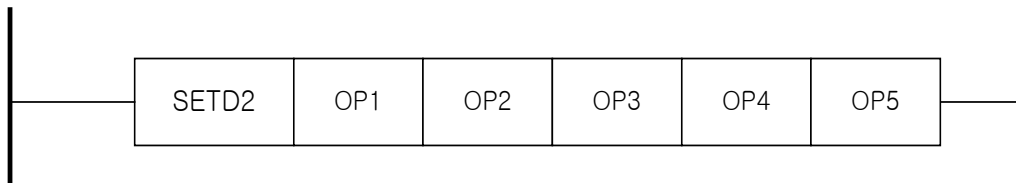
2개의 초기값을 설정하여 지정된 모드 상태에 따라 설정값으로 초기화 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	초기값 선택 비트
OP2	-	○	-	○	-	초기화 할 레지스터
OP3	-	○	○	-	-	초기화 할 데이터 폭(Width)
OP4	-	○	○	○	○	초기값 1
OP5	-	○	○	○	○	초기값 2

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



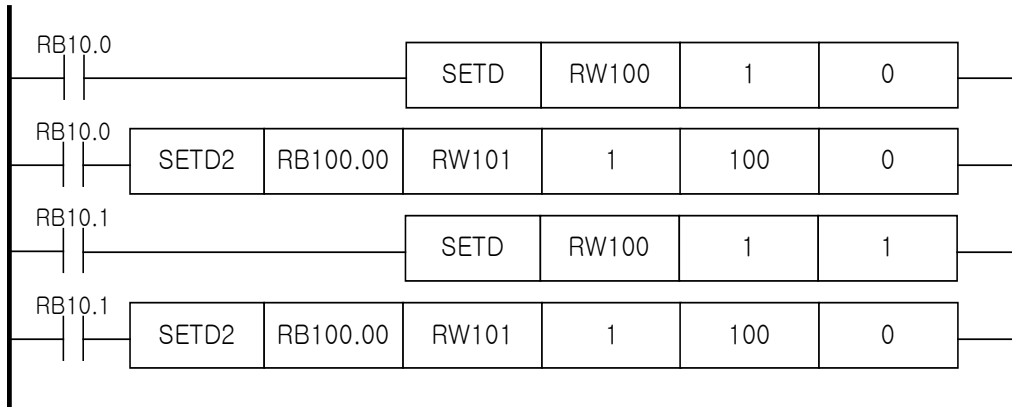
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	OP1 = 0 : OP4의 데이터로 초기화 OP1 = 1 : OP5의 데이터로 초기화
Operand 2	초기화 할 레지스터를 지정
Operand 3	폭(Width)을 지정합니다. OP2에 지정한 레지스터를 포함하여 그 이후 Width 만큼의 레지스터들이 모두 초기화 됩니다
Operand 4	첫번째 초기값을 지정합니다
Operand 5	두번째 초기값을 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.00] 값이 0이면 [RW101]에 OP4의 데이터를 설정하고, [RB100.00] 값이 1이면 [RW101]에 OP5의 데이터를 설정합니다.

[SETD2] 명령 수행결과 [RB10.0]이 ON이면 [RW101]에는 100이 저장되고, [RB10.1]이 ON이면 [RW101]에는 0이 저장됩니다.



5.6.8 SETB (Set Bit Data)

■ [기능]

레지스터의 특정 영역(비트)에 값을 기록합니다.

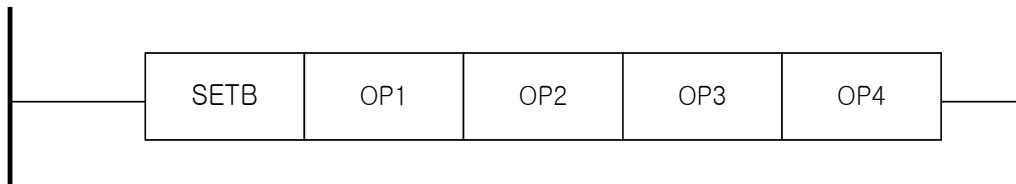
즉, 레지스터를 구성하는 전체 비트들 중에서 일부 비트들에 값을 기록합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	초기화 할 레지스터
OP2	-	○	○	-	-	초기화 할 데이터 폭(Width)
OP3	-	○	○	-	-	반복 회수
OP4	-	○	○	-	-	초기값

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



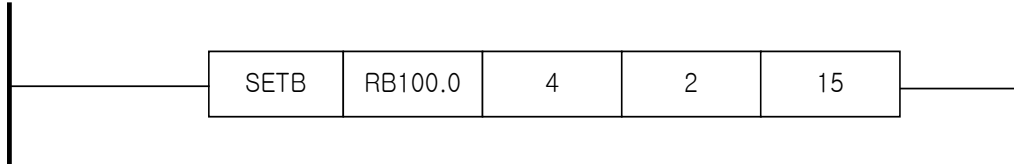
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 기록할 레지스터를 지정합니다. 값을 기록하기 시작할 BIT의 위치도 함께 지정되어야 하기 때문에, BIT형으로 지정합니다
Operand 2	폭(Width)을 지정합니다. OP1에 지정한 시작 BIT부터 Width 만큼의 BIT가 값을 기록할 영역이 됩니다. 이 영역에 OP4의 값이 기록됩니다
Operand 3	반복 회수를 지정합니다
Operand 4	초기값을 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.0]부터 4개의 Bit에 2회 반복하여 15를 설정합니다.

[SETB] 명령 수행결과 [RW100]에는 255가 저장됩니다.



1회 수행

RW100	0000 0000 0000 0000 0000 0000 0000 1111	15
-------	---	----



2회 수행

RW100	0000 0000 0000 0000 0000 0000 1111 1111	255
-------	---	-----

5.6.9 SETB2 (Set Bit Data)

■ [기능]

레지스터의 특정 영역(비트)에 값을 기록합니다.

즉, 레지스터를 구성하는 전체 비트들 중에서 일부 비트들에 값을 기록합니다.

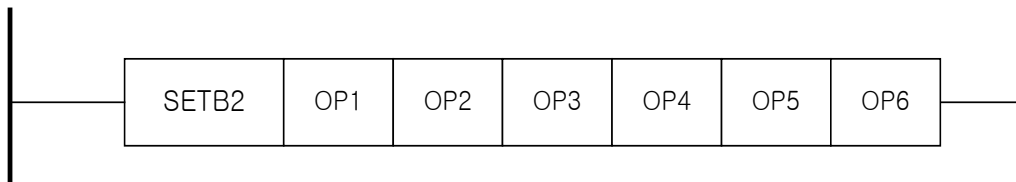
2개의 초기값을 설정하여 지정된 모드 상태에 따라 설정값으로 초기화 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	초기값 선택 비트
OP2	○	-	-	-	-	초기화 할 레지스터
OP3	-	○	○	-	-	초기화 할 데이터 폭(Width)
OP4	-	○	○	-	-	반복 회수
OP5	-	○	○	-	-	초기값 1
OP6	-	○	○	-	-	초기값 2

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



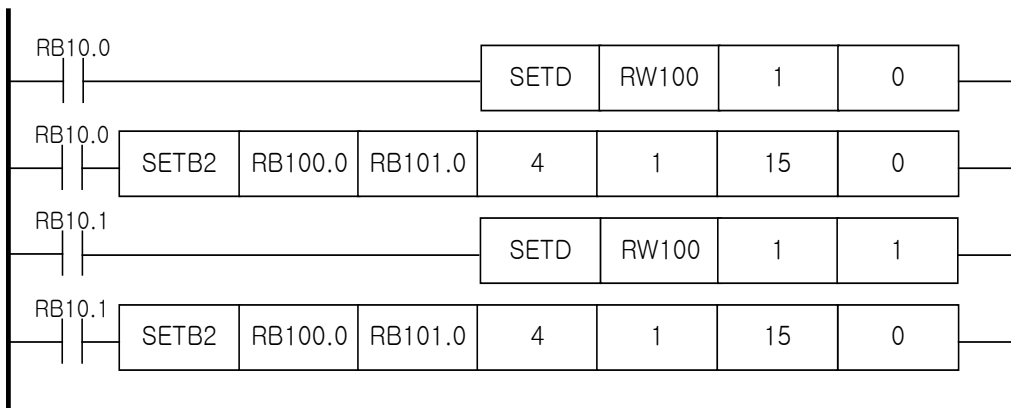
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	OP1 = 0 : OP5의 데이터로 초기화 OP1 = 1 : OP6의 데이터로 초기화
Operand 2	값을 기록할 레지스터를 지정합니다. 값을 기록하기 시작할 BIT의 위치도 함께 지정되어야 하기 때문에, BIT형으로 지정합니다
Operand 3	폭(Width)을 지정합니다. OP1에 지정한 시작 BIT부터 Width 만큼의 BIT가 값을 기록할 영역이 됩니다.
Operand 4	반복 회수를 지정합니다
Operand 5	첫번째 초기값을 지정합니다
Operand 6	두번째 초기값을 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.00] 값이 0이면 [RB101.0]부터 4개의 Bit에 OP5의 데이터를 설정하고, [RB100.00] 값이 1이면 OP6의 데이터를 설정합니다.

[SETB2] 명령 수행결과 [RB10.0]이 ON이면 [RW101]에는 15가 저장되고, [RB10.1]이 ON이면 [RW101]에는 0이 저장됩니다.



5.6.10 GETB (Get Bit Data)

■ [기능]

레지스터의 특정 영역(비트)에서 값을 읽어옵니다.

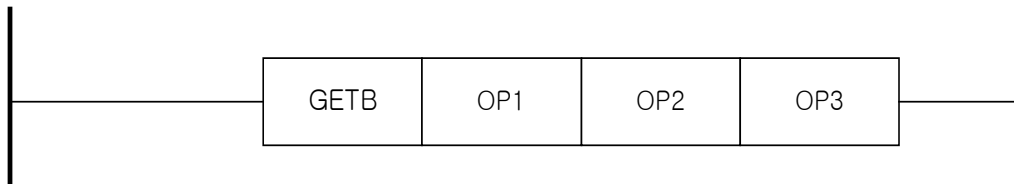
즉, 레지스터를 구성하는 전체 비트들 중에서 일부 비트들로 이루어진 값을 읽어옵니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	읽을 레지스터
OP2	-	○	○	-	-	읽을 데이터 폭(Width)
OP3	-	○	-	-	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



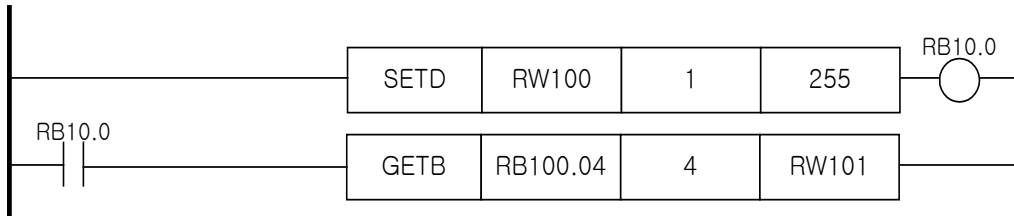
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 읽을 레지스터를 지정합니다. 값을 읽기 시작할 BIT의 위치도 함께 지정되어야 하기 때문에, BIT형으로 지정합니다
Operand 2	폭(Width)을 지정합니다. OP1에 지정한 시작 BIT부터 Width 만큼의 BIT가 값을 읽을 영역이 됩니다. 이 영역에 있는 BIT들로 이루어진 값을 읽어서 OP3에 기록합니다
Operand 3	읽은 값을 저장할 레지스터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB100.4]부터 4개 Bit의 값을 읽어 [RW101]에 저장합니다.

[GETB] 명령 수행결과 [RW101]에는 15가 저장됩니다.



5.6.11 SRCH (Search Data)

■ [기능]

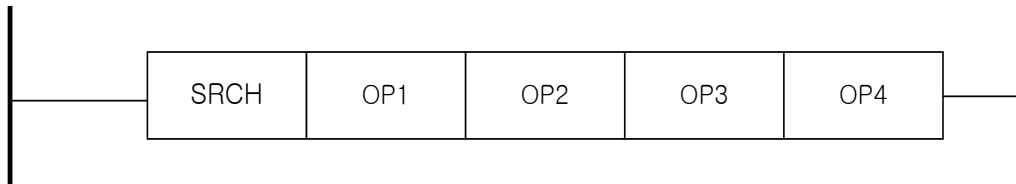
지정된 범위의 레지스터를 검색하여 Data와 일치하는 레지스터 위치를 찾아줍니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	검색 시작 레지스터
OP2	-	○	○	-	-	검색할 데이터 폭(Width)
OP3	-	○	○	○	○	검색할 데이터
OP4	-	○	-	-	-	검색한 데이터의 위치 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



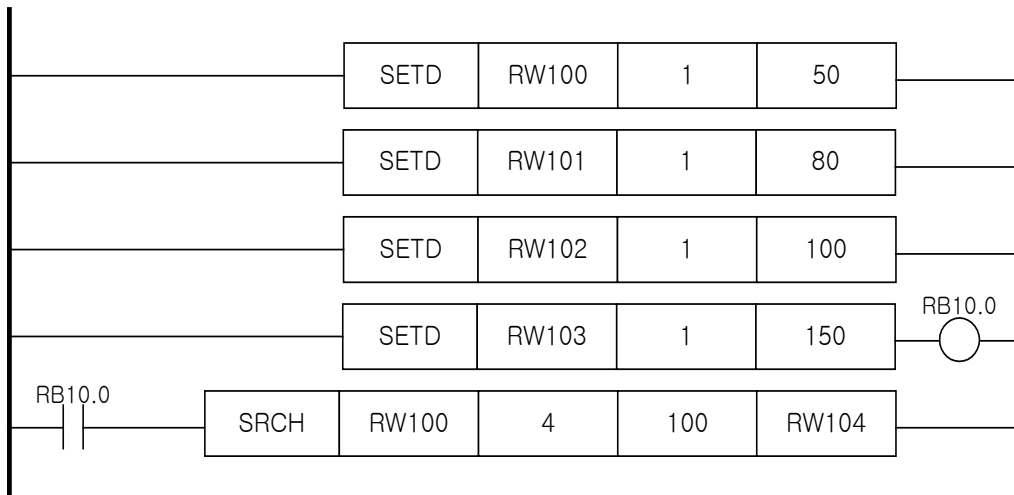
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	검색을 시작할 레지스터를 지정
Operand 2	폭(Width)을 지정합니다. OP1에 지정한 레지스터를 포함하여 그 이후 Width 만큼의 레지스터들이 검색할 범위 입니다
Operand 3	위치를 검색할 Data를 지정
Operand 4	검색한 Data의 위치를 저장할 레지스터를 지정합니다. 검색 시작위치로부터의 거리를 의미합니다 (0~Width-1) -1 이면 검색 결과가 없음을 의미합니다.
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 4개 레지스터에서 값이 100인 레지스터를 검색하여 [RW104]에 데이터의 위치값을 저장합니다.

[SRCH] 명령 수행결과 [RW104]에는 2가 저장됩니다.



5.6.12 SORT (Sorting)

■ [기능]

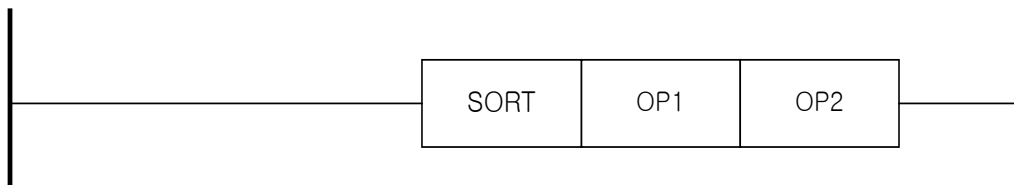
지정한 범위에 있는 레지스터의 값들을 크기 순으로(작은 값->큰 값) Sorting 합니다.
(Quick Sort 알고리즘 사용)

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	Sorting 할 레지스터
OP2	-	○	○	-	-	Sorting 할 데이터 폭(Width)

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	Sorting할 레지스터의 시작 위치를 지정
Operand 2	Sorting할 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 4개 레지스터 값을 정렬합니다.

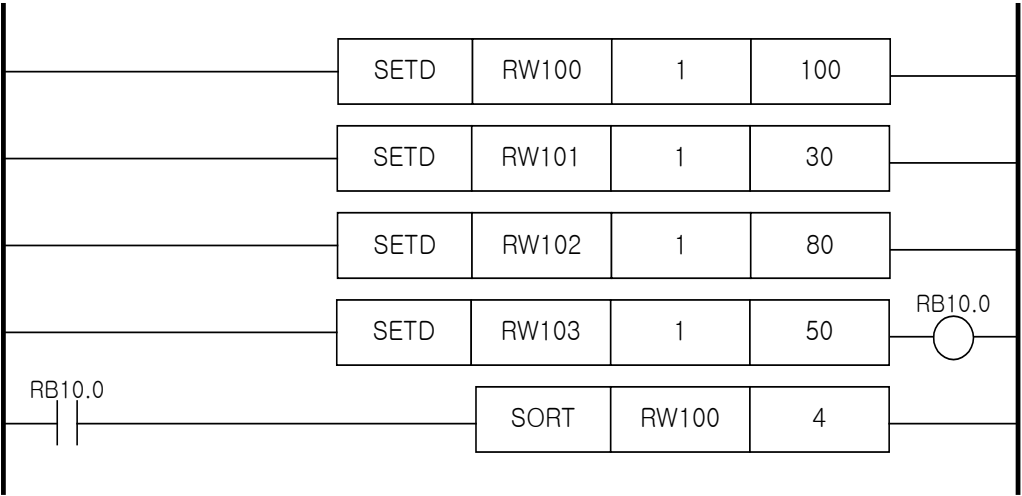
[SORT] 명령 수행결과 레지스터의 값들이 다음과 같이 정렬됩니다.

[RW100] = 30

[RW101] = 50

[RW102] = 60

[RW103] = 100



5.6.13 INL (=)

■ [기능]

대입 명령어 입니다.

실제 명령 입력은 기호(=)를 사용하여 입력합니다.

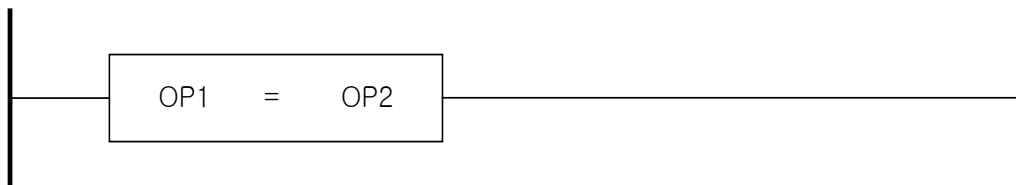
= 명령어의 오른쪽 레지스터(or CW)의 값을 왼쪽 레지스터에 대입합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	대입되는 레지스터
OP2	-	○	○	-	-	대입할 값이나 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

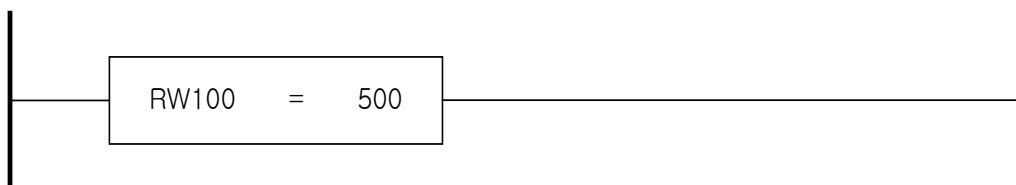


■ [설명]

Operand 1	대입되는 레지스터를 지정
Operand 2	대입할 값이나 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]에 500을 대입합니다.



5.6.14 INR (->)

■ [기능]

대입 명령어 입니다.

실제 명령 입력은 기호(->)를 사용하여 입력합니다.

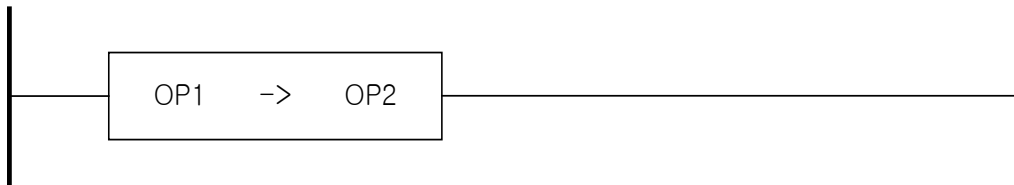
-> 명령어의 왼쪽 레지스터(or CW)의 값을 오른쪽 레지스터에 대입합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	대입할 값이나 레지스터
OP2	-	○	-	-	-	대입되는 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]

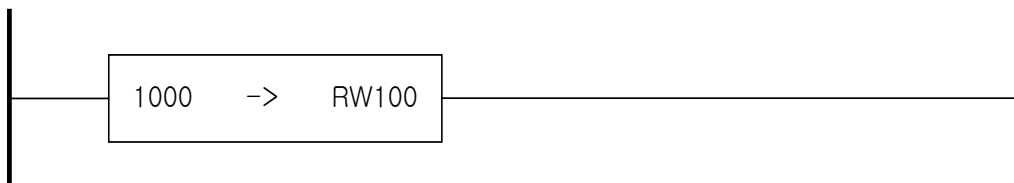


■ [설명]

Operand 1	대입할 값이나 레지스터를 지정
Operand 2	대입되는 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]에 1000을 대입합니다.



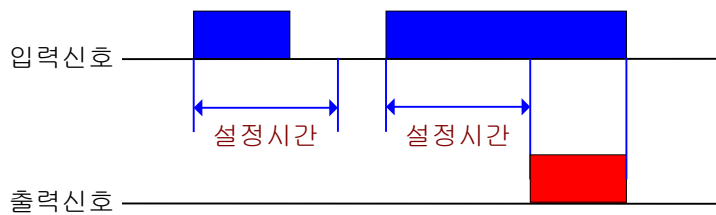
5.7 타이머 / 카운터 명령

5.7.1 TMRON (On Delay Timer)

■ [기능]

On Delay Timer의 기능을 하는 명령어입니다.

< On Delay Timer >



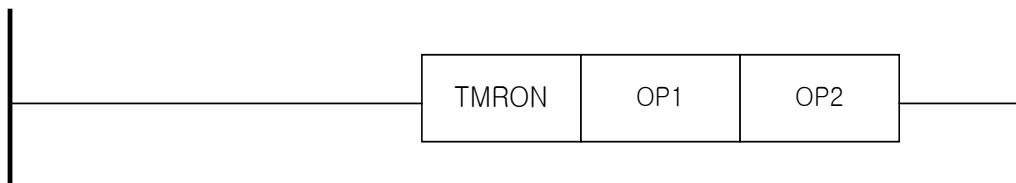
1. 입력신호가 ON된 시점부터 시간을 재서 설정한 시간보다 긴 경우에 출력접점이 ON 됩니다.
2. 만약에 설정한 시간에 도달하기 전에 입력신호가 OFF되고 다시 입력신호가 ON이 되면 처음부터 시간을 다시 재기 시작합니다.
3. 출력신호는 입력신호가 ON된 후부터 설정시간이 지난 뒤에 ON이 되고, 입력신호가 OFF되면 바로 OFF됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	타이머 설정값
OP2	○	-	-	-	-	결과 출력 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



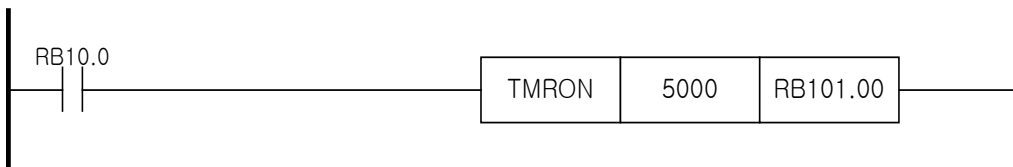
■ [설명]

입력신호(IS)	타이머 입력신호(입력 접점)를 BIT형 레지스터로 지정 합니다. 입력신호가 Off→On로 되면, 타이머를 초기화 하고 OP1에 경과 시간을 기록합니다. OP1에 기록된 경과 시간이 OP2에 설정한 값과 같아지거나 커지면, 출력은 On으로 됩니다
Operand 1	타이머 설정 값을 지정합니다. 이 곳에 설정한 값의 단위는 1ms 입니다. 설정 가능한 범위는 0 ~ 2147483000 입니다. (0 ~ 2147483초)
Operand 2	결과를 출력할 레지스터(출력 접점)를 지정합니다. 입력신호가 설정한 시간 이상으로 On이 되면, 출력은 On 입력신호가 Off되면, 출력은 바로 Off 됩니다.
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON되면 타이머가 5000msec 동안 동작됩니다.

설정된 타이머에 도달하면 [RB101.00]에 ON이 출력됩니다.

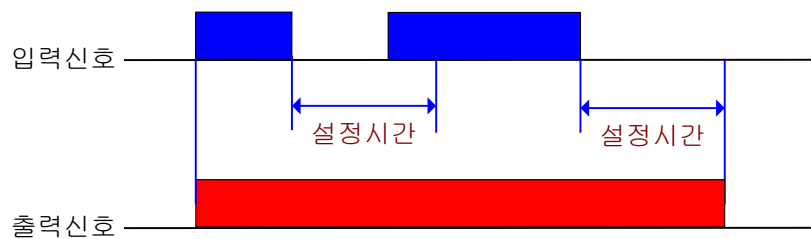


5.7.2 TMROFF (Off Delay Timer)

■ [기능]

Off Delay Timer의 기능을 하는 명령어입니다.

< Off Delay Timer >



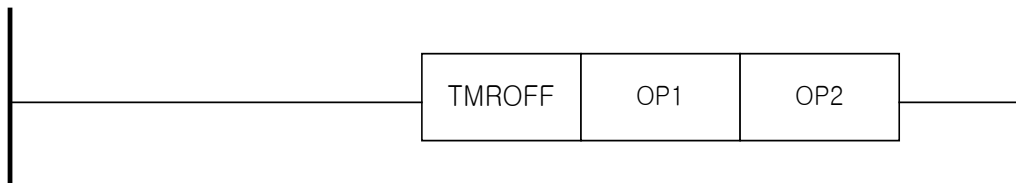
1. 입력신호가 On 되면 출력신호도 On 됩니다. (입력신호의 길이는 상관 없습니다)
2. 입력신호가 On에서 Off로 바뀌는 순간부터 시간을 재기 시작해서 설정시간이 경과할 때까지 출력을 On으로 유지하다가 Off를 출력합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	타이머 설정값
OP2	○	-	-	-	-	결과 출력 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



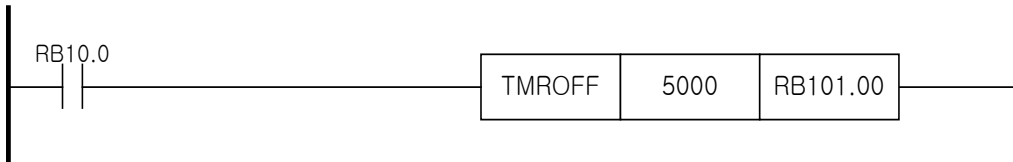
■ [설명]

입력신호(IS)	타이머 입력신호(입력 접점)를 BIT형 레지스터로 지정 합니다. 입력신호가 Off→On로 되면, 출력은 바로 On으로 됩니다. 입력신호가 On→Off로 되면, OP1에 경과 시간이 기록됩니다. OP1에 기록된 경과 시간이 OP2에 설정한 값과 같아지거나 커지면, 출력은 Off 됩니다
Operand 1	타이머 설정 값을 지정합니다. 이 곳에 설정한 값의 단위는 1ms 입니다. 설정 가능한 범위는 0 ~ 2147483000 입니다. (0~2147483초)
Operand 2	결과를 출력할 레지스터(출력 접점)를 지정합니다. 입력신호가 Off→On로 되면, 출력은 바로 On으로 됩니다. 입력신호가 On→Off로 되면, OP2에 설정한 시간만큼 경과 후에 출력이 Off 됩니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON에서 OFF로 전환되면 [RB101.00]에 ON이 출력되면서 타이머가 5000msec 동안 동작되고 됩니다.

설정된 타이머에 도달하면 [RB101.00]에 OFF가 출력됩니다.

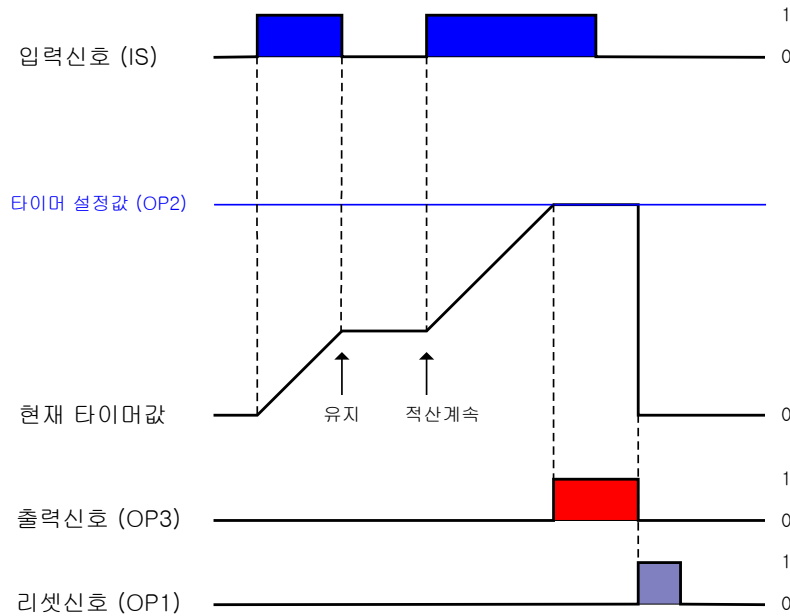


5.7.3 TMRONT (적산식 On Delay Timer)

■ [기능]

적산식 On Delay Timer의 기능을 하는 명령어입니다.

< 적산식 On Delay Timer >



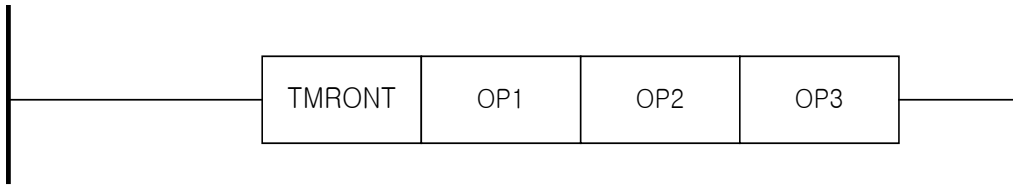
1. 입력신호가 ON이 되는 시간만 타이머가 누적됩니다.
2. 누적되는 On 타이머 시간이 OP2의 타이머 설정 값에 도달하면, 출력을 On으로 합니다.
한번 On 된 출력 신호는 리셋 신호가 On 될 때까지 계속 유지됩니다
3. 리셋 신호가 On이 되면, 출력은 즉시 Off 됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	타이머 Reset 신호
OP2	-	○	○	-	-	타이머 설정값
OP3	○	-	-	-	-	결과 출력 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

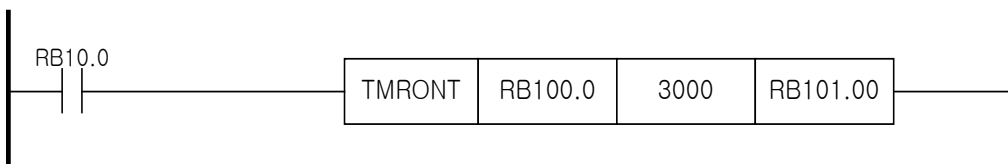
입력신호(IS)	타이머 입력신호(입력 접점)를 BIT형 레지스터로 지정 합니다. 누적되는 경과 시간이 OP2에 설정한 값과 같아지거나 커지면, 출력은 On으로 됩니다
Operand 1	타이머 리셋신호를 BIT형 레지스터로 지정합니다. 리셋 신호가 On이 되면, 출력은 무조건 Off로 되고 누적 타이머 값은 초기화 됩니다
Operand 2	타이머 설정 값을 지정합니다. 이 곳에 설정한 값의 단위는 1ms 입니다. 설정 가능한 범위는 0 ~ 2147483000 입니다. (0 ~ 2147483초)
Operand 3	결과를 출력할 레지스터(출력 접점)를 지정합니다. 누적 타이머 값이 OP2의 타이머 설정 값보다 크거나 같으면, 출력은 On이 됩니다. 리셋 신호가 On이 되면, 출력은 Off됩니다. 한번 On으로 된 출력은 리셋 신호가 On이 될 때까지, 계속 On으로 유지 됩니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON되면 3000msec 동안 타이머가 동작됩니다.

도중에 [RB10.0]이 OFF되면 누적 타이머는 현재 값을 유지합니다.

누적 타이머가 설정된 범위까지 도달되면 [RB101.00]에 ON이 출력됩니다.

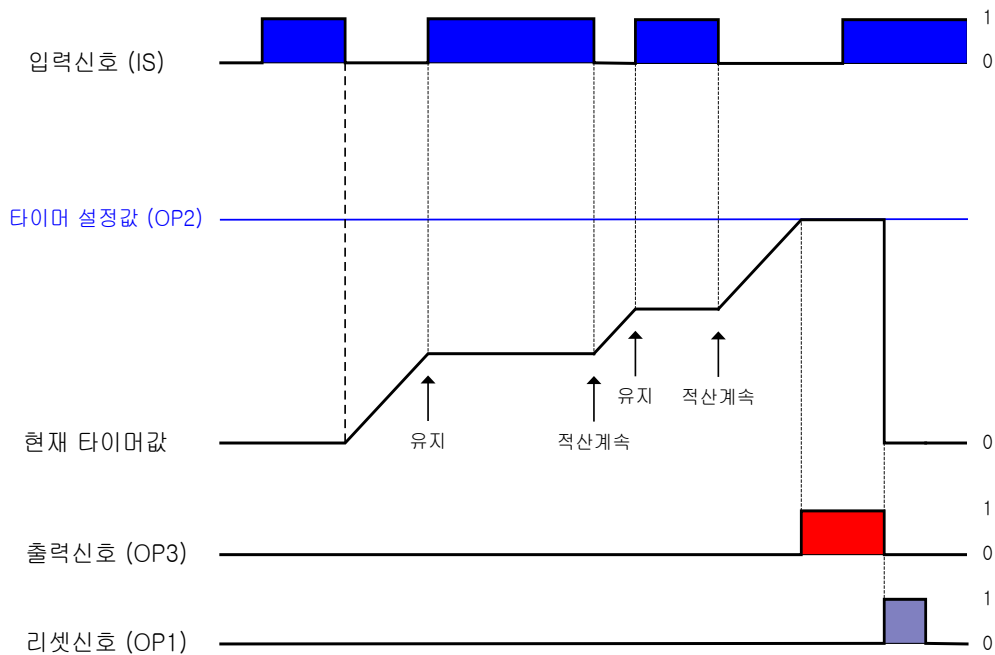


5.7.4 TMROFFT (적산식 Off Delay Timer)

■ [기능]

적산식 Off Delay Timer의 기능을 하는 명령어입니다.

< 적산식 Off Delay Timer >



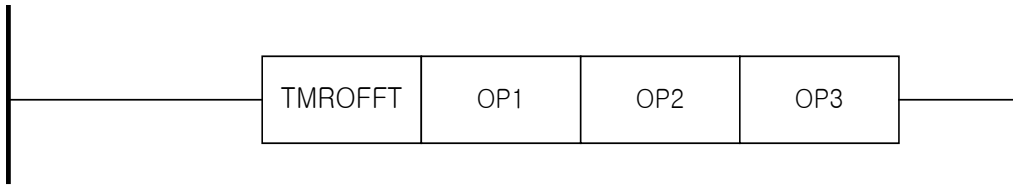
1. 입력신호가 On에서 Off으로 되면, Off가 되는 시간만 타이머가 누적됩니다.
2. 누적되는 Off 타이머 시간이 OP2의 타이머 설정 값에 도달하면, 출력을 On으로 합니다.
한번 On 된 출력 신호는 리셋 신호가 On 될 때까지 계속 유지됩니다.
3. 리셋 신호가 On이 되면, 출력은 즉시 Off 됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	타이머 Reset 신호
OP2	-	○	○	-	-	타이머 설정값
OP3	○	-	-	-	-	결과 출력 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

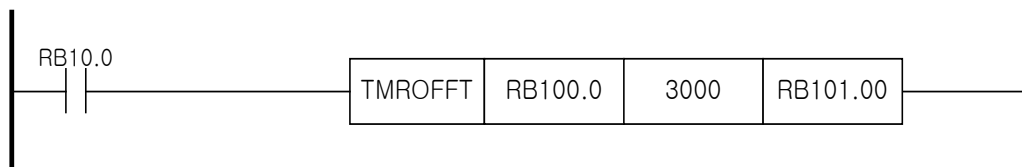
입력신호(IS)	타이머 입력신호(입력 접점)를 BIT형 레지스터로 지정 합니다. 누적되는 경과 시간이 OP2에 설정한 값과 같아지거나 커지면, 출력은 On으로 됩니다
Operand 1	타이머 리셋 신호를 BIT형 레지스터로 지정합니다. 리셋 신호가 On이 되면, 출력은 무조건 Off로 되고 누적 타이머 값은 초기화 됩니다
Operand 2	타이머 설정 값을 지정합니다. 이 곳에 설정한 값의 단위는 1ms 입니다. 설정 가능한 범위는 0 ~ 2147483000 입니다.
Operand 3	결과를 출력할 레지스터(출력 접점)를 지정합니다. 누적 타이머 값이 OP2의 타이머 설정 값보다 크거나 같으면, 출력은 On이 됩니다. 리셋 신호가 On이 되면, 출력은 Off됩니다. 한번 On으로 된 출력은 리셋 신호가 On이 될 때까지, 계속 On으로 유지 됩니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RB10.0]이 ON에서 OFF로 전환되면 3000msec 동안 타이머가 동작됩니다.

도중에 [RB10.0]이 ON되면 누적 타이머는 현재 값을 유지합니다.

누적 타이머가 설정된 범위까지 도달되면 [RB101.00]에 ON이 출력됩니다.



5.7.5 CNTUP (UP 카운터)

■ [기능]

UP 카운터의 기능을 합니다.

OP2 의 값에 따라서 두 가지 종류(링카운터, 단순카운터)의 동작을 합니다.

1. 링카운터 (Ring Counter)

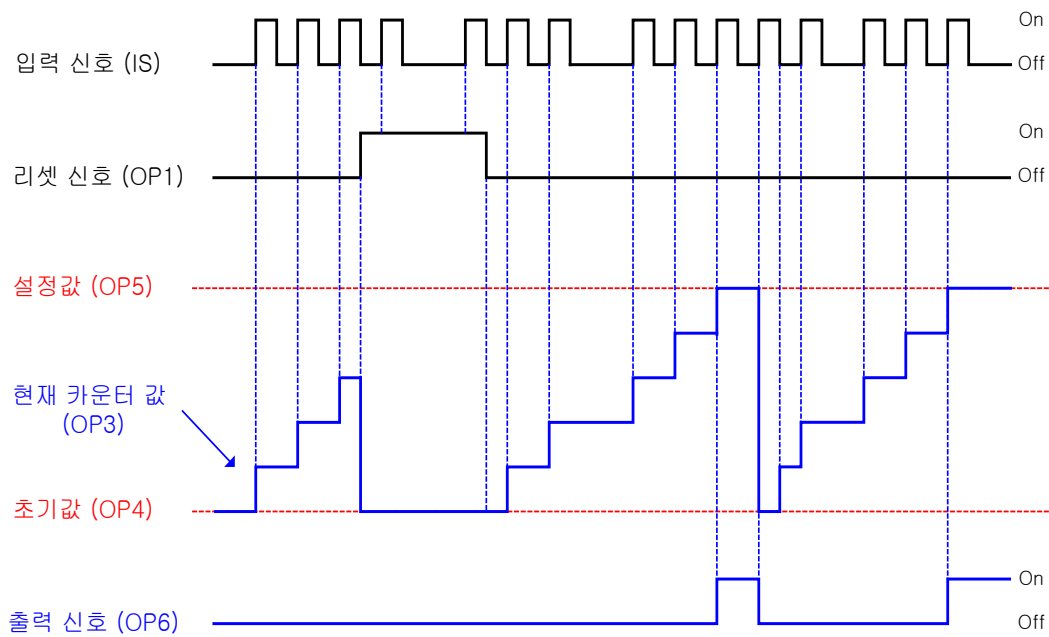
입력 신호로 펄스 1개가 입력되면 카운터 값이 1 증가 합니다.

리셋 신호가 On되면 카운터 값은 초기값으로 초기화 되며, 리셋 신호가 On 되어 있는 동안에는 입력 펄스가 들어와도 카운터 값이 증가하지 않습니다.

카운터 값이 설정 값에 도달하면 출력 신호가 On 됩니다.

이 상태에서 입력 신호가 다시 Off→On으로 되면, 카운터 값을 초기값으로 초기화 합니다. 따라서, 출력 신호는 Off 됩니다. 이때, 이 입력 신호를 놓치지 않기 위해서 On 되었던 신호가 Off로 되는 순간에 카운터 값을 1 증가 시킵니다.

UP-링카운터의 동작을 표현하면 다음과 같습니다.



2. 단순 카운터

입력 신호로 펄스 1개가 입력되면 카운터 값이 1 증가 합니다.

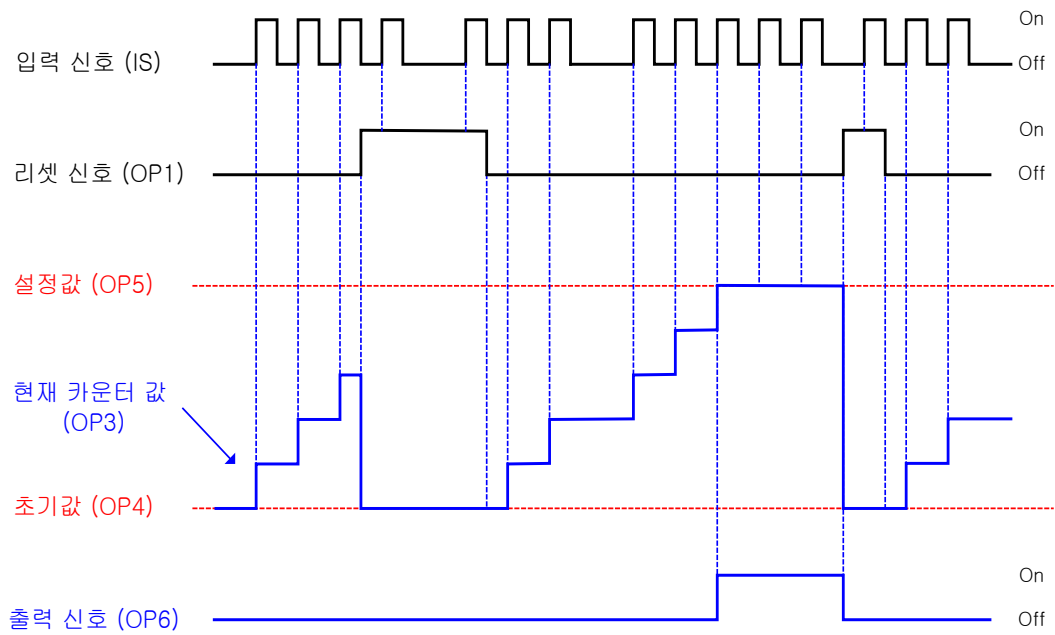
리셋 신호가 On되면 카운터 값은 초기값으로 초기화 되며, 리셋 신호가 On 되어 있는 동안에는 입력 펄스가 들어와도 카운터 값이 증가하지 않습니다.

카운터 값이 설정 값에 도달하면 출력 신호가 On 됩니다.

이 상태에서 입력 펄스가 들어와도 카운터 값은 더 이상 증가하지 않습니다.

이 상태에서 리셋 신호가 On 되면 카운터 값은 초기값으로 초기화 되고, 출력 신호는 Off 됩니다.

UP-단순 카운터의 동작을 표현하면 다음과 같습니다.

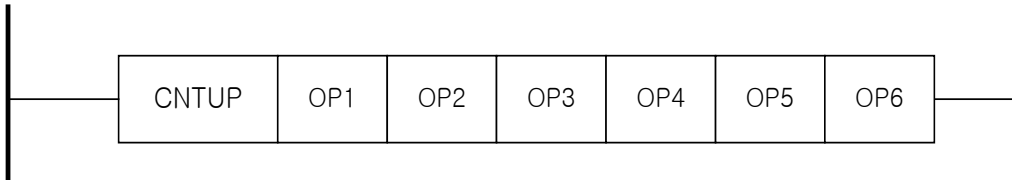


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	Reset 신호
OP2	○	-	-	-	-	카운터 타입
OP3	-	○	-	-	-	현재 카운터 값 저장 레지스터
OP4	-	○	○	-	-	카운터 초기값
OP5	-	○	○	-	-	카운터 설정값
OP6	○	-	-	-	-	카운터 출력 신호

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

입력신호(IS)	카운터 입력 신호로 펄스 입력을 받습니다
Operand 1	리셋 신호 입니다
Operand 2	카운터 타입을 설정합니다. 이 값이 0 이면 링카운터(Ring Counter)가 되고, 1이면 단순 카운터가 됩니다.
Operand 3	현재 카운터 값을 저장하는 레지스터 입니다
Operand 4	카운터가 초기화 될 때 사용할, 카운터 초기값입니다. 카운터가 초기화 될 때 현재 카운터 값(OP3)은 OP4의 값으로 초기화 됩니다
Operand 5	카운터 설정 값을 지정합니다. (설정값(OP5) > 초기값(OP4) 이어야 합니다) 현재 카운터 값(OP3)이 카운터 설정 값(OP5) 보다 크거나 같으면, 카운터 출력 신호(OP6)가 On 됩니다
Operand 6	카운터 출력 신호(출력 점점) 입니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[링카운터]

[RB10.1]이 OFF에서 ON으로 전환될 때마다 [RW102] 값이 1씩 증가합니다.

링카운터 상태이므로 [RW102]가 10에서 1증가되면 다시 0으로 초기화됩니다.

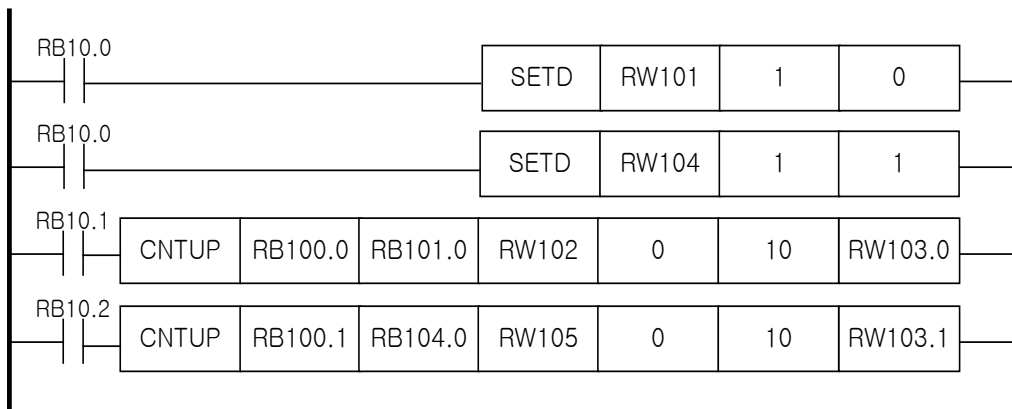
[RB100.0]이 ON되면 [RW102] 값은 0으로 초기화됩니다.

[단순카운터]

[RB10.2]가 OFF에서 ON으로 전환될 때마다 [RW105] 값이 1씩 증가합니다.

단순카운터 상태이므로 [RW105]가 10에서 1증가되면 10을 유지합니다.

[RB100.1]이 ON되면 [RW105] 값은 0으로 초기화됩니다.



5.7.6 CNTDN (DOWN 카운터)

■ [기능]

DOWN 카운터의 기능을 합니다.

PLC가 동작하면 현재 카운터 값은 초기값으로 초기화 되고, 입력 펄스가 들어올 때마다 카운터 값이 1씩 감소합니다.

OP2 의 값에 따라서 두 가지 종류(링카운터, 단순카운터)의 동작을 합니다.

1. 링카운터 (Ring Counter)

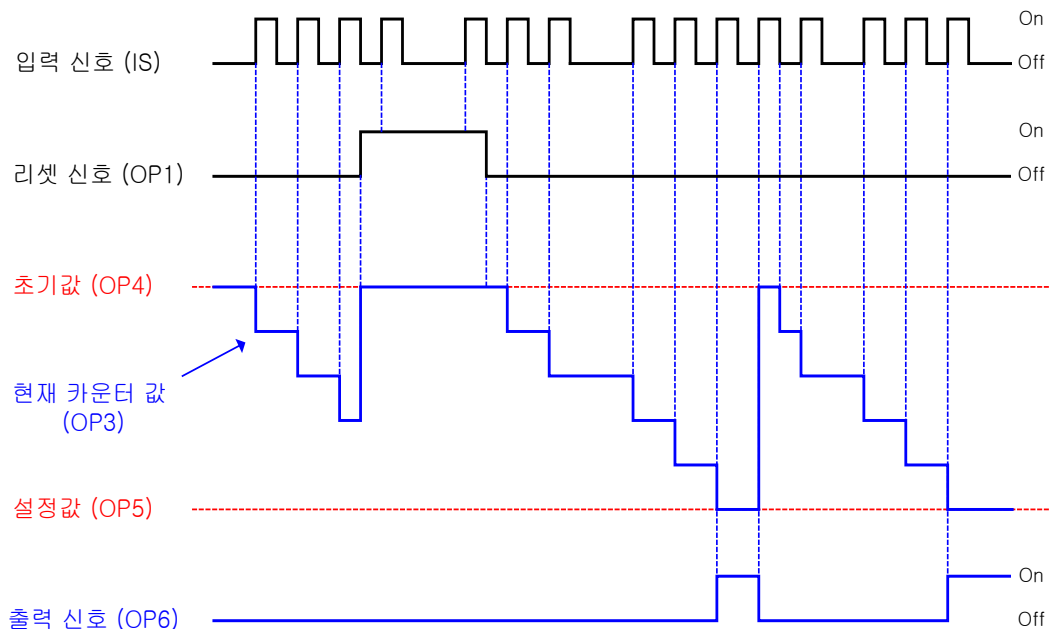
입력 신호로 펄스 1개가 입력되면 카운터 값이 1 감소합니다.

리셋 신호가 On되면 카운터 값은 초기값으로 초기화 되며, 리셋 신호가 On 되어 있는 동안에는 입력 펄스가 들어와도 카운터 값이 감소하지 않습니다.

카운터 값이 설정 값과 같아지면 출력 신호가 On 됩니다.

이 상태에서 입력 신호가 다시 Off→On으로 되면, 카운터 값을 초기값으로 초기화 합니다. 따라서, 출력 신호는 Off 됩니다. 이때, 이 입력 신호를 놓치지 않기 위해서 On 되었던 신호가 Off로 되는 순간에 카운터 값을 1 감소 시킵니다.

DOWN-링카운터의 동작을 표현하면 다음과 같습니다.



2. 단순 카운터

입력 신호로 펄스 1개가 입력되면 카운터 값이 1 감소 합니다.

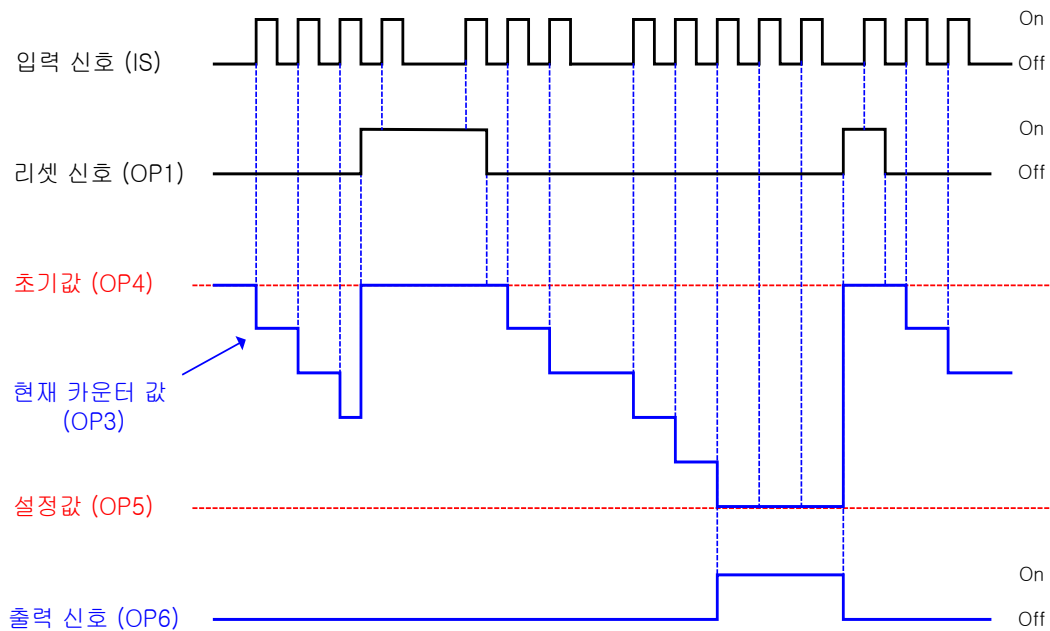
리셋 신호가 On되면 카운터 값은 초기값으로 초기화 되며, 리셋 신호가 On 되어 있는 동안에는 입력 펄스가 들어와도 카운터 값이 감소하지 않습니다.

카운터 값이 설정 값과 같아지면 출력 신호가 On 됩니다.

이 상태에서 입력 펄스가 들어와도 카운터 값은 더 이상 감소하지 않습니다.

이 상태에서 리셋 신호가 On 되면 카운터 값은 초기값으로 초기화 되고, 출력 신호는 Off 됩니다.

DOWN-단순 카운터의 동작을 표현하면 다음과 같습니다.

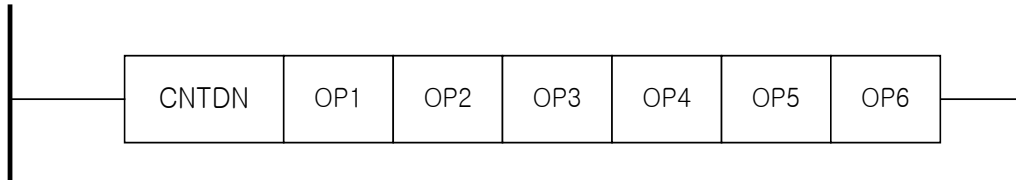


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	Reset 신호
OP2	○	-	-	-	-	카운터 타입
OP3	-	○	-	-	-	현재 카운터 값 저장 레지스터
OP4	-	○	○	-	-	카운터 초기값
OP5	-	○	○	-	-	카운터 설정값
OP6	○	-	-	-	-	카운터 출력 신호

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

입력신호(IS)	카운터 입력 신호로 펄스 입력을 받습니다
Operand 1	리셋 신호입니다
Operand 2	카운터 타입을 설정합니다 이 값이 0 이면 링카운터(Ring Counter)가 되고, 1이면 단순 카운터가 됩니다
Operand 3	현재 카운터 값을 저장하는 레지스터입니다
Operand 4	카운터가 초기화 될 때 사용할, 카운터 초기값입니다. 카운터가 초기화 될 때 현재 카운터 값(OP3)은 OP4의 값으로 초기화 됩니다
Operand 5	카운터 설정 값을 지정합니다. (설정값(OP5) < 초기값(OP4) 이어야 합니다) 현재 카운터 값(OP3)이 카운터 설정 값(OP5) 보다 작거나 같으면, 카운터 출력 신호(OP6)가 On 됩니다
Operand 6	카운터 출력 신호(출력 점점)입니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[링카운터]

[RB10.1]이 OFF에서 ON으로 전환될 때마다 [RW102] 값이 1씩 감소합니다.

링카운터 상태이므로 [RW102]가 5에서 1 감소되면 다시 10으로 초기화됩니다.

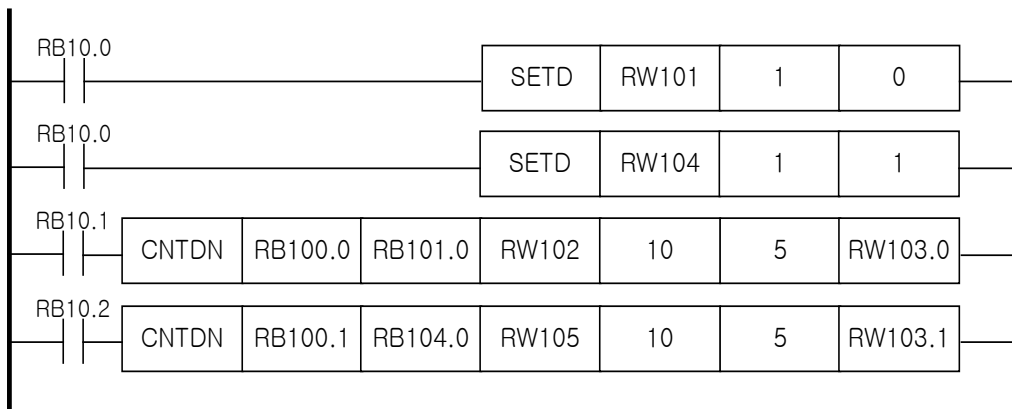
[RB100.0]이 ON되면 [RW102] 값은 10으로 초기화됩니다.

[단순카운터]

[RB10.2]가 OFF에서 ON으로 전환될 때마다 [RW105] 값이 1씩 감소합니다.

단순카운터 상태이므로 [RW105]가 5에서 1 감소되면 5를 유지합니다.

[RB100.1]이 ON되면 [RW105] 값은 10으로 초기화됩니다.



5.7.7 CNTUPDN (UP DOWN 카운터)

■ [기능]

UP 카운터와 DOWN 카운터의 기능을 합니다.

CNTUP, CNTDN 명령과는 달리 링카운터(Ring Counter)로 고정 되어 있습니다.

1. 기능 설명

UP 신호로 펄스 1개가 입력되면 카운터 값이 1 증가 합니다.

DOWN 신호로 펄스 1개가 입력되면 카운터 값이 1 감소 합니다.

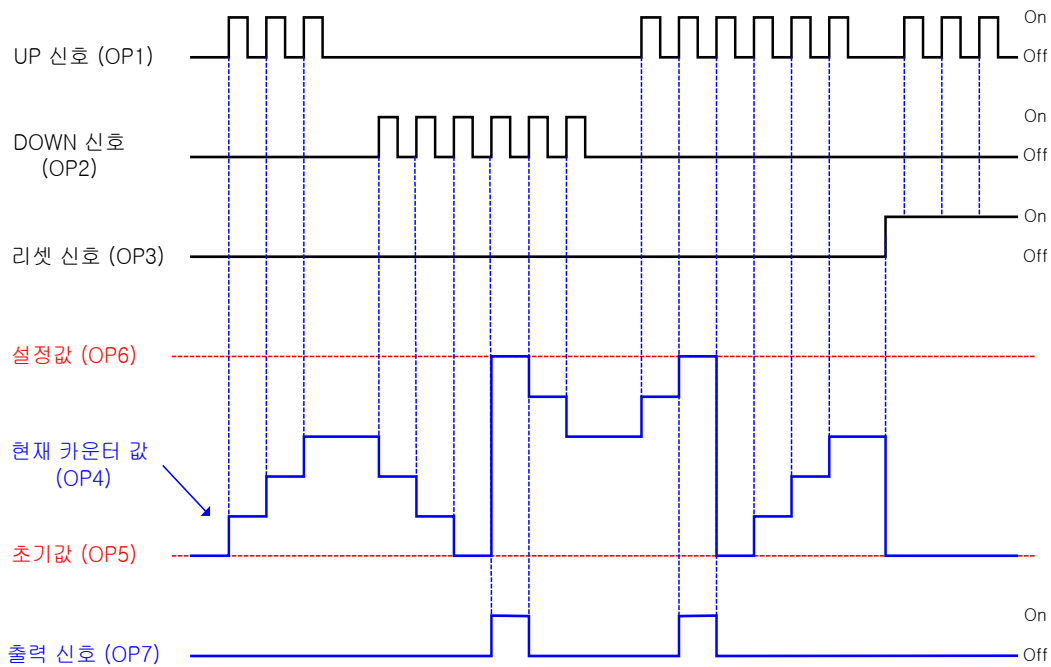
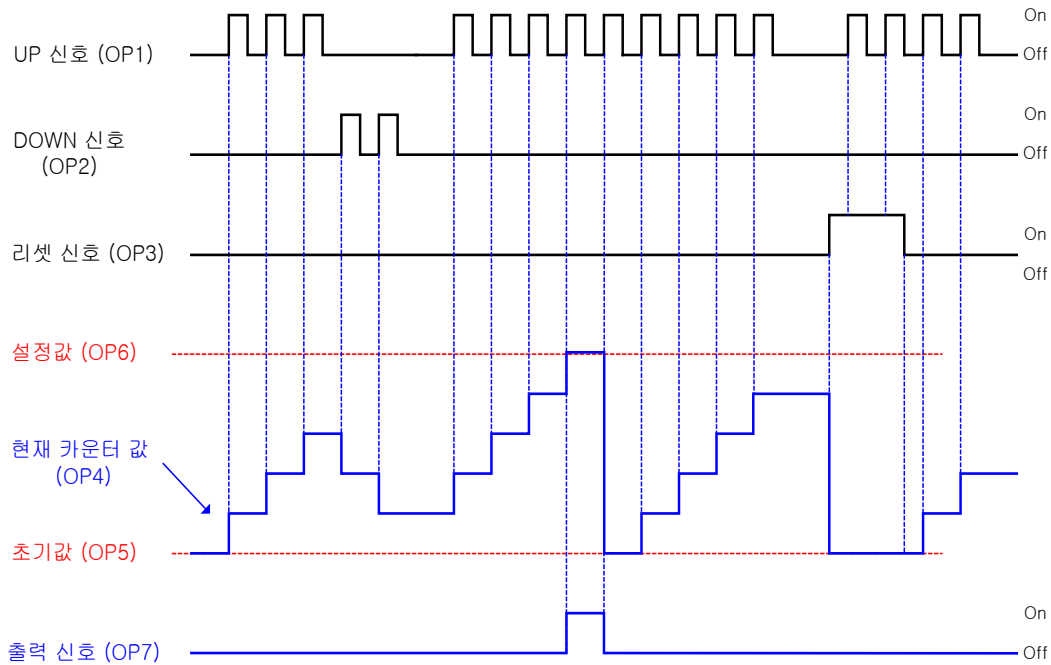
리셋 신호가 On되면 카운터 값은 0으로 초기화 되며, 리셋 신호가 On 되어 있는 동안에는 UP 신호나 DOWN 신호가 들어와도 카운터 값이 변하지 않습니다.

UP 신호가 들어와서 카운터 값이 설정 값에 도달하면 출력 신호가 On 됩니다.

이 상태에서 UP 신호가 다시 Off→On으로 되면, 카운터 값을 초기값으로 초기화 합니다. 따라서, 출력 신호는 Off 됩니다.

카운터 값이 0인 상황에서 DOWN 신호가 들어오면, 링카운터 이므로 카운터 값은 설정값으로 됩니다. 카운터 값이 설정값이 되었으므로 출력 신호는 On 됩니다.

2. UP/DOWN 카운터의 동작을 표현하면 다음과 같습니다.

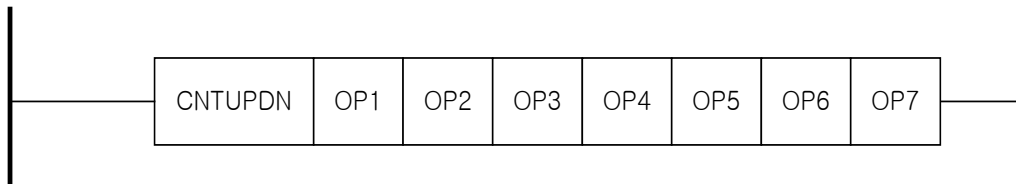


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	UP 카운터 입력 신호
OP2	○	-	-	-	-	DOWN 카운터 입력 신호
OP3	○	-	-	-	-	Reset 신호
OP4	-	○	-	-	-	현재 카운터 저장 레지스터
OP5	-	○	○	-	-	카운터 초기값
OP6	-	○	○	-	-	카운터 설정값
OP7	○	-	-	-	-	카운터 출력 신호

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



■ [설명]

Operand 1	UP 카운터 입력 신호입니다
Operand 2	DOWN 카운터 입력 신호입니다
Operand 3	리셋 신호 입니다.
Operand 4	현재 카운터 값을 저장하는 레지스터 입니다
Operand 5	카운터가 초기화 될 때 사용할, 카운터 초기값입니다 카운터가 초기화 될 때 현재 카운터 값(OP4)은 OP5의 값으로 초기화 됩니다
Operand 6	카운터 설정 값을 지정합니다. (설정값(OP6) > 초기값(OP5) 이어야 합니다) 현재 카운터 값(OP4)이 카운터 설정 값(OP6)과 같아지면, 카운터 출력 신호(OP7)가 On 됩니다
Operand 7	카운터 출력 신호(출력 점점) 입니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

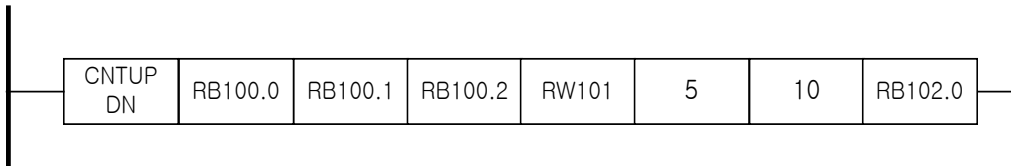
■ [예제]

[RB100.0]이 OFF에서 ON으로 전환될 때마다 [RW101] 값이 1씩 증가합니다.

[RB100.1]이 OFF에서 ON으로 전환될 때마다 [RW101] 값이 1씩 감소합니다.

[링카운터]이므로 [RW101]가 10에서 1 증가되면 다시 5로 초기화 되고, 5에서 1 감소되면 다시 10으로 초기화 됩니다.

[RB100.2]이 ON되면 [RW101] 값은 5로 초기화됩니다.



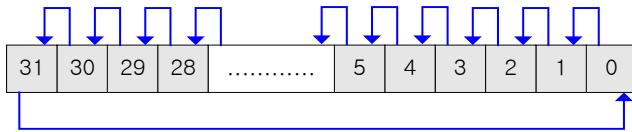
5.8 데이터 조작 명령

5.8.1 ROTL (비트 좌회전, Rotation Left)

■ [기능]

지정한 레지스터의 모든 비트 값을 왼쪽 방향으로 지정한 크기 만큼 회전 합니다.

1회전은 모든 비트의 값들을 왼쪽 방향으로 1만큼 이동(Shift)하고 31번째 비트의 값은 0번째 비트로 이동하는 것을 의미 합니다.



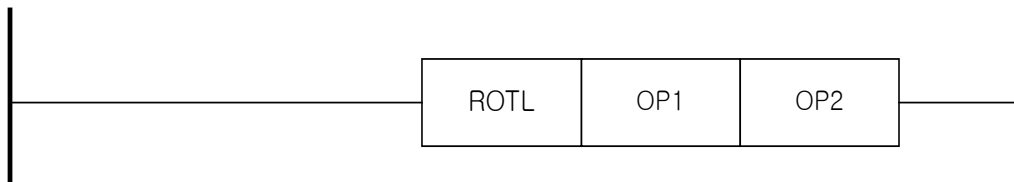
n회전은 1회전을 n번 반복하는 것을 의미합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	비트 좌회전 할 레지스터
OP2	-	○	○	-	-	회전 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

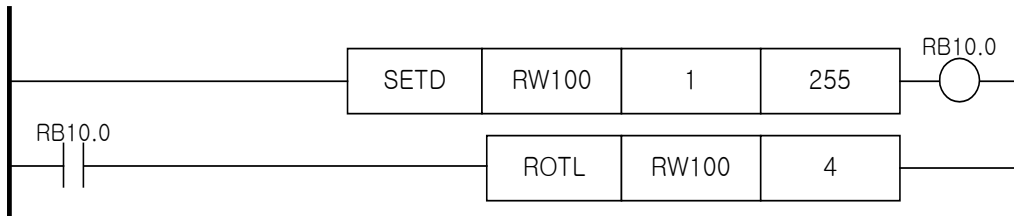
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	비트 좌회전 할 레지스터를 지정
Operand 2	회전 이동할 크기를 지정합니다. n을 입력하면 1회전이 n번 일어납니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터를 왼쪽으로 4Bit만큼 회전합니다.

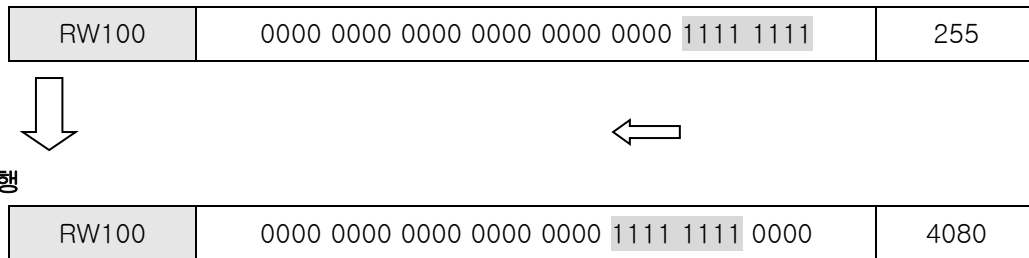
31Bit 자리를 넘어 이동되는 데이터는 0Bit 자리로 이동됩니다.

[ROTL] 명령 수행결과 [RW100]에는 4080이 저장됩니다.



4만큼 이동

ROTL 명령 수행



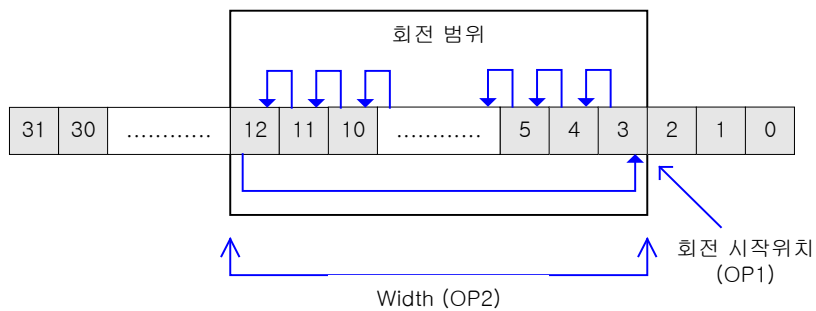
5.8.2 ROTL2 (Advanced Rotation Left)

■ [기능]

지정한 범위에서 비트 값을 왼쪽으로 회전 이동 합니다.

ROTL 명령은 레지스터의 모든 비트 값을 왼쪽으로 회전하는 명령인 반면에,

ROTL2 명령은 레지스터에서 특정 범위의 비트들을 왼쪽으로 회전하는 명령입니다.

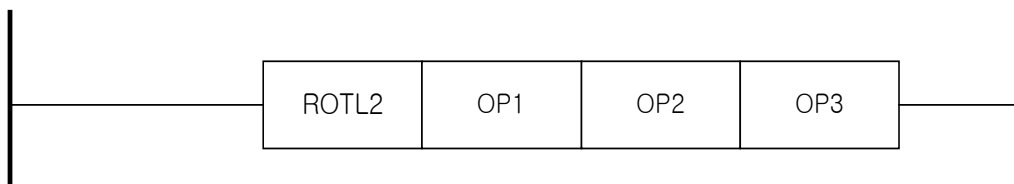


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	—	—	—	—	비트 좌회전할 레지스터
OP2	—	○	○	—	—	회전 이동할 폭(Width)
OP3	—	○	○	—	—	회전 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

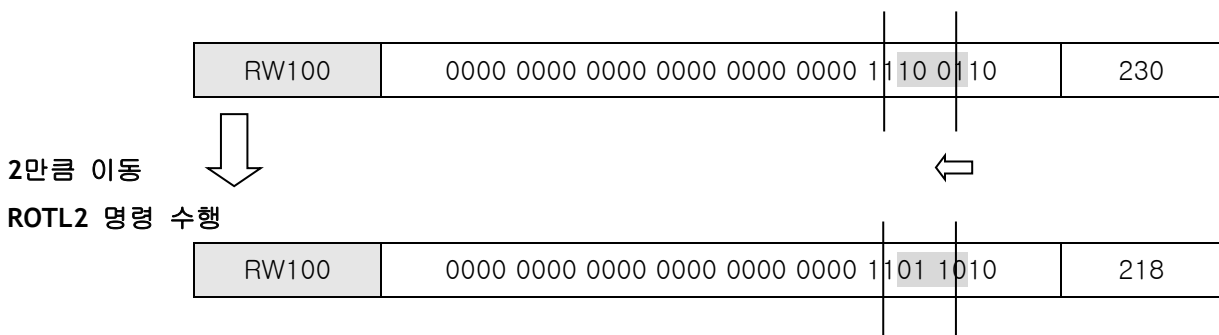
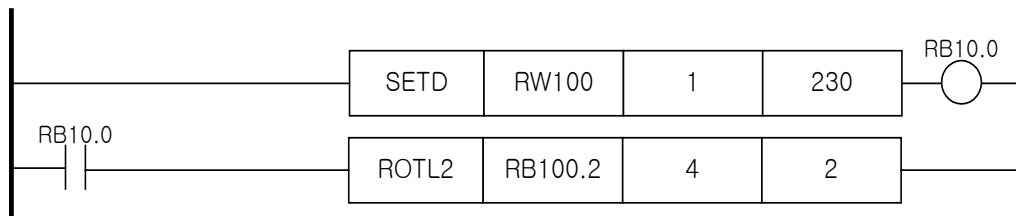
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	회전 이동할 레지스터를 지정합니다. 레지스터에서 회전 범위를 지정하기 위해서는 회전을 시작할 BIT의 위치가 필요합니다. 따라서, 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 2	회전 이동할 폭(Width)을 지정합니다. OP1에서 지정한 시작 BIT와 OP2에서 지정한 Width를 사용하면 회전 이동의 범위가 결정됩니다
Operand 3	회전 이동할 크기를 지정합니다. n을 입력하면 1회전이 n번 일어납니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100.2]부터 4Bit의 데이터를 왼쪽으로 2Bit만큼 회전합니다.

설정 범위를 넘어 이동되는 데이터는 처음으로 이동됩니다.

[ROTL2] 명령 수행결과 [RW100]에는 218이 저장됩니다.

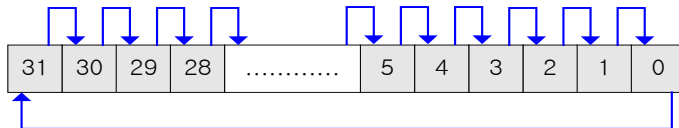


5.8.3 ROTR (비트 우회전, Rotation Right)

■ [기능]

지정한 레지스터의 모든 비트 값을 오른쪽 방향으로 지정한 크기 만큼 회전 합니다.

1회전은 모든 비트의 값들을 오른쪽 방향으로 1만큼 이동(Shift)하고 0번째 비트의 값은 31번째 비트로 이동하는 것을 의미 합니다.



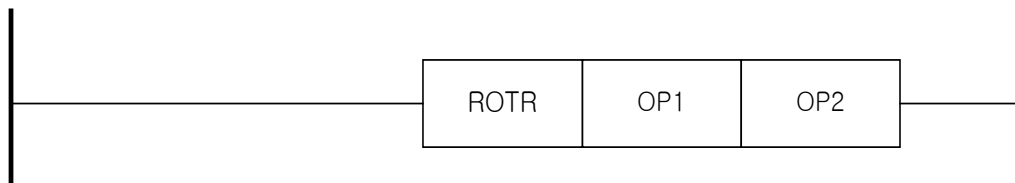
n회전은 1회전을 n번 반복하는 것을 의미합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	비트 우회전 할 레지스터
OP2	-	○	○	-	-	회전 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

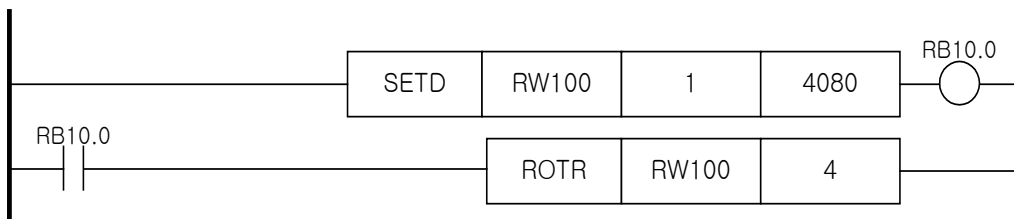
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	비트 우회전 할 레지스터를 지정
Operand 2	회전 이동할 크기를 지정합니다. n을 입력하면 1회전이 n번 일어납니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터를 오른쪽으로 4Bit만큼 회전합니다.

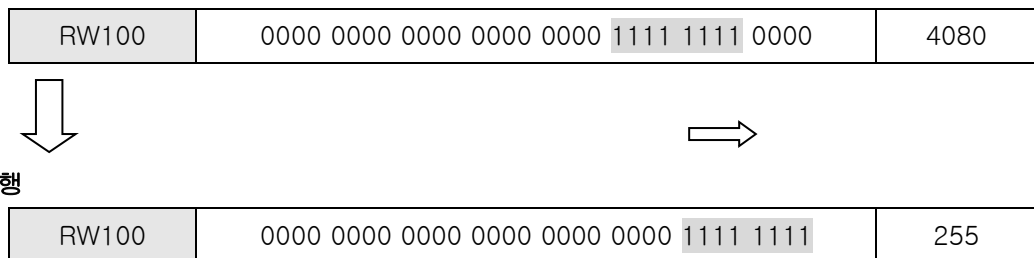
0Bit 자리를 넘어 이동되는 데이터는 31Bit 자리로 이동됩니다.

[ROTR] 명령 수행결과 [RW100]에는 255이 저장됩니다.



4만큼 이동

ROTR 명령 수행



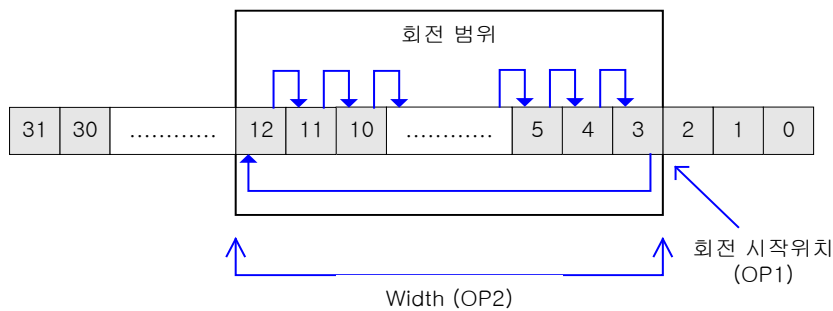
5.8.4 ROTR2 (Advanced Rotation Right)

■ [기능]

지정한 범위에서 비트 값을 오른쪽으로 회전 이동 합니다.

ROTR 명령은 레지스터의 모든 비트 값을 오른쪽으로 회전하는 명령인 반면에,

ROTR2 명령은 레지스터에서 특정 범위의 비트를 오른쪽으로 회전하는 명령입니다.

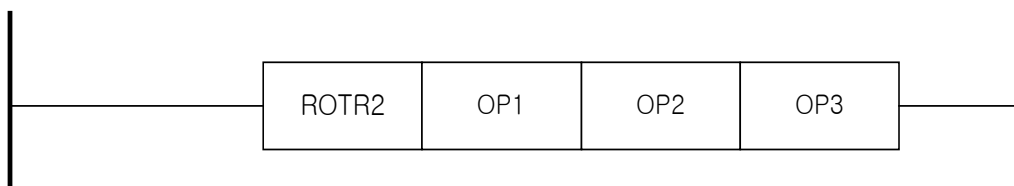


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	비트 우회전할 레지스터
OP2	-	○	○	-	-	회전 이동할 폭(Width)
OP3	-	○	○	-	-	회전 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

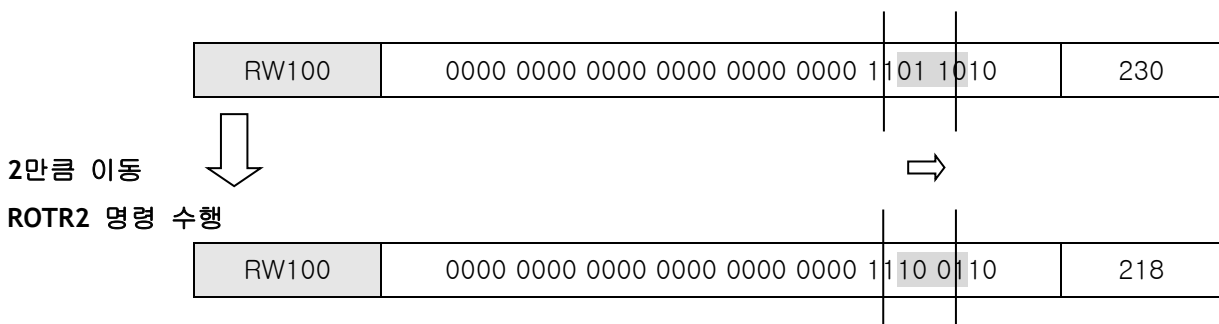
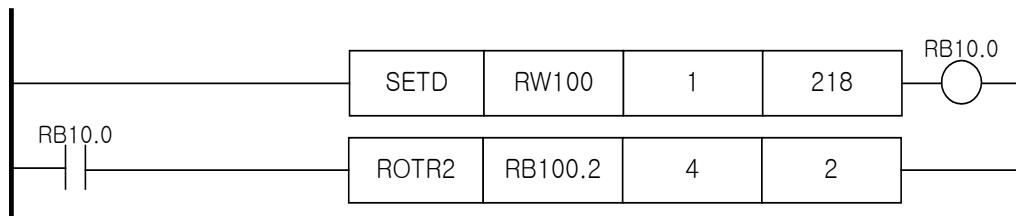
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	회전 이동할 레지스터를 지정합니다. 레지스터에서 회전 범위를 지정하기 위해서는 회전을 시작할 BIT의 위치가 필요합니다. 따라서, 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 2	회전 이동할 폭(Width)을 지정합니다. OP1에서 지정한 시작 BIT와 OP2에서 지정한 Width를 사용하면 회전 이동의 범위가 결정됩니다
Operand 3	회전 이동할 크기를 지정합니다. n을 입력하면 1회전이 n번 일어납니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100.2]부터 4Bit의 데이터를 오른쪽으로 2Bit만큼 회전합니다.

설정 범위를 넘어 이동되는 데이터는 처음으로 이동됩니다.

[ROTR2] 명령 수행결과 [RW100]에는 230이 저장됩니다.



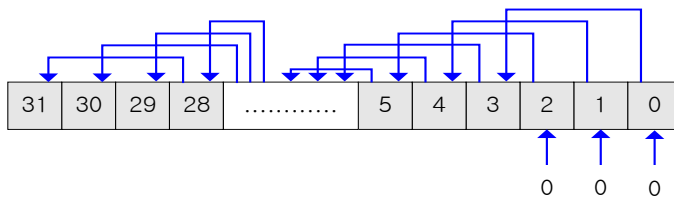
5.8.5 SFTL (비트 좌 Shift, Shift Left)

■ [기능]

지정한 레지스터의 모든 비트 값을 왼쪽 방향으로 지정한 크기 만큼 쉬프트(Shift) 합니다.

왼쪽 방향으로 1만큼 쉬프트(Shift)하면, 가장 왼쪽 비트 값은 이동할 곳이 없으므로 기록이 되지 않고 사라집니다. 가장 오른쪽의 비트는 원래의 비트 값이 왼쪽으로 이동하였으므로 지정된 값이 없습니다. 이 값은 0으로 됩니다. 마찬가지로, 왼쪽 방향으로 3만큼 쉬프트(Shift)하면 가장 왼쪽 3개의 비트 값은 사라지고 가장 오른쪽의 3개의 비트 값은 0이 됩니다.

< 왼쪽 방향으로 3 쉬프트 >

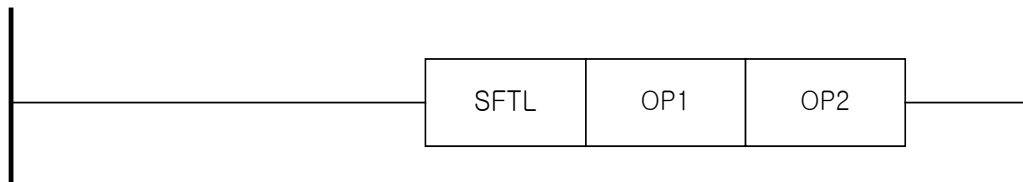


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	비트 좌 Shift 할 레지스터
OP2	-	○	○	-	-	Shift 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

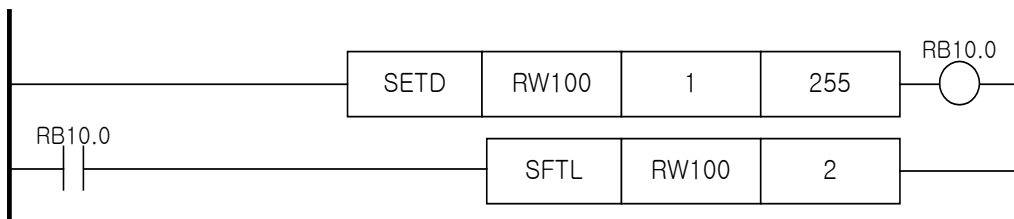
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	왼쪽으로 비트 Shift 이동 할 레지스터를 지정
Operand 2	Shift 이동할 크기를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터를 왼쪽으로 4Bit만큼 Shift합니다.

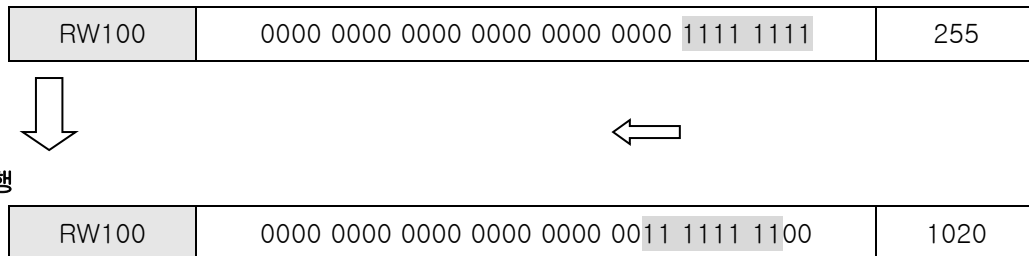
31Bit 자리를 넘어 이동되는 데이터는 제거됩니다.

[SFTL] 명령 수행결과 [RW100]에는 1020이 저장됩니다.



2만큼 이동

SFTL 명령 수행



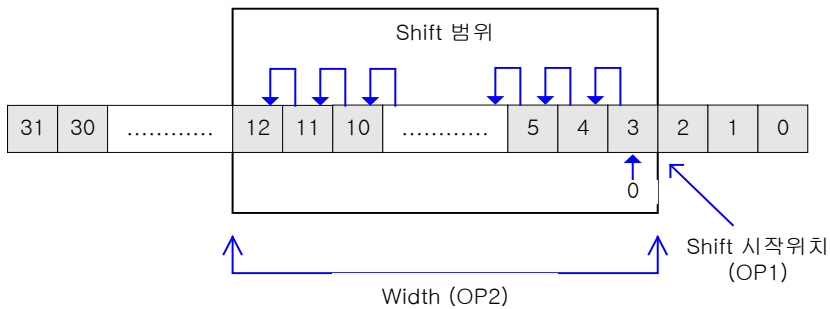
5.8.6 SFTL2 (Advanced Shift Left)

■ [기능]

지정한 범위에서 비트 값을 왼쪽으로 Shift 이동 합니다.

SFTL 명령은 레지스터의 모든 비트 값을 왼쪽으로 Shift 이동하는 명령인 반면에,

SFTL2 명령은 레지스터에서 특정 범위의 비트들을 왼쪽으로 Shift 이동하는 명령입니다.

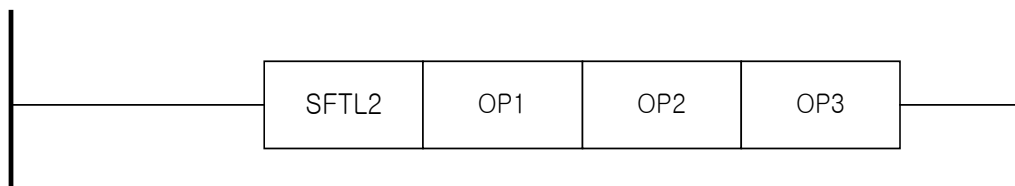


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	비트 좌 Shift할 레지스터
OP2	-	○	○	-	-	Shift 이동할 폭(Width)
OP3	-	○	○	-	-	Shift 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

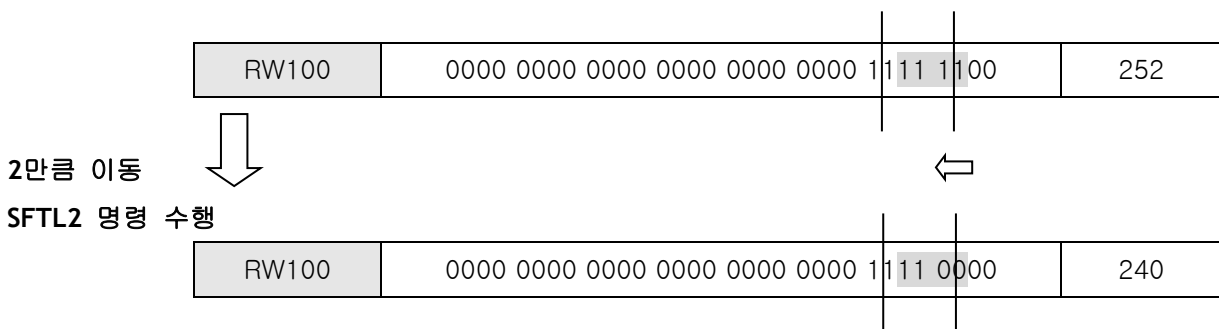
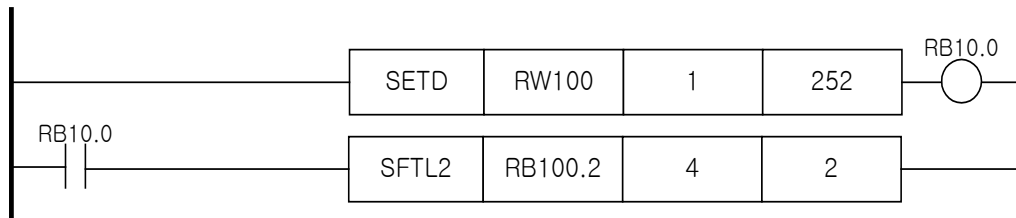
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	Shift 이동할 레지스터를 지정합니다. 레지스터에서 Shift 범위를 지정하기 위해서는 Shift를 시작할 BIT의 위치가 필요합니다. 따라서, 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 2	Shift 이동할 폭(Width)을 지정합니다. OP1에서 지정한 시작 BIT와 OP2에서 지정한 Width를 사용하면 Shift 이동의 범위가 결정됩니다
Operand 3	Shift 이동할 크기를 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100.2]부터 4Bit의 데이터를 왼쪽으로 2Bit만큼 Shift회전합니다.

설정 범위를 넘어 이동되는 데이터는 제거됩니다.

[SFTL2] 명령 수행결과 [RW100]에는 240이 저장됩니다.



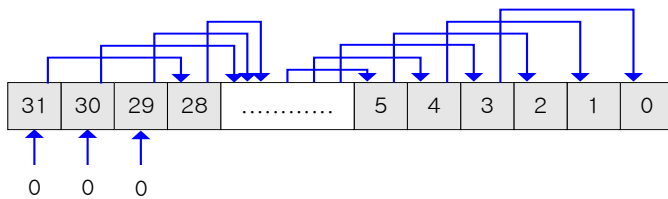
5.8.7 SFTR (비트 우 Shift, Shift Right)

■ [기능]

지정한 레지스터의 모든 비트 값을 오른쪽 방향으로 지정한 크기 만큼 쉬프트(Shift) 합니다.

오른쪽 방향으로 1만큼 쉬프트(Shift)하면, 가장 오른쪽 비트 값은 이동할 곳이 없으므로 기록이 되지 않고 사라집니다. 가장 왼쪽의 비트는 원래의 비트 값이 오른쪽으로 이동하였으므로 지정된 값이 없습니다. 이 값은 0으로 됩니다. 마찬가지로, 오른쪽 방향으로 3만큼 쉬프트(Shift)하면 가장 오른쪽 3개의 비트 값은 사라지고 가장 왼쪽의 3개의 비트 값은 0이 됩니다.

< 오른쪽 방향으로 3 쉬프트 >

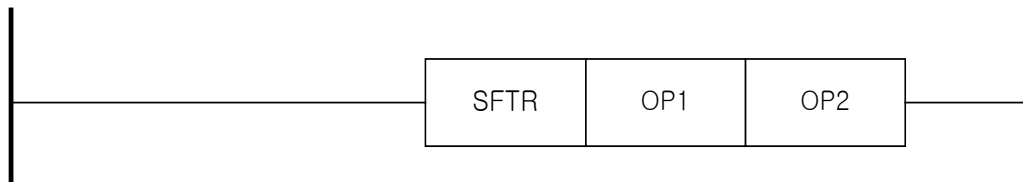


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	-	-	비트 우 Shift할 레지스터
OP2	-	○	○	-	-	Shift 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

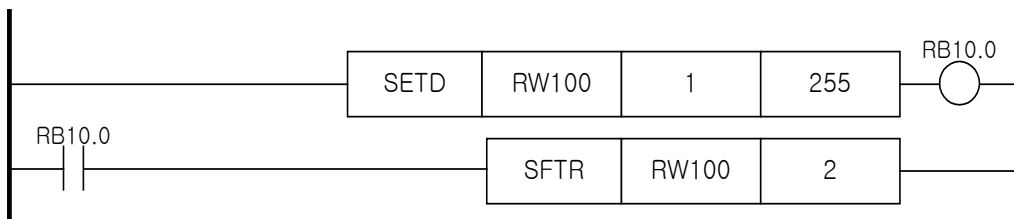
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	오른쪽으로 비트 Shift 이동 할 레지스터를 지정
Operand 2	Shift 이동할 크기를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 데이터를 오른쪽으로 4Bit만큼 Shift합니다.

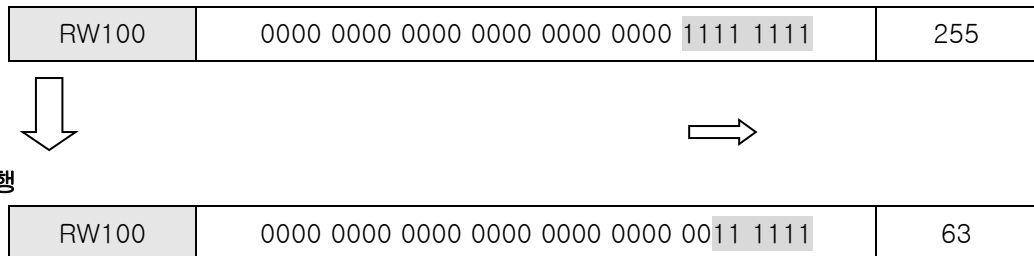
0Bit 자리를 넘어 이동되는 데이터는 제거됩니다.

[SFTR] 명령 수행결과 [RW100]에는 63이 저장됩니다.



2만큼 이동

SFTR 명령 수행

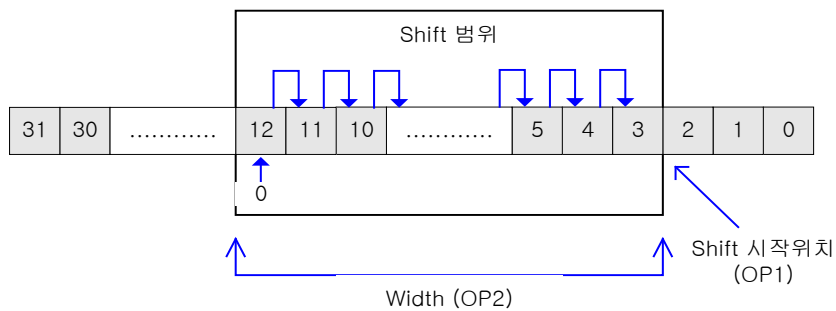


5.8.8 SFTR2 (Advanced Shift Right)

■ [기능]

지정한 범위에서 비트 값을 오른쪽으로 Shift 이동 합니다.

SFTR 명령은 레지스터의 모든 비트 값을 오른쪽으로 Shift 이동하는 명령인 반면에,
SFTR2 명령은 레지스터에서 특정 범위의 비트들을 오른쪽으로 Shift 이동하는 명령입니다.

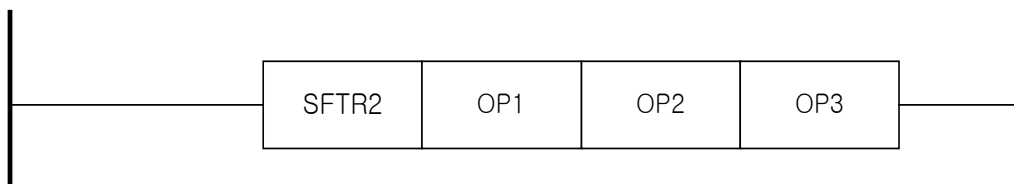


■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	비트 우 Shift 할 레지스터
OP2	-	○	○	-	-	Shift 이동할 폭(Width)
OP3	-	○	○	-	-	Shift 이동할 크기

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

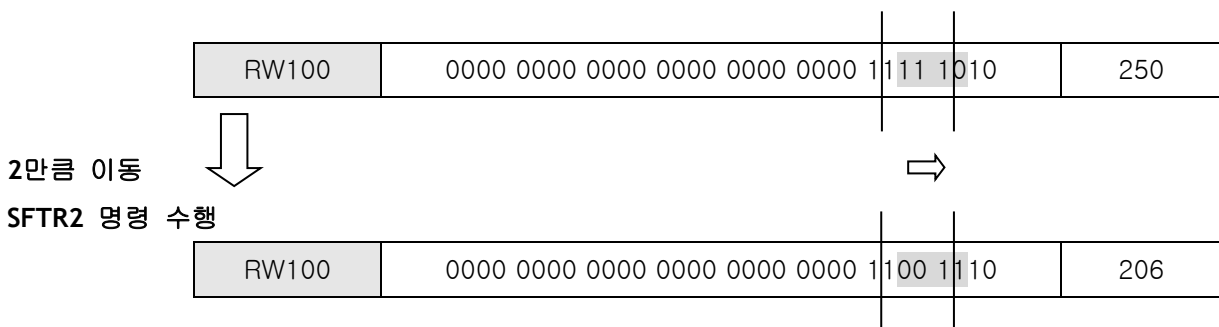
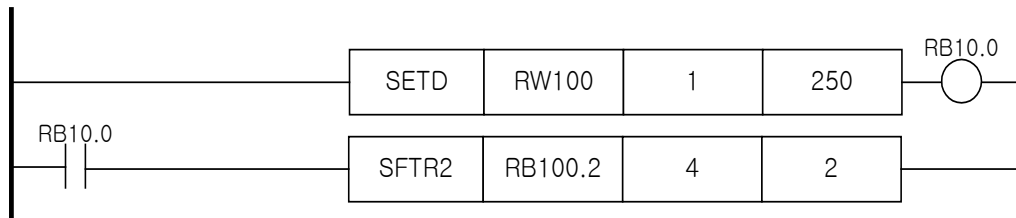
실행 조건	명령 실행 조건을 삽입 가능
Operand 1	Shift 이동할 레지스터를 지정합니다. 레지스터에서 Shift 범위를 지정하기 위해서는 Shift를 시작할 BIT의 위치가 필요합니다. 따라서, 시작 BIT 위치를 지정하기 위해서 레지스터를 WORD형이 아닌 BIT 형으로 지정합니다
Operand 2	Shift 이동할 폭(Width)을 지정합니다. OP1에서 지정한 시작 BIT와 OP2에서 지정한 Width를 사용하면 Shift 이동의 범위가 결정됩니다
Operand 3	Shift 이동할 크기를 지정합니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100.2]부터 4Bit의 데이터를 오른쪽으로 2Bit만큼 Shift회전합니다.

설정 범위를 넘어 이동되는 데이터는 제거됩니다.

[SFTR2] 명령 수행결과 [RW100]에는 206이 저장됩니다.



5.9 Math Function 명령

5.9.1 SQRT (제곱근, Square Root)

■ [기능]

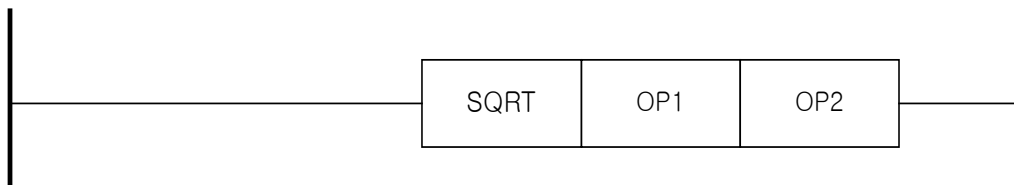
제곱근을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	제곱근 계산할 값을 지정
OP2	-	○	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



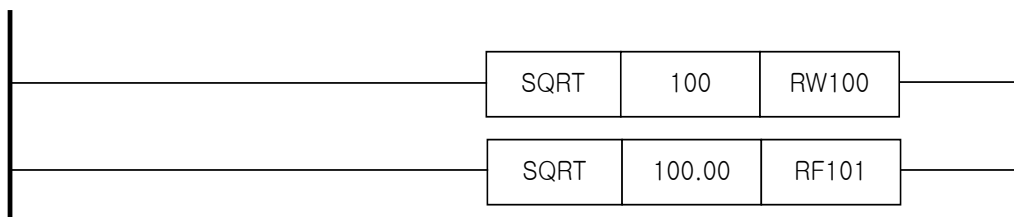
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	계산할 값을 지정
Operand 2	계산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

데이터의 제곱근을 계산하여 출력합니다.

[SQRT] 명령 수행결과 [RW100]에는 10, [RF101]에는 10.00이 저장됩니다.



5.9.2 DEGTORAD (DEG⇒RAD 변환)

■ [기능]

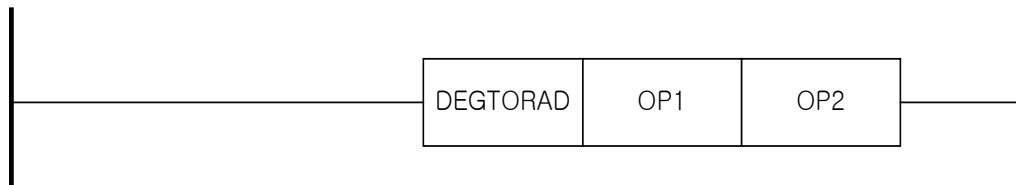
Degree 단위의 각도 값을 Radian 단위의 각도 값으로 환산해 주는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	Radian 단위로 변환할 레지스터
OP2	-	○	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



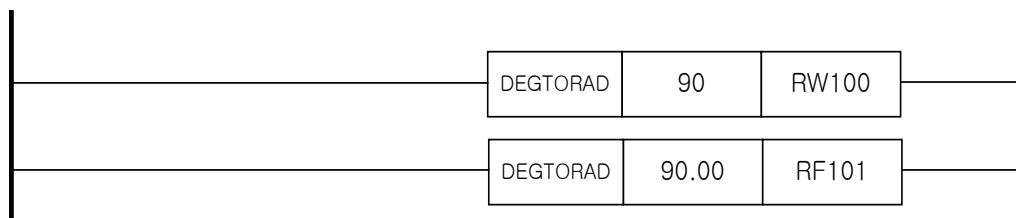
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	값을 변환할 레지스터를 지정
Operand 2	계산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

Degree 단위의 각도 값을 Radian 단위의 각도 값으로 환산해 출력합니다.

[DEGTORAD] 명령 수행결과 [RW100]에는 1, [RF101]에는 1.58이 저장됩니다.



5.9.3 RADTODEG (RAD⇒DEG 변환)

■ [기능]

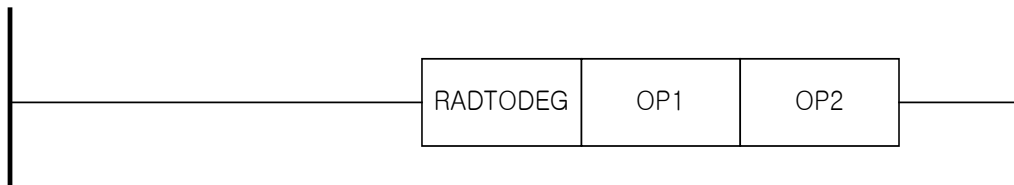
Radian 단위의 각도 값을 Degree 단위의 각도 값으로 환산해 주는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	Degree 단위로 변환할 레지스터
OP2	-	○	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



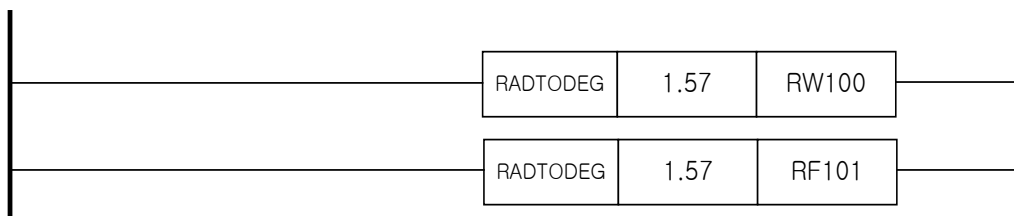
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 변환할 레지스터를 지정
Operand 2	계산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

Radian 단위의 각도 값을 Degree 단위의 각도 값으로 환산해 출력합니다.

[RADTODEG] 명령 수행결과 [RW100]에는 90, [RF101]에는 90.00이 저장됩니다.



5.9.4 SIN (Sine 함수)

■ [기능]

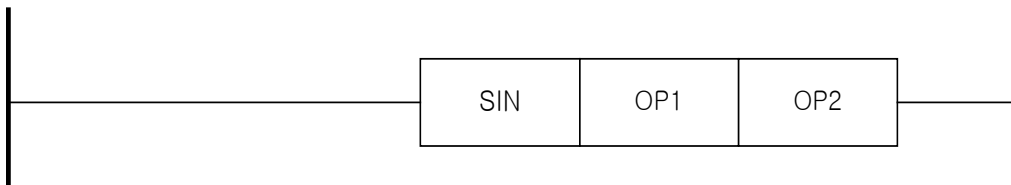
SIN 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	SIN 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



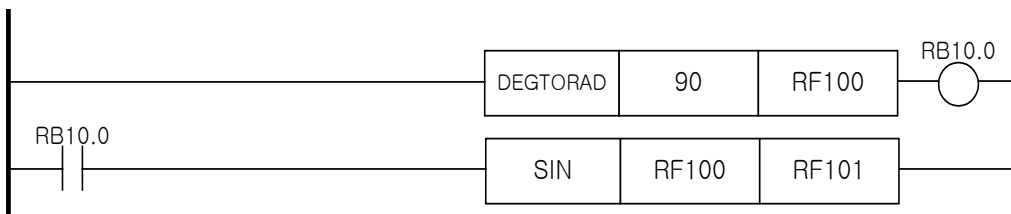
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 SIN 값을 출력합니다.

[SIN] 명령 수행결과 [RF101]에는 1.00이 저장됩니다.



5.9.5 COS (Cosine 함수)

■ [기능]

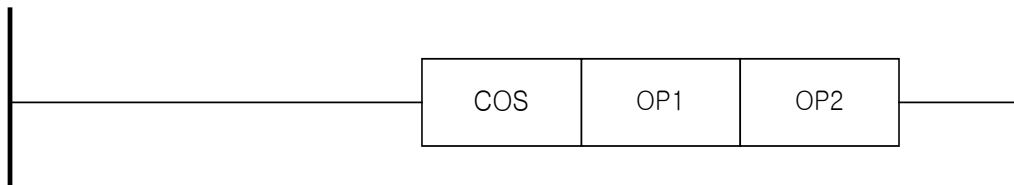
COSINE 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	COS 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



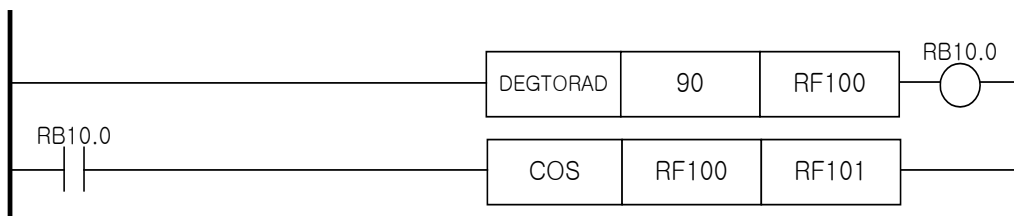
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 COS 값을 출력합니다.

[COS] 명령 수행결과 [RF101]에는 1.000이 저장됩니다.



5.9.6 TAN (Tangent 함수)

■ [기능]

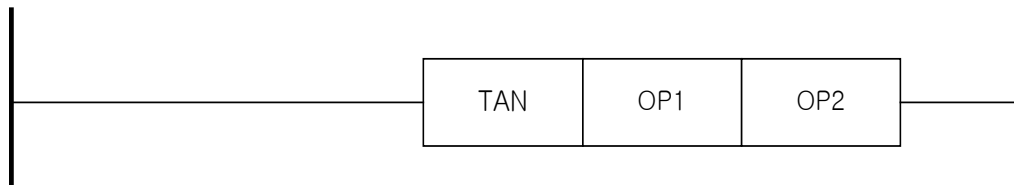
TANGENT 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	TAN 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



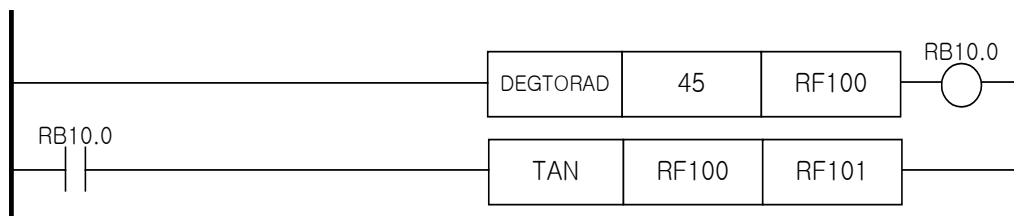
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 TAN 값을 출력합니다.

[TAN] 명령 수행결과 [RF101]에는 1.00이 저장됩니다.



5.9.7 ASIN (Arcsine 함수)

■ [기능]

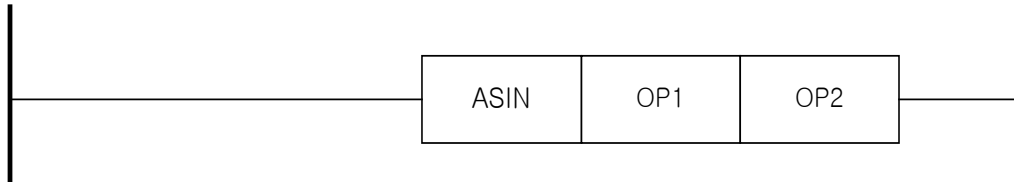
ARCSIN 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	ASIN 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



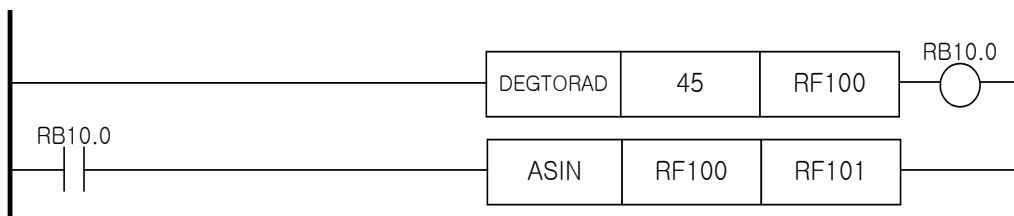
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 ASIN 값을 출력합니다.

[ASIN] 명령 수행결과 [RF101]에는 0.90이 저장됩니다.



5.9.8 ACOS (Arccosine 함수)

■ [기능]

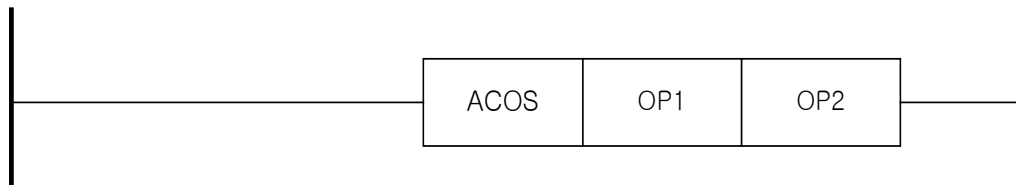
ARCCOSINE 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	ACOS 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



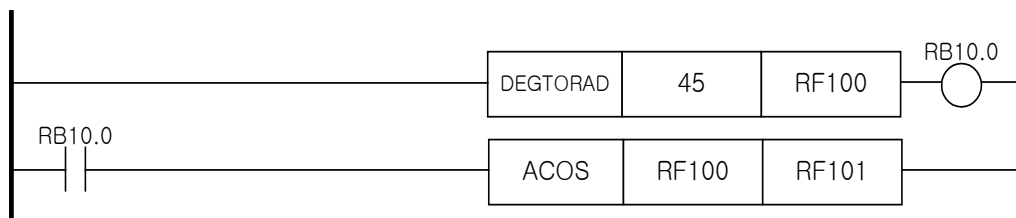
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 ACOS 값을 출력합니다.

[ACOS] 명령 수행결과 [RF101]에는 0.67이 저장됩니다.



5.9.9 ATAN (Arctangent 함수)

■ [기능]

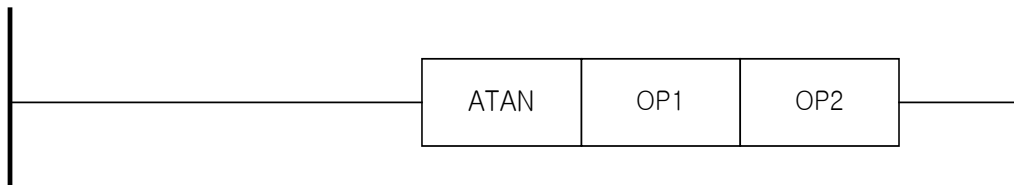
ARCTANGENT 함수 값을 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	ATAN 계산할 레지스터
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



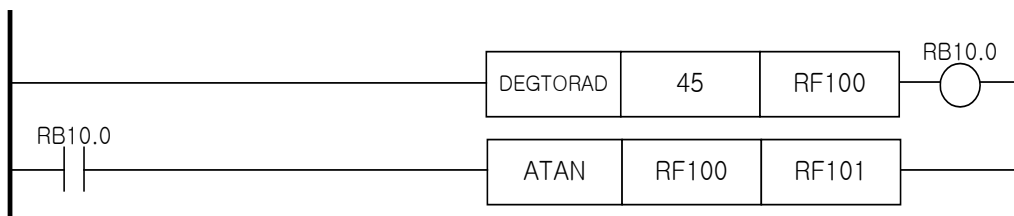
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	계산할 값(Radian 단위)을 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 Radian 데이터의 ATAN 값을 출력합니다.

[ATAN] 명령 수행결과 [RF101]에는 0.670이 저장됩니다.



5.9.10 EXP (밑이 e인 지수 함수)

■ [기능]

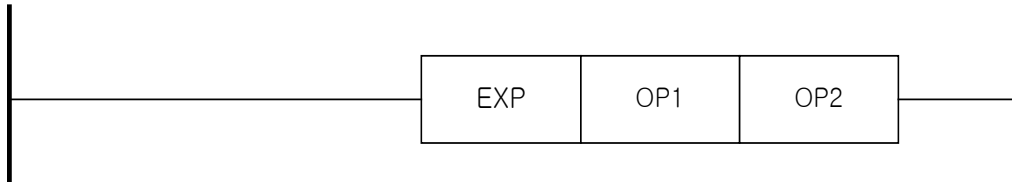
밑이 e(자연로그)인 지수를 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	지수
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



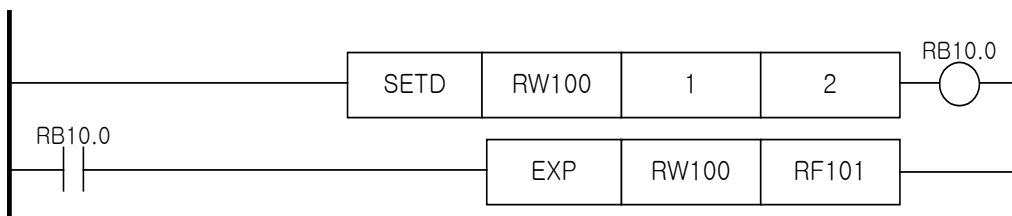
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	지수를 지정
Operand 2	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

밑이 e(자연로그)인 지수 2의 결과 값을 출력합니다.

[EXP] 명령 수행결과 [RF101]에는 7.39이 저장됩니다.



5.9.11 POW (지수 함수)

■ [기능]

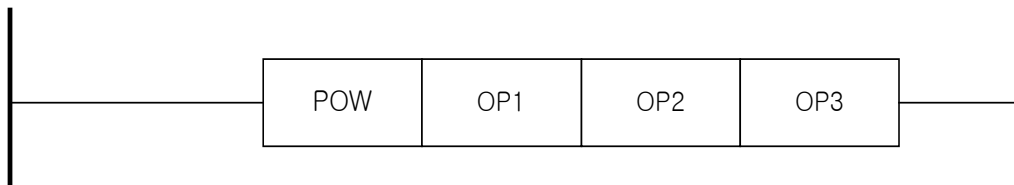
입력한 밑의 지수를 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	밑
OP2	-	○	○	○	○	지수
OP3	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



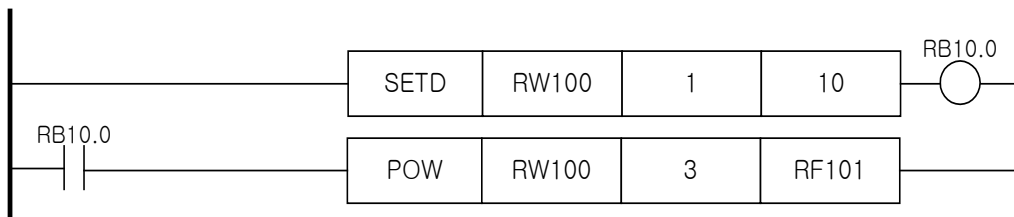
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	밑을 지정
Operand 2	지수를 지정
Operand 3	계산 결과를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

밑이 10이고 지수 3의 결과 값을 출력합니다.

[POW] 명령 수행결과 [RF101]에는 1000.000이 저장됩니다.



5.9.12 LN (자연로그)

■ [기능]

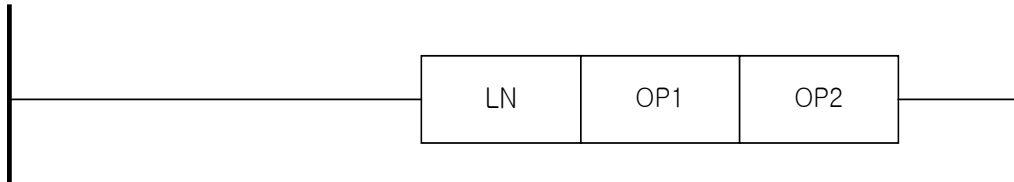
자연로그(LN)를 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	자연로그 계산할 값
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



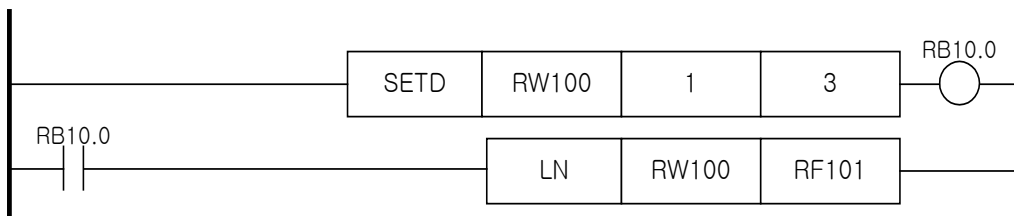
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	계산할 값을 지정
Operand 2	계산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

3의 자연로그 값을 출력합니다.

[LN] 명령 수행결과 [RF101]에는 1.10이 저장됩니다.



5.9.13 LOG (상용로그)

■ [기능]

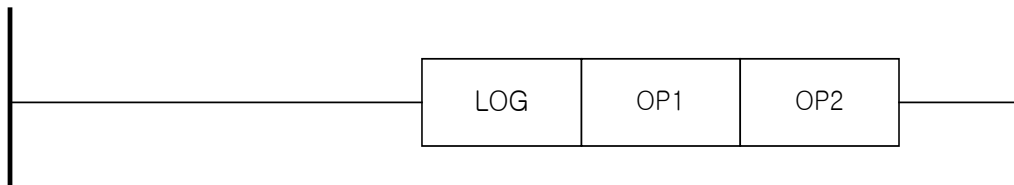
상용로그(Log)를 계산하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	○	○	상용로그 계산할 값
OP2	-	-	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



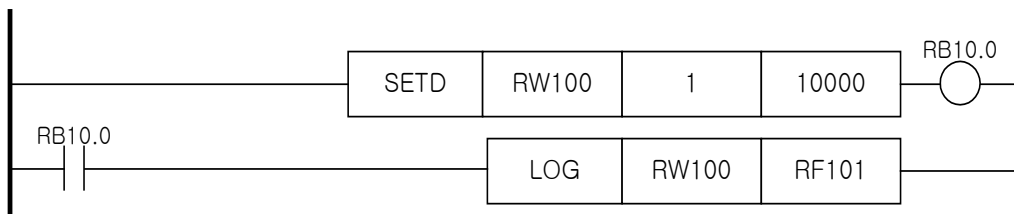
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	계산할 값을 지정
Operand 2	계산 결과를 저장할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

10000의 상용로그 값을 출력합니다.

[LOG] 명령 수행결과 [RF101]에는 4.00이 저장됩니다.



5.9.14 ROUND (반올림)

■ [기능]

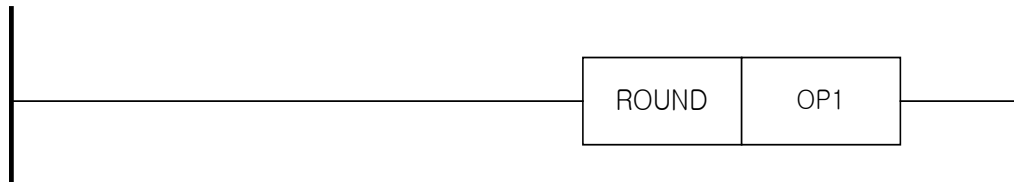
지정한 레지스터의 값을 반올림하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	-	○	-	반올림할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



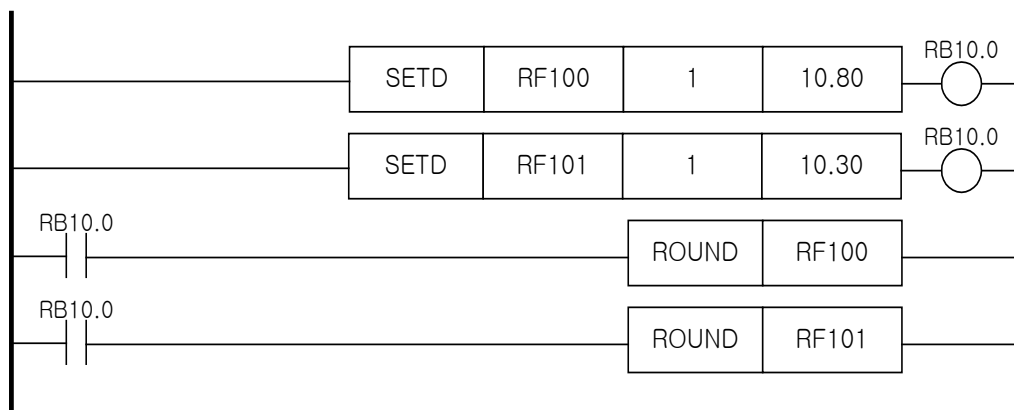
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	반올림할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

레지스터의 값을 반올림하여 출력합니다.

[ROUND] 명령 수행결과 [RF100]에는 11.00, [RF101]에는 10.00이 저장됩니다.



5.9.15 FUP (올림)

■ [기능]

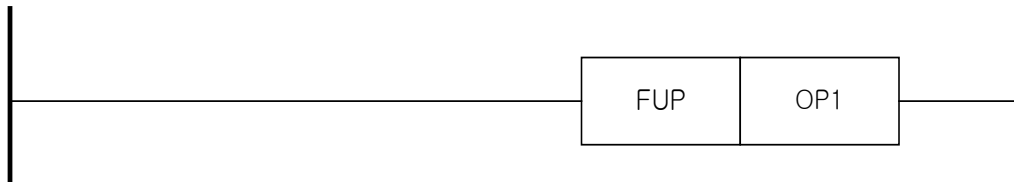
지정한 레지스터의 값을 올림하는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	-	○	-	올림할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



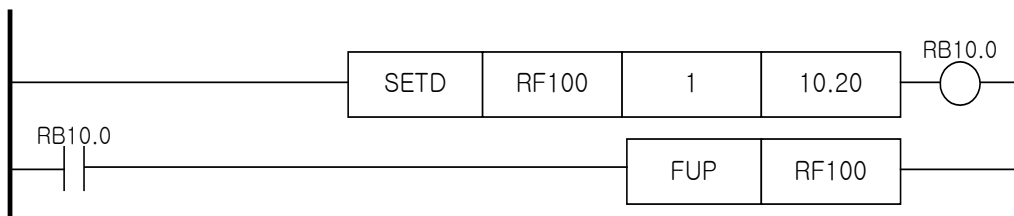
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	올림할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 데이터를 올림하여 출력합니다.

[FUP] 명령 수행결과 [RF100]에 11.00이 저장됩니다.



5.9.16 FIX (버림)

■ [기능]

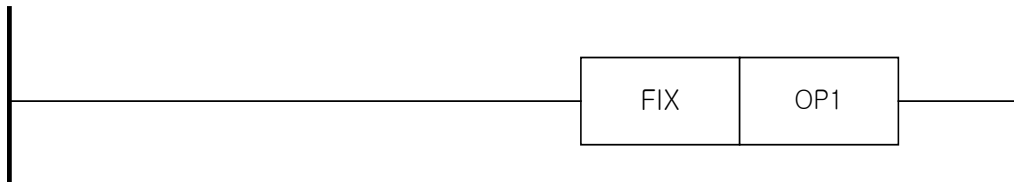
지정한 레지스터의 값을 내림하는 명령입니다. 즉, 소수점 이하의 값을 모두 버리는 명령입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	-	○	-	버림할 레지스터

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



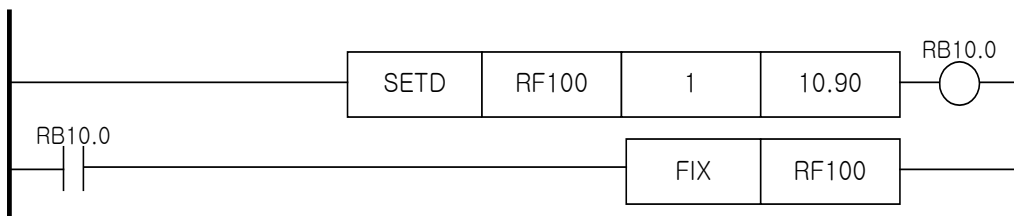
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	버림할 레지스터를 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RF100]의 데이터를 내림하여 출력합니다.

[FIX] 명령 수행결과 [RF100]에 10.00이 저장됩니다.



5.10 테이블 데이터 처리 명령

5.10.1 사용자 정의 스택 (User Define Stack)

■ [설명]

1. 사용자 정의 스택은 총 10개를 만들 수 있으며, 각 스택의 크기는 자유롭게 설정할 수 있습니다.
2. 사용자는 PUSH 명령을 사용하여 스택에 원하는 데이터를 입력할 수 있습니다.
3. 스택에 저장된 데이터 중에서 가장 먼저 저장된 데이터부터 읽고 싶으면 FIFO 명령을 사용하고, 가장 나중에 저장한 데이터부터 읽고 싶으면 LIFO 명령을 사용합니다.

① STSET (Stack 설정)

■ [기능]

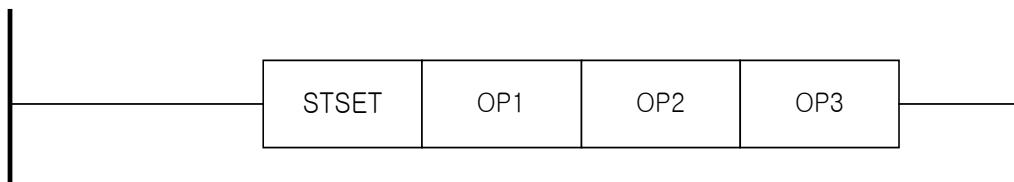
1. 사용자 정의 스택을 만듭니다.
- STSET 명령으로 만들어지지 않은 스택은 사용할 수 없습니다.
2. 스택영역으로 지정된 레지스터들을 0으로 초기화 합니다.
 3. STSET 명령은 START PLC에서만 사용하십시오.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	스택 번호
OP2	-	○	-	○	-	스택 영역의 시작 레지스터
OP3	-	-	○	-	-	스택 영역 크기

* 명령 실행조건, 결과 출력 삼입 가능

■ [Ladder]



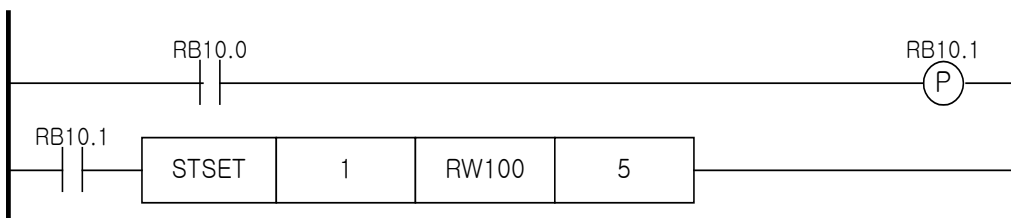
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	새로 만들 스택의 번호를 입력합니다. 입력 가능한 값은 1부터 10까지이며, 총 10개의 스택을 정의 할 수 있습니다 (TSTSET으로 정의한 ‘사용자 정의 테이블 스택’의 개수와는 상관 없습니다)
Operand 2	스택 영역으로 사용할 첫번째 레지스터를 지정 동시에, 스택에서 사용할 데이터의 TYPE을 지정합니다. 예를 들어, RW1000으로 지정을 하면 이 스택은 WORD형 데이터를 저장하고 읽을 수 있는 스택이 됩니다
Operand 3	스택 영역의 크기를 지정합니다. (입력 가능한 값은 1부터 1000까지 입니다) OP2에서 지정한 레지스터부터 OP3에서 지정한 크기 만큼의 레지스터들이 스택이 사용하는 영역이 됩니다
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 5개의 레지스터를 1번 스택으로 설정합니다.

따라서, [RW100]~[RW104]의 레지스터는 다른 용도로 사용을 하면 절대로 안됩니다.



② STPUSH (Push Data)

■ [기능]

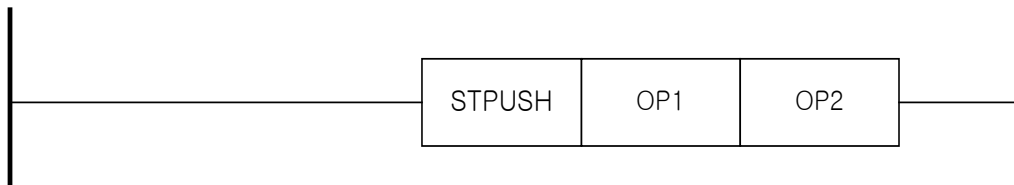
지정한 스택에 데이터를 격납(Push) 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	데이터 저장할 스택 번호
OP2	-	○	○	○	○	저장할 데이터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



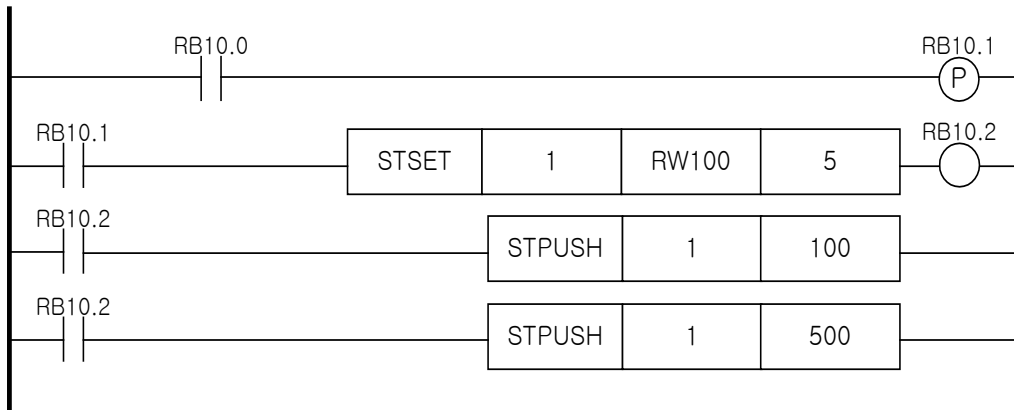
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	데이터를 격납 할 스택을 지정
Operand 2	격납 할 데이터를 지정 격납 할 데이터는 스택의 데이터 Type과 동일해야 함 스택의 데이터 Type은 선언할 때 Operand2에 의해서 결정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

설정된 1번 스택영역에 100, 500 데이터를 저장합니다.

[STPUSH] 명령 수행결과 [RW100]에는 100, [RW101]에는 500이 저장됩니다.



③ STFIFO (First Input First Out)

■ [기능]

지정한 스택에 격납되어 있는 데이터들 중에서 가장 먼저 격납(Push)된 데이터를 가져오는(Pop) 명령입니다.

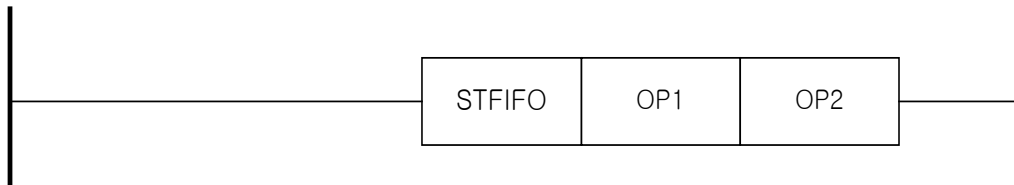
FIFO 명령이 수행되어서 데이터를 가져오면(Pop) 그 데이터는 스택에서 없어집니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	읽을 스택 번호
OP2	-	○	-	○	-	읽어올 값을 저장할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

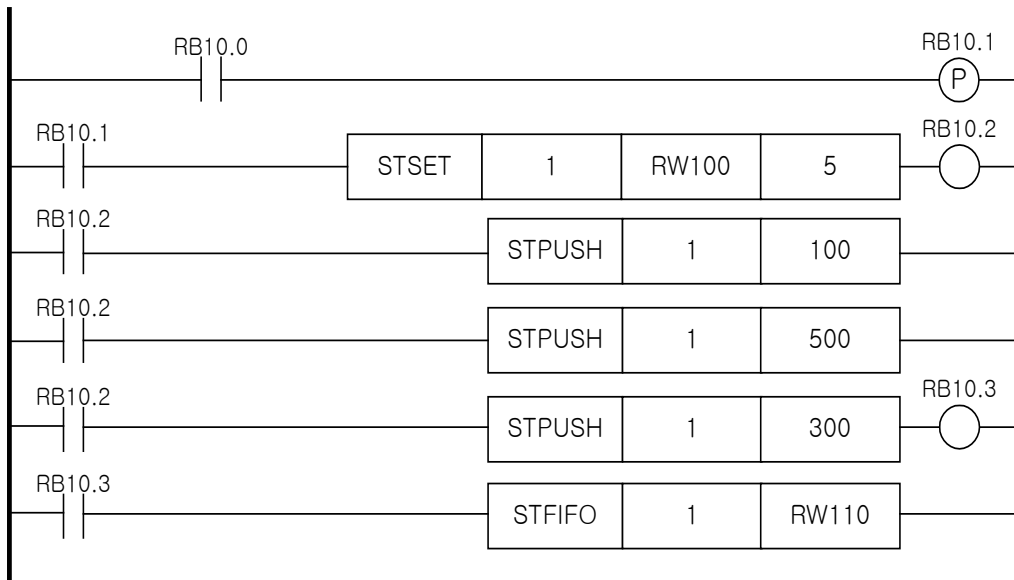


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	데이터를 가져올 스택을 지정
Operand 2	스택에서 가져온 값을 저장할 레지스터를 지정 레지스터의 데이터는 스택의 데이터 Type과 동일해야 함 스택의 데이터 Type은 스택 선언할 때 Operand2에 의해서 결정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

1번 스택에 저장된 데이터 중 가장 먼저 저장된 데이터를 [RW110]에 출력합니다. [STFIFO] 명령 수행결과 [RW110]에는 100이 저장됩니다.



④ STLIFO (Last Input First Out)

■ [기능]

지정한 스택에 격납되어 있는 데이터들 중에서 가장 나중에(최근에) 격납(Push)된 데이터를 가져오는(Pop) 명령입니다.

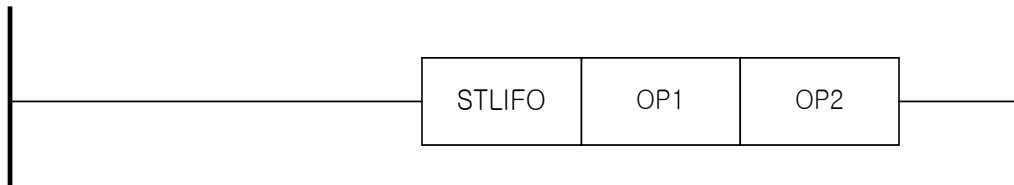
LIFO 명령이 수행되어서 데이터를 가져오면(Pop) 그 데이터는 스택에서 없어집니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	-	○	-	-	읽을 스택 번호
OP2	-	○	-	○	-	읽어온 값을 저장할 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

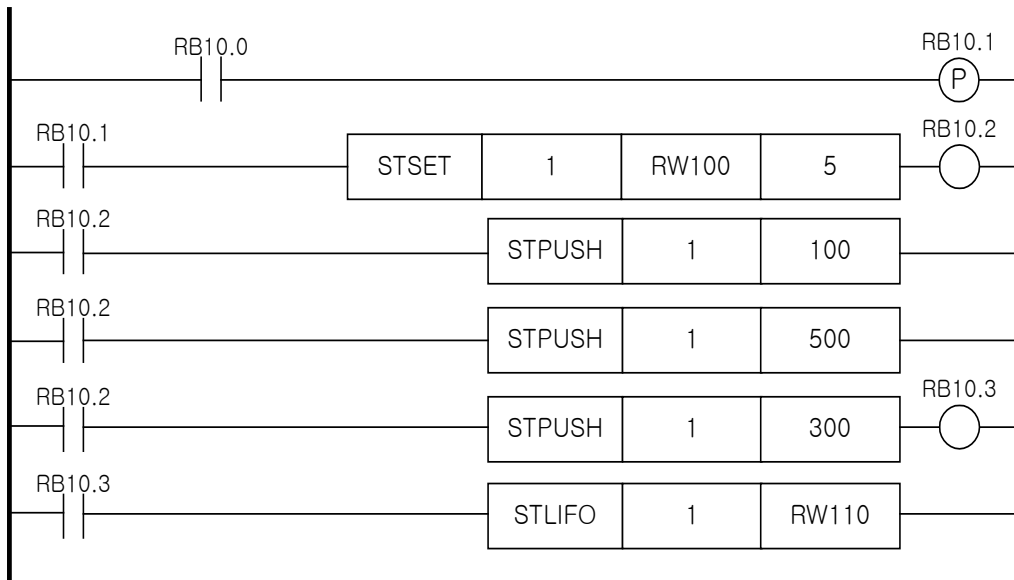


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	데이터를 가져올 스택을 지정
Operand 2	스택에서 가져온 값을 저장할 레지스터를 지정합니다. 레지스터의 데이터는 스택의 데이터 Type과 동일해야 함 스택의 데이터 Type은 선언할 때 Operand2에 의해서 결정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

1번 스택에 저장된 데이터 중 가장 나중에 저장된 데이터를 [RW110]에 출력합니다. [STLIFO] 명령 수행결과 [RW110]에는 300이 저장됩니다.



5.10.2 테이블 데이터

■ [설명]

1. PLC 사용자는 테이블을 선언하여 데이터를 편리하게 관리할 수 있습니다.
 2. 테이블의 크기(1차원)는 사용자가 자유롭게 설정할 수 있습니다.
 3. 사용자는 테이블의 원하는 위치에 데이터를 입력할 수 있고, 원하는 위치에서 데이터를 가져올 수 있습니다.
 4. 테이블을 사용하면 복잡한 데이터들을 보다 효율적으로 관리할 수 있습니다.
 5. 테이블에 사용할 수 있는 데이터의 타입은 WORD형과 FLOAT형 두 가지 입니다.
- 테이블을 선언할 때에 테이블에서 사용할 데이터 타입이 결정되며, 같은 테이블에서 두 가지 데이터 타입을 동시에 사용할 수는 없습니다.

① TBRD (테이블 값 읽기)

■ [기능]

테이블의 원하는 위치에서 값을 읽어 옵니다.

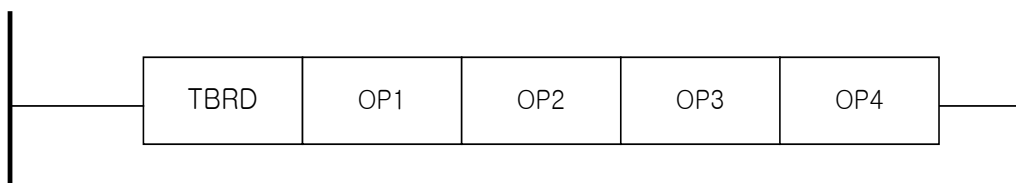
즉, OP1이 지정한 테이블에서 OP3가 지정한 위치의 값을 읽은 후에 그 값을 OP4에 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	○	-	-	읽을 테이블 데이터 번호
OP4	-	○	-	○	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



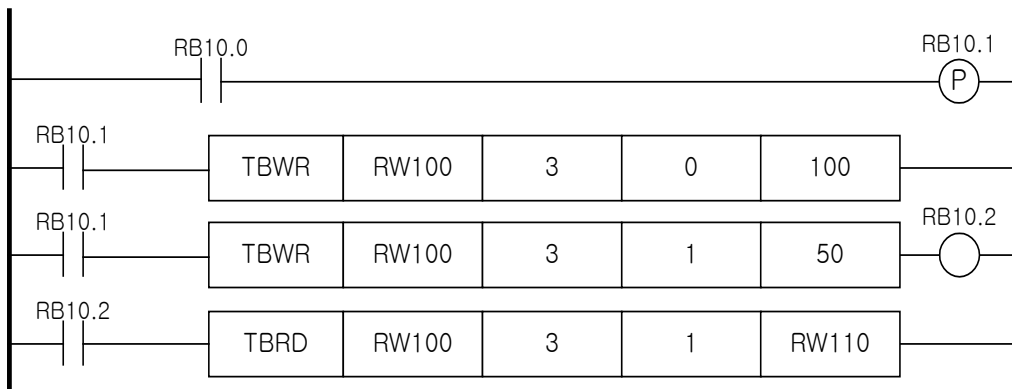
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터를 읽을 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	읽을 테이블 데이터의 번호를 지정 ($0 \leq OP3 < OP2$) OP3은 0보다 크거나 같고, OP2의 크기보다 작아야 합니다.
Operand 4	읽은 테이블 데이터를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 3개의 레지스터를 테이블로 설정하고, 저장된 데이터 중 2번째 데이터를 [RW110]에 출력합니다.

[TBRD] 명령 수행결과 [RW110]에는 50이 저장됩니다.



② TBWR (테이블에 값 기록)

■ [기능]

테이블의 원하는 위치에 값을 기록 합니다.

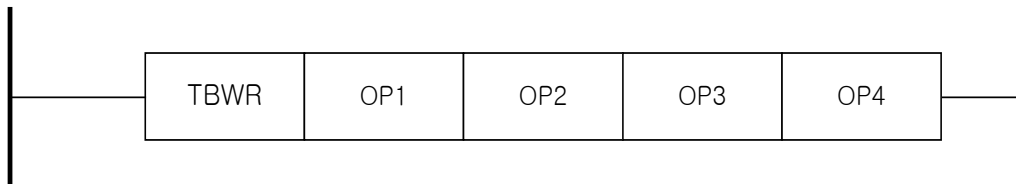
즉, OP1이 지정한 테이블에서 OP3이 지정한 위치에 OP4의 값을 저장합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 데이터 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	○	-	-	기록할 테이블 데이터 번호
OP4	-	○	○	○	○	기록할 데이터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



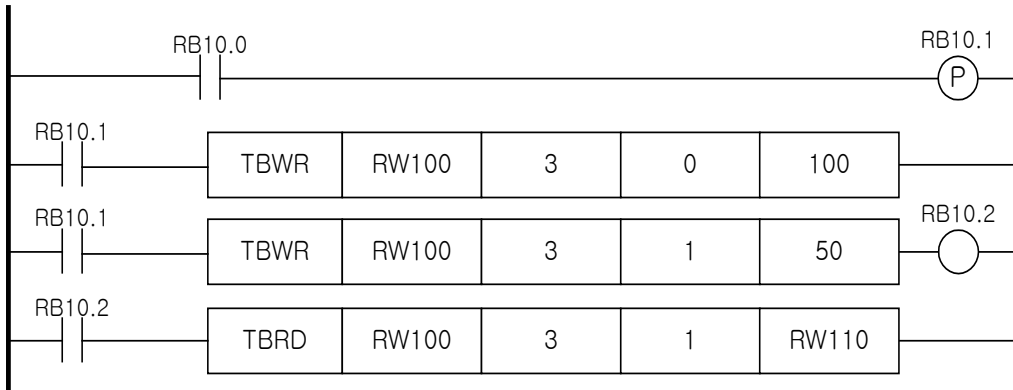
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터를 기록할 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	기록할 테이블 데이터의 번호를 지정 (0≤OP3<OP2) OP3은 0보다 크거나 같고, OP2의 크기보다 작아야 합니다.
Operand 4	기록할 테이블 데이터나 데이터가 저장되어 있는 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 3개의 레지스터를 테이블로 설정하고, 테이블의 1번째 레지스터에 100을 입력하고 2번째 레지스터에 50을 입력합니다.

[TBWR] 명령 수행결과 [RW100]에는 100, [RW101]에는 50이 저장됩니다.



③ TBSRCH (테이블에서 데이터 검색)

■ [기능]

사용자가 지정한 데이터가 테이블 내에 있는지 검색합니다.

테이블 내에 데이터가 있으면 그 위치를 알려줍니다.

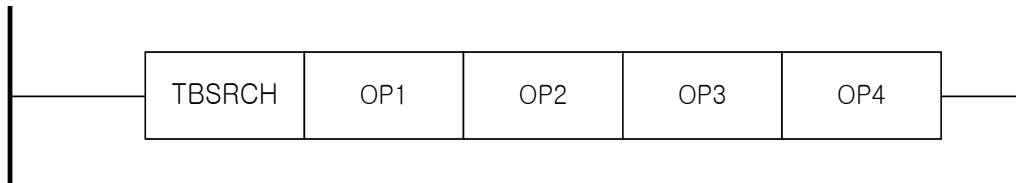
테이블 내에 찾는 데이터가 여러 개가 있을 경우에는 가장 앞에 있는 데이터의 위치를 알려줍니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 데이터 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	○	○	○	검색할 데이터
OP4	-	○	-	-	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



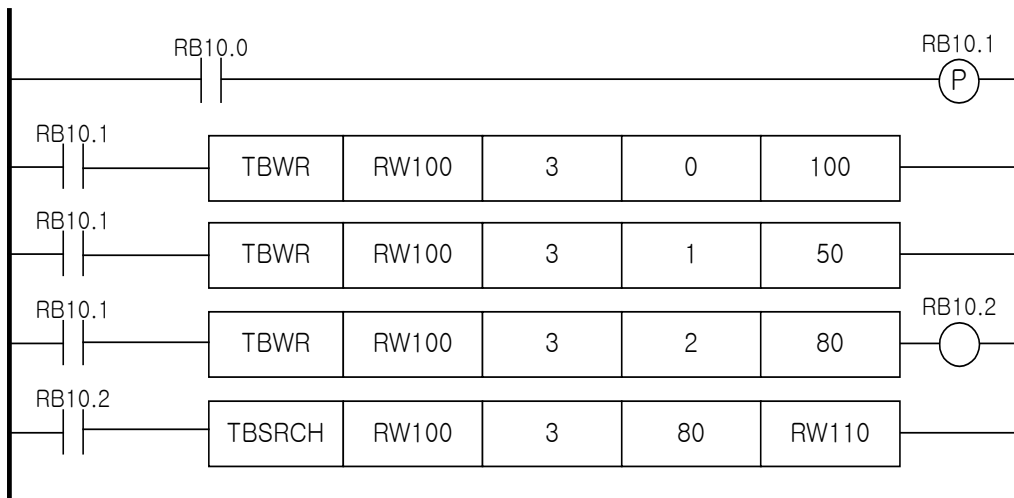
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터를 검색할 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	검색할 테이블 데이터나 데이터가 저장되어 있는 레지스터 지정
Operand 4	검색 결과가 저장될 레지스터 지정 결과는 테이블의 번호가 기록됩니다. (0~n-1) -1 - 검색 데이터 없음
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 테이블로 설정된 3개의 레지스터에서 데이터가 80인 레지스터를 검색하여 위치 데이터를 [RW110]에 출력합니다.

[TBSRCH] 명령 수행결과 [RW110]에 2가 저장됩니다.



④ TBMAX (테이블 최대값 검색)

■ [기능]

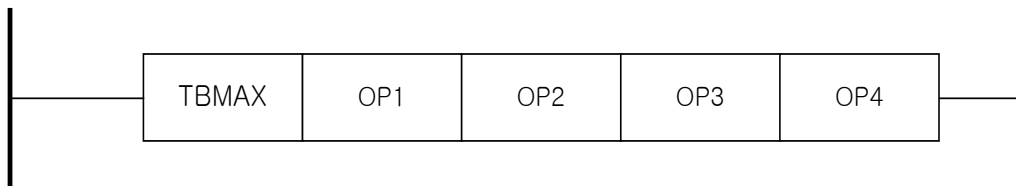
테이블에서 최대 값을 검색하여, 최대 값과 그 위치를 알려 줍니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 데이터 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	-	-	-	최대값 저장 레지스터
OP4	-	○	-	-	-	최대값 위치 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



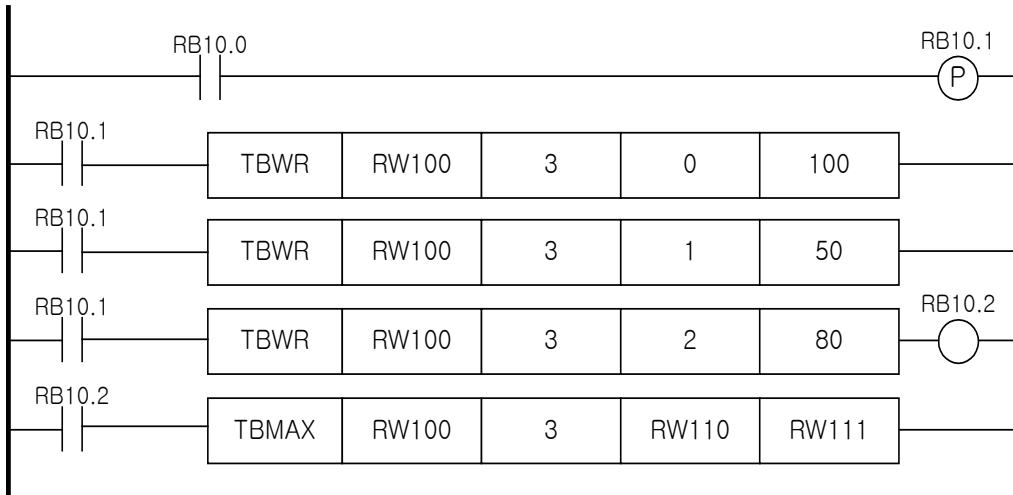
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터의 최대값을 검색할 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	검색한 최대값을 기록할 레지스터 지정
Operand 4	최대값의 위치를 기록할 레지스터 지정 테이블의 번호가 기록됩니다. (0~n-1)
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 테이블로 설정된 3개의 레지스터에서 데이터가 가장 큰 레지스터를 검색하여 최대 값과 위치 데이터를 출력합니다.

[TBMAX] 명령 수행결과 [RW110]에는 100, [RW111]에는 0이 저장됩니다.



⑤ TBMIN (테이블 최소값 검색)

■ [기능]

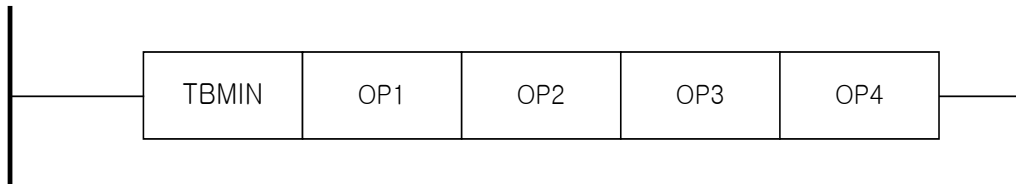
테이블에서 최소 값을 검색하여, 최소 값과 그 위치를 알려 줍니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 데이터 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	-	-	-	최소값 저장 레지스터
OP4	-	○	-	-	-	최소값 위치 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



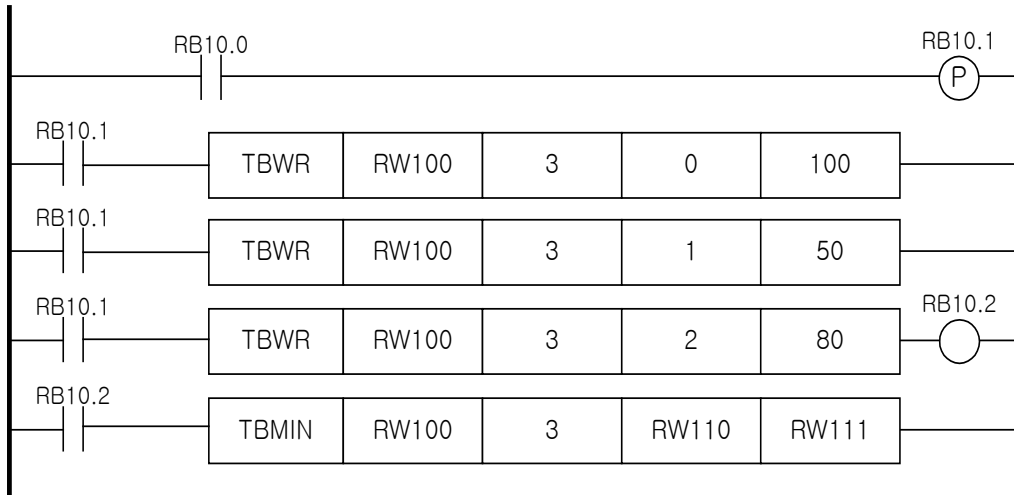
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터의 최소값을 검색할 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	검색한 최소값을 기록할 레지스터 지정
Operand 4	최소값의 위치를 기록할 레지스터 지정 테이블의 번호가 기록됩니다. (0~n-1)
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 테이블로 설정된 3개의 레지스터에서 데이터가 가장 작은 레지스터를 검색하여 최소값과 위치 데이터를 출력합니다.

[TBMIN] 명령 수행결과 [RW110]에는 50, [RW111]에는 1이 저장됩니다.



⑥ TBSUM (테이블 합계)

■ [기능]

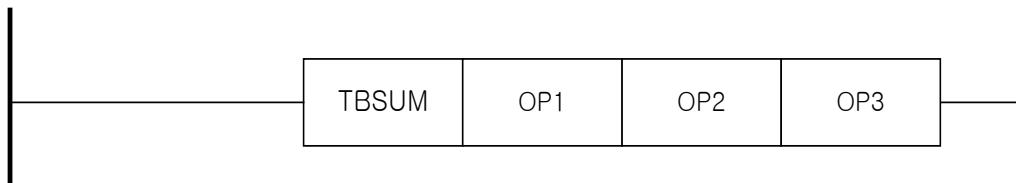
지정한 테이블 내에 있는 모든 데이터들의 합을 계산합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	테이블 데이터 시작 레지스터
OP2	-	○	○	-	-	테이블 폭(Width)
OP3	-	○	-	-	-	테이블 총합 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	테이블 데이터의 총합을 계산할 시작 레지스터 지정
Operand 2	테이블의 폭(Width)을 지정
Operand 3	테이블 데이터의 총합을 기록할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]부터 테이블로 설정된 3개의 레지스터에서 데이터의 총합을 계산하여 [RW110]에 출력합니다.

[TBSUM] 명령 수행결과 [RW110]에 230이 저장됩니다.



5.11 데이터 제어 명령

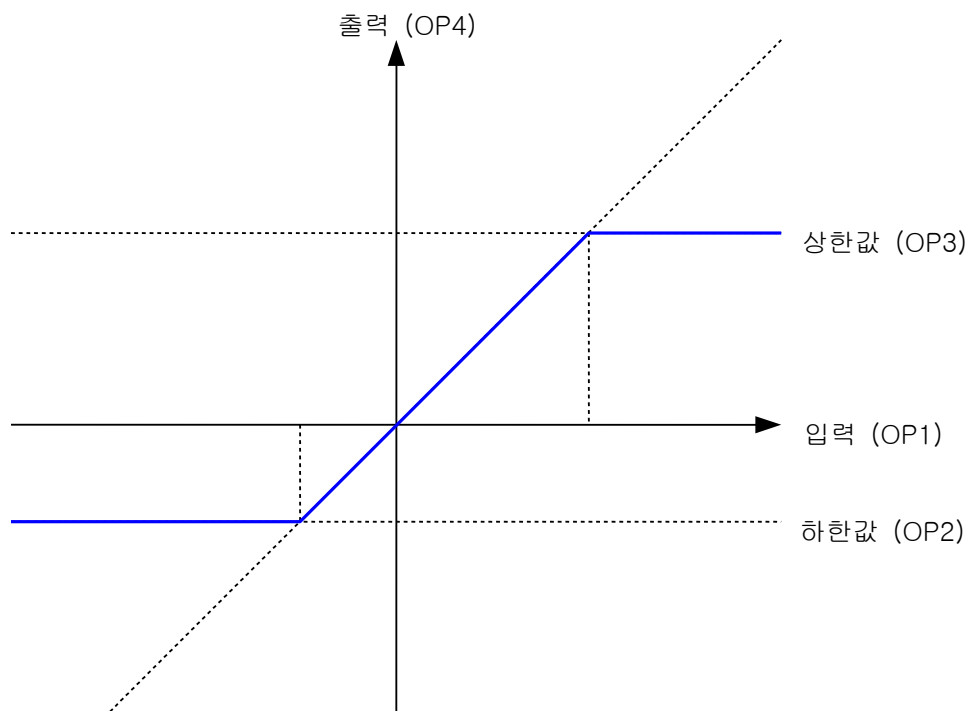
5.11.1 CTRLMT (상하한 리미트 제어)

■ [기능]

입력 데이터가 상한 값보다 크면, 출력은 상한 값이 됩니다.

입력 데이터가 하한 값보다 작으면, 출력은 하한 값이 됩니다.

입력 데이터가 지정한 범위 내에 있으면, 출력은 입력 값과 동일합니다.

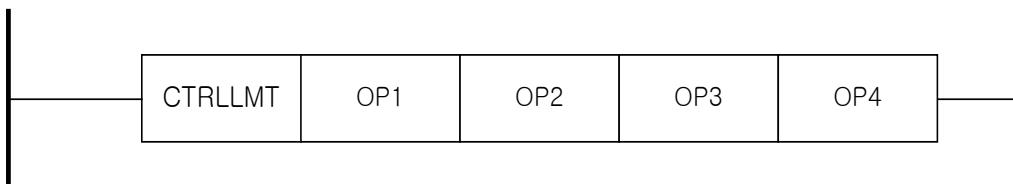


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	입력 데이터
OP2	-	○	○	○	○	Limit 제어 하한 값
OP3	-	○	○	○	○	Limit 제어 상한 값
OP4	-	○	-	○	-	출력 데이터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



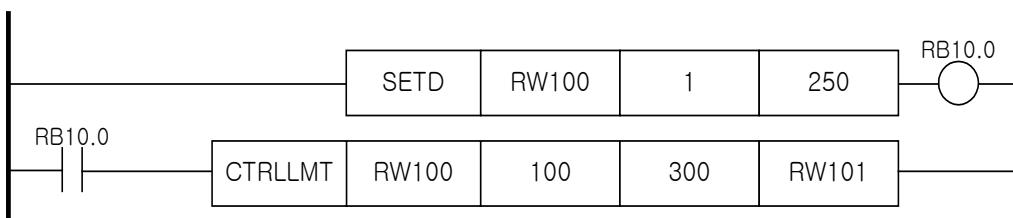
■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	입력 데이터를 지정
Operand 2	Limit 제어 하한 값을 입력합니다.
Operand 3	Limit 제어 상한 값을 입력합니다.
Operand 4	출력 데이터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

[RW100]의 [250]에 대한 상·하한 Limit 제어 값을 출력합니다.

[CTRLMT] 명령 수행결과 입력 데이터가 100~300 사이의 값이므로 [RW101]에 출력 데이터 250이 저장됩니다.



5.11.2 DELAY (일차 지연)

■ [기능]

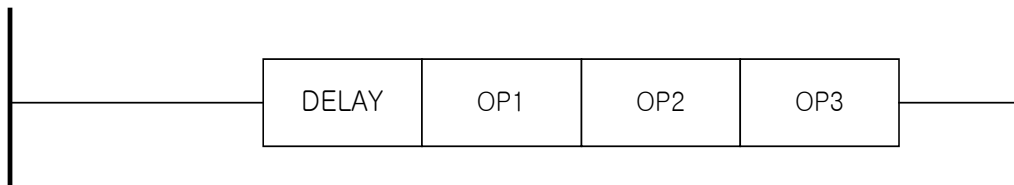
일차 지연 제어를 합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	Delay 시간
OP2	-	○	-	○	-	입력 데이터
OP3	-	○	-	○	-	출력 데이터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

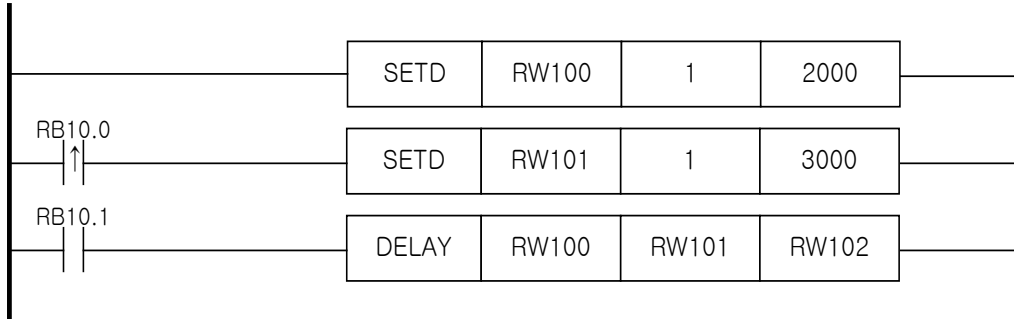


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	DELAY 시간 입력
Operand 2	입력 데이터를 지정
Operand 3	출력 데이터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

입력된 DELAY 시간 [RW100]에 대한 [RW101] 데이터 값의 DELAY 값을 [RW102]에 출력합니다.



5.11.3 LINEACC (직선 가속기)

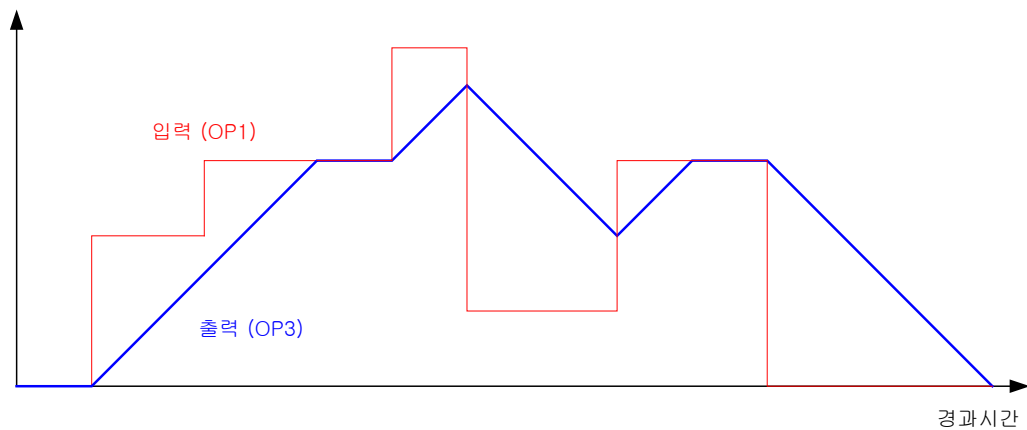
■ [기능]

입력 값이 0이면 출력 값도 0입니다.

입력 값이 변하면 출력 값은 서서히 변하여 입력 값과 같아집니다.

이때, 출력 값의 변화는 1차 함수(직선) 형태입니다.

출력 값의 변화율(직선 기울기)은 Operand 2에서 사용자가 설정 합니다.

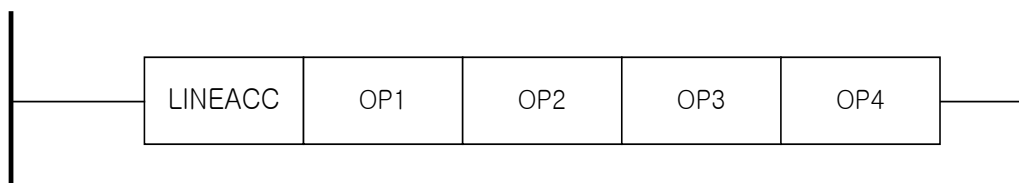


■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	-	○	-	입력 데이터
OP2	-	○	○	○	○	경과시간 동안의 이동량
OP3	-	○	○	○	○	경과 시간
OP4	-	○	-	○	-	출력 데이터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



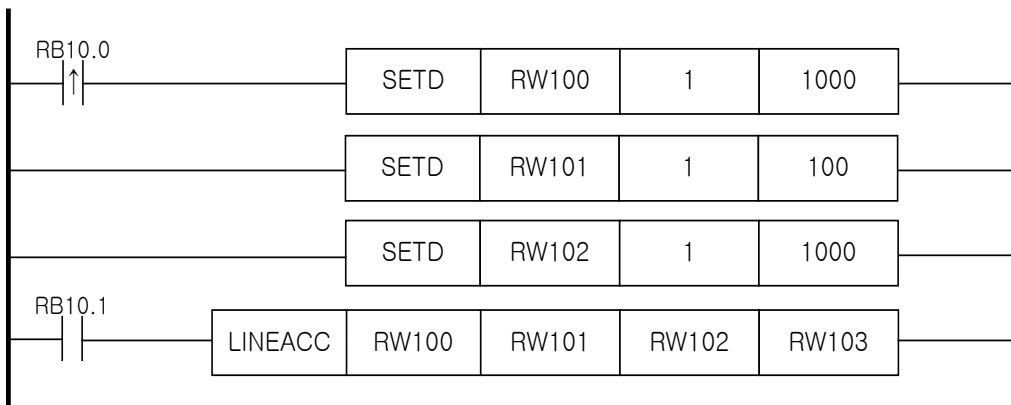
■ [설명]

실행 조건	명령 실행 조건을 삼입 가능
Operand 1	입력 데이터를 지정
Operand 2	경과시간 동안 이동한 데이터 입력 (Operand3 값과 직선의 기울기를 변화율(기울기) = 이동 데이터 / 경과시간 [1/msec]
Operand 3	경과 시간 [msec]
Operand 4	출력 데이터
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [설명]

[RW100] 입력 데이터의 직선 기울기에 대한 변화 값을 [RW103]에 출력합니다.

직선 기울기 = [RW101] / [RW102] = 100 / 1000 = 0.1



5.12 통신 명령

5.12.1 RDRMT (원격지 레지스터 읽기)

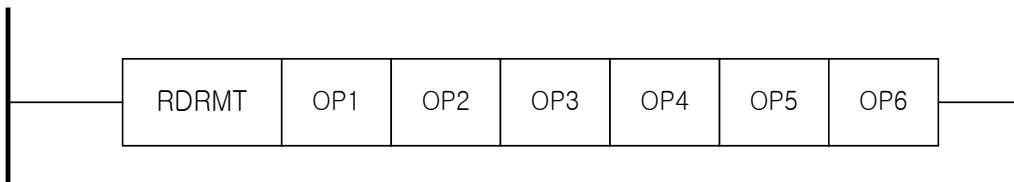
■ [기능]

통신연결 되어 있는 원격지 GX PLC 모듈의 레지스터의 데이터를 지정한 레지스터에 저장합니다.
GX간 통신시 사용합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	Device ID
OP2	-	○	○	-	-	Port 번호
OP3	-	○	○	-	-	Node 번호
OP4	-	○	-	-	-	원격지의 원본 레지스터
OP5	-	○	-	-	-	로컬의 목표 레지스터
OP6	-	○	○	-	-	레지스터 폭(Width)

■ [Ladder]

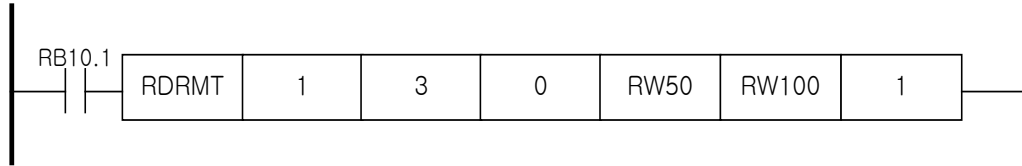


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	Device ID를 지정
Operand 2	Port 번호를 지정
Operand 3	Node 번호를 지정
Operand 4	Source 레지스터를 지정 (원격지의 원본 레지스터)
Operand 5	Target 레지스터를 지정 (저장될 로컬 레지스터)
Operand 6	레지스터의 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

Device ID 1을 통해 원격지 GX의 [RW50] 데이터를 읽어 로컬 레지스터 [RW100]에 저장합니다.



5.12.2 WRRMT (원격지 레지스터 쓰기)

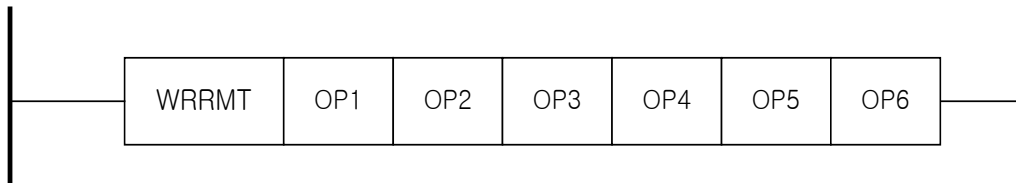
■ [기능]

통신연결 되어 있는 원격지 GX PLC 모듈의 레지스터에 지정한 레지스터의 데이터 값을 저장합니다. GX간 통신시 사용합니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	Device ID
OP2	-	○	○	-	-	Port 번호
OP3	-	○	○	-	-	Node 번호
OP4	-	○	-	-	-	로컬의 원본 레지스터
OP5	-	○	-	-	-	원격지의 목표 레지스터
OP6	-	○	○	-	-	레지스터 폭(Width)

■ [Ladder]

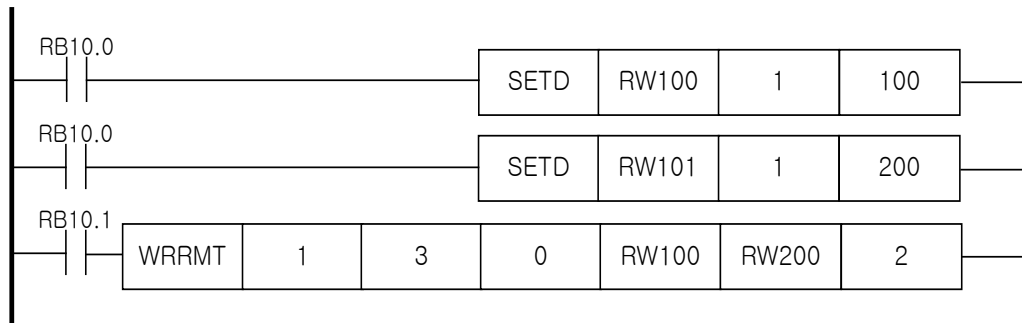


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	Device ID를 지정
Operand 2	Port 번호를 지정
Operand 3	Node 번호를 지정
Operand 4	Source 레지스터를 지정 (로컬의 원본 레지스터)
Operand 5	Target 레지스터를 지정 (저장될 원격지 레지스터)
Operand 6	레지스터의 폭(Width)을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

로컬 레지스터의 [RW100]부터 2개의 레지스터 데이터를 Device ID 1을 통해 원격지 레지스터 [RW200]에 저장합니다.



5.12.3 SETREG (레지스터 설정)

■ [기능]

지정된 레지스터를 지정한 값으로 설정 합니다.

인자를 통해 설정할 레지스터의 어드레스를 지정할 수 있습니다. 외부 모듈에서 레지스터 설정 시 사용됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	설정할 레지스터 이름
OP2	-	○	○	-	-	설정할 레지스터 번호
OP3	-	○	○	-	-	설정 값

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

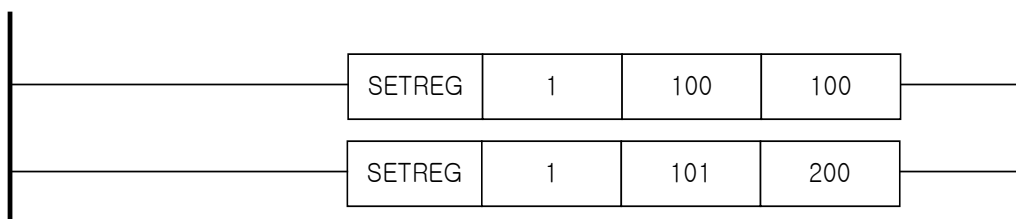


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	설정할 레지스터의 이름을 지정
Operand 2	설정할 레지스터의 번호를 지정
Operand 3	설정값을 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

P 레지스터(1)의 100번지에 100, 101번지에 200을 설정합니다.



5.12.4 GETREG (레지스터 읽기)

■ [기능]

지정된 레지스터의 값을 읽어 지정한 레지스터에 저장 합니다.

인자를 통해 읽어올 레지스터의 어드레스를 설정할 수 있습니다. 외부 모듈에서 레지스터 값을 읽을 경우 사용됩니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	-	○	○	-	-	읽을 레지스터 이름
OP2	-	○	○	-	-	읽을 레지스터 번호
OP3	-	○	-	-	-	결과 저장 레지스터

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]

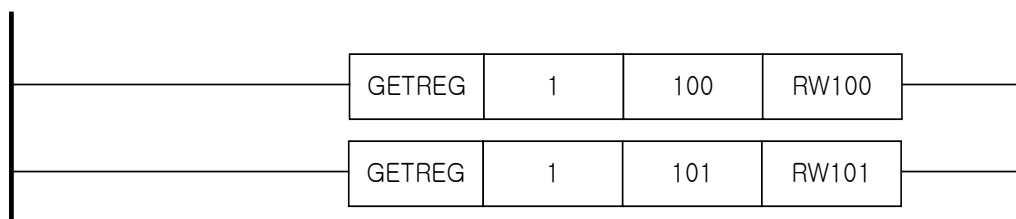


■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	값을 읽을 레지스터의 이름을 지정
Operand 2	값을 읽을 레지스터의 번호를 지정
Operand 3	데이터를 저장할 레지스터 지정
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

P 레지스터(1)의 100번지 값을 [RW200], 101번지 값을 [RW101]에 저장합니다.



5.12.5 SETEVT (사용자 이벤트)

■ [기능]

사용자 이벤트를 생성하여 설정/해제 할 수 있는 명령입니다.

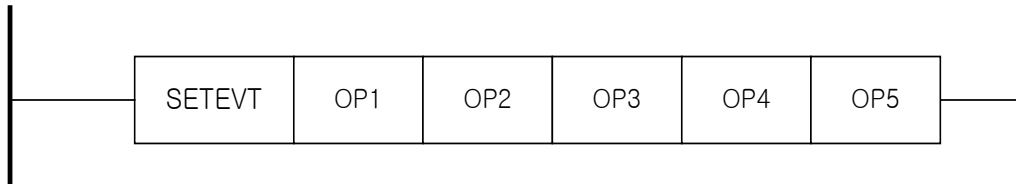
사용할 수 있는 사용자 이벤트 코드는 2000~2255번 까지 입니다.

■ [형식]

	B	W	CW	F	CF	내 용
OP1	○	-	-	-	-	사용자 이벤트 설정/해제
OP2	○	-	-	-	-	LED 표시 여부
OP3	-	○	○	-	-	사용자 이벤트 Code
OP4	-	○	○	-	-	사용자 이벤트 Data 1
OP5	-	○	○	-	-	사용자 이벤트 Data 2

* 명령 실행조건, 결과 출력 삽입 가능

■ [Ladder]



■ [설명]

실행 조건	명령 실행 조건을 삽입 가능
Operand 1	OP1 = 0 : 사용자 이벤트 해제 OP1 = 1 : 사용자 이벤트 설정
Operand 2	OP2 = 0 : 사용자 이벤트를 LED에 표시 안함 OP2 = 1 : 사용자 이벤트를 LED에 표시 함
Operand 3	사용자 이벤트 코드
Operand 4	사용자 이벤트 Data 1
Operand 5	사용자 이벤트 Data 2
실행 결과	명령이 정상적으로 수행되면 1(HIGH) 출력

■ [예제]

다음과 같은 사용자 이벤트를 생성하여 설정합니다.

[Code : 2000], [Data1:333], [Data2:350]

